

私有云环境下基于微服务架构的深度学习容器化平台设计与实现

作者姓名_____游海波_____

学校导师姓名、职称_____吴家骥 教授_____

企业导师姓名、职称_____尚 坤 研究员_____

申请学位类别_____电子信息硕士_____

学校代码 10701
分 类 号 TP39

学 号 20181213863
密 级 公开

西安电子科技大学

硕士学位论文

私有云环境下基于微服务架构的深度学习容器化平台设计与实现

作者姓名：游海波

领 域：新一代电子信息技术(含量子技术等)

学位类别：电子信息硕士

学校导师姓名、职称：吴家骥 教授

企业导师姓名、职称：尚 坤 研究员

学 院：广州研究院

提交日期：2023 年 6 月

Design and Implementation of Containerized Deep Learning Platform Based on Microservices Architecture in Private Cloud

A thesis submitted to
XIDIAN UNIVERSITY
in partial fulfillment of the requirements
for the degree of Master
in Electronic Information

By

Haibo You

Supervisor: Jiaji Wu

Title: Professor

Supervisor: Kun Shang

Title: Research Fellow

June 2023

摘要

随着以深度学习为代表的人工智能技术的不断发展,人工智能已经逐渐从相关从业研究者的理论技术研究步入到实际的工程应用中,融入到人类生活的方方面面,为人类的生活带来了极大的便利,这也表明深度学习算法被越来越多的业务场景所使用去解决现实问题。虽然深度学习经过多年的发展,已经形成了相对完善的深度学习开发训练流程,积累了大量的开源研究成果,但如何快速应用现有研究成果,进行可验证、可复用、可迭代的开发学习,对于一般的开发者仍然具有不小的困难,主要体现在以下几方面:(1)计算资源可访问性制约,计算资源成本高昂,缺少计算资源或计算资源无法进行有效管理,导致计算资源不能按需进行合理分配,产生资源浪费。(2)深度学习开发过程步骤繁琐、前期准备工作繁重。如:环境搭建困难,开发环境搭建步骤繁琐、维护困难,需要大量经验,对于初学者和非专业人员非常不友好;数据获取困难,训练数据昂贵、质量参差不齐等;模型部署困难,部署框架繁多;没有便捷的服务访问、监控和异常数据回收机制。(3)团队协作效率低下,项目管理流程混乱、成员权限缺少有效的管理等。

针对以上深度学习开发过程中的问题,本文主要面向中小型团队,在私有云环境下采用微服务架构的设计思想借助 Docker 容器化技术设计并实现深度学习平台。主要的工作内容如下:

(1)以微服务架构设计整个平台系统。平台最顶层业务层面划分为主机管理、镜像管理、数据集管理、项目管理、开发环境管理、模型管理以及模型托管七大业务模块,通过以 Docker 容器化的方式构建身份认证与授权服务、分布式对象存储服务、镜像存储与分发服务、基于对象存储的 Git 版本管理服务、系统监控服务以及 Java 后台核心服务,进而支撑业务模块的实现;通过 Kubernetes 集群调度部署模型,最终提供推理服务实现深度学习应用落地。

(2)细粒度的资源权限管理与授权机制。本文通过整合 Keycloak 认证与授权服务,结合深度学习开发场景,自行设计细粒度的资源权限管理与授权机制,实现对系统中不同类型资源的细粒度访问控制;将身份认证与授权服务从 Java 后台核心服务中抽离,以符合微服务架构的形式进行开发部署,通过 HTTP/REST 协议进行服务间通信,各业务模块直接调用服务接口实现资源权限管理。

(3)基于分布式对象存储的 Git 版本管理。以分布式对象存储为基础,将 Git 版本库中的所有文件对象存储在对象存储服务中,依托于 Git 实现库 JGit,结合对象存储的挂载机制,自行设计并实现支持 HTTP 协议的后端服务,以支持团队协作开发、高效的项目版本管理。

本文采用微服务架构设计并实现的深度学习容器化平台，主要面向中小团队，以 Docker 容器的方式为用户快速构建深度学习开发环境，形成一个从项目立项、数据资源共享及在线存储、开发环境构建、模型管理到项目落地实施的一站式深度学习开发平台。

关键词：微服务， 深度学习， Docker， 对象存储， Kubernetes

ABSTRACT

As the artificial intelligence technology represented by deep learning continues to develop, artificial intelligence has gradually entered practical engineering applications from the theoretical and technical research of relevant practitioners, and has been integrated into all aspects of human life, bringing great convenience to human life. This also indicates that deep learning algorithms are being used in more and more business scenarios to solve real-world problems. Although deep learning has developed a relatively complete development and training process over the years, and has accumulated a large number of open-source research results, how to quickly apply existing research results for verifiable, reusable, and iterative development learning still poses significant challenges for general developers, mainly manifested in the following aspects: (1) the accessibility constraints of computing resources, the high cost of computing resources, the lack of computing resources or the inability to effectively manage computing resources, resulting in the inability to reasonably allocate computing resources on demand, causing resource waste. (2) The deep learning development process is cumbersome, and the preparatory work is heavy. The environment setup is difficult, and the development environment setup steps are tedious and difficult to maintain, requiring a lot of experience, which is very unfriendly to beginners and non-professionals. Data acquisition is difficult, training data is expensive, and the quality is uneven, and model deployment is difficult. There are many deployment frameworks, and there is no convenient service access, monitoring, and abnormal data recovery mechanism. (3) Team collaboration efficiency is low, and project management processes are chaotic, and member permissions lack effective management, etc.

To address the above problems in the deep learning development process, this thesis mainly targets small and medium-sized teams and uses the design idea of microservices architecture in a private cloud environment to design and implement a deep learning platform with Docker containerization technology. The main work contents are as follows:

(1) Design the entire platform system using microservices architecture. The platform's top-level business layer is divided into seven major business modules: host management, image management, dataset management, project management, development environment management, model management, and model hosting. By building identity authentication

and authorization services, distributed object storage services, image storage and distribution services, Git version management services based on object storage, system monitoring services, and Java backend core services in a Docker containerized manner, it supports the implementation of business modules. The model is deployed through Kubernetes cluster scheduling to provide inference services to achieve the landing of deep learning applications.

(2) Fine-grained resource permission management and authorization mechanism. By integrating Keycloak authentication and authorization services and combining the deep learning development scenario, this article independently designs a fine-grained resource permission management and authorization mechanism to achieve fine-grained access control for different types of resources in the system. The identity authentication and authorization service is extracted from the Java backend core service and developed and deployed in a form that conforms to the microservices architecture, and the services communicate with each other through HTTP/REST protocol, and each business module directly calls the service interface to implement resource permission management.

(3) Git version management based on distributed object storage. Based on distributed object storage, all file objects in the Git version repository are stored in the object storage service. With the support of Git's library JGit and the mounting mechanism of object storage, this article independently designs and implements a backend service that supports HTTP protocol to support team collaboration development and efficient project version management.

The deep learning containerized platform designed and implemented using microservices architecture in this article is mainly aimed at small and medium-sized teams, and uses Docker containers to quickly build deep learning development environments for users, forming a set of from project initiation, data resource sharing and online storage, development environment construction, model management to project landing and implementation.

Keywords: microservices, deep learning, Docker, object storage, Kubernetes

插图索引

图 1.1 容器化技术发展历程	3
图 1.2 论文组织结构图	7
图 2.1 Docker 架构图	9
图 2.2 Docker 调用流程图	10
图 2.3 虚拟机与 Docker 技术对比	11
图 2.4 Kubernetes 集群架构图	12
图 2.5 kube-scheduler 调度流程	14
图 2.6 Ceph 架构图	16
图 2.7 Ceph 数据存储过程	17
图 2.8 Git HTTP 下载交互过程	19
图 3.1 深度学习平台层次架构	21
图 3.2 深度学习开发流程图	22
图 3.3 用户管理用例图	22
图 3.4 系统管理员用例图	23
图 3.5 数据集管理用例图	26
图 4.1 深度学习容器化平台系统架构图	29
图 4.2 深度学习容器化平台技术架构图	31
图 4.3 虚拟专用网络拓扑图	33
图 4.4 DNS 域名解析流程图	33
图 4.5 深度学习容器化平台系统功能模块图	34
图 4.6 深度学习容器化平台核心功能关系图	35
图 4.7 添加主机流程图	36
图 4.8 镜像发布流程图	37
图 4.9 数据集创建流程图	38
图 4.10 数据集 Bucket 存储结构图	39
图 4.11 项目默认文件结构	40
图 4.12 开发环境容器创建启动流程图	41
图 4.13 模型存储流程图	42
图 4.14 推理服务启动流程图	43
图 4.15 深度学习平台 E-R 图	44
图 5.1 微信扫码登录时序图	51

图 5.2 平台登录界面	51
图 5.3 身份认证相关类图	52
图 5.4 Resource 对象 JSON 模板样例	53
图 5.5 Role-based Policy 对象 JSON 模板样例	54
图 5.6 Permission 对象 JSON 模板样例	55
图 5.7 受保护资源实体创建流程图	56
图 5.8 Java 后台鉴权时序图	56
图 5.9 Ceph 对象存储相关类图	57
图 5.10 STS 授权及使用流程图	59
图 5.11 仓库存储示意图	60
图 5.12 版本管理相关类图	61
图 5.13 Push 操作第一次请求引用发现流程图	62
图 5.14 Push 操作第二次请求数据交互流程图	62
图 5.15 监控模块部分可视化界面	63
图 5.16 主机管理模块相关类图	64
图 5.17 主机选取流程图	65
图 5.18 主机管理界面	66
图 5.19 镜像管理模块相关类图	66
图 5.20 Docker Registry 认证流程图	67
图 5.21 镜像管理界面	67
图 5.22 数据集模块相关类图	68
图 5.23 开发环境管理相关类图	69
图 5.24 容器启动 Docker Compose 配置文件模板	70
图 5.25 开发环境生命周期	71
图 5.26 动态路由访问时序图	72
图 5.27 JupyterLab 开发环境页面	72
图 5.28 模型托管相关	73
图 5.29 推理服务接口调用界面	74
图 6.1 Docker 以及 Docker Compose 的安装 Shell 脚本	76
图 6.2 Docker 版本信息	77
图 6.3 k8s 集群组件运行状态	78
图 6.4 Ceph 集群关键组件运行状态	78
图 6.5 Ceph 集群管理界面	78
图 6.6 对象存储服务文件单次上传速率图	83

表格索引

表 2.1 Docker 容器与虚拟机对照表	11
表 2.2 对象存储、块存储和文件存储特点对照表	15
表 2.3 主流版本控制解决方案对照表	19
表 4.1 深度学习容器化平台服务器参数表	32
表 4.2 数据集角色权限表	37
表 4.3 项目角色权限表	40
表 4.4 模型角色权限表	42
表 4.5 项目表	46
表 4.6 开发环境容器表	47
表 5.1 Keycloak 核心 API 列表	50
表 5.2 Scope 列表	53
表 5.3 角色-Permission 对照表	55
表 5.4 Canned ACL 表	58
表 5.5 ACL 权限与 Action 映射关系表	58
表 6.1 系统软件环境	75
表 6.2 Kubernetes 集群组件列表	77
表 6.3 用户管理测试用例表	79
表 6.4 主机管理测试用例表	80
表 6.5 镜像管理测试用例表	80
表 6.6 项目资源管理测试用例表	81
表 6.7 开发环境管理测试用例表	82
表 6.8 模型托管测试用例表	82
表 6.9 存储性能测试结果	83
表 6.10 JMeter 线程参数配置表	84
表 6.11 数据获取接口性能测试结果	84

符号对照表

符号	符号名称
GB	吉字节
TB	太字节
EB	艾字节

缩略语对照表

缩略语	英文全称	中文对照
LXC	Linux Containers	Linux 容器
CNCF	Cloud Native Computing Foundation	云原生计算基金会
GKE	Google Kubernetes Engine	谷歌 kubernetes 引擎
EKS	Elastic Kubernetes Service	弹性 kubernetes 服务
AKS	Azure Kubernetes Service	Azure Kubernetes 服务
ACK	Alibaba Cloud Kubernetes	阿里云容器服务
CCE	Cloud Container Engine	云容器引擎
AI	Artificial Intelligence	人工智能
Amazon S3	Amazon Simple Storage Service	亚马逊简易存储服务
DI-X	Data Intelligence X	数据智能 X
COS	Cloud Object Storage	云对象存储
PAI	Platform of Artificial Intelligence	人工智能平台
OSS	Object Storage Service	对象存储服务
OCI	Open Container Initiative	开放容器计划
OS	Operating System	操作系统
DNS	Domain Name System	域名系统
RBD	RADOS Block Device	RADOS 块设备
Ceph OSD	Ceph Object Storage Device	Ceph 对象存储设备
Ceph MDS	Ceph Metadata Server	Ceph 元数据服务
PG	placement groups	归置组
SSD	Solid State Disk	固态硬盘
HDD	Hard Disk Drive	硬盘驱动器
SSH	Secure Shell Protocol	安全外壳协议
RBAC	Role Based Access Control	基于角色的访问控制
UBAC	User Based Access Control	基于用户的访问控制
ABAC	Attribute Based Access Control	基于属性的访问控制
ACL	Access control list	访问控制列表
AWS STS	AWS Security Token Service	AWS 安全令牌服务
MLOps	Machine Learning Operations	机器学习操作

目 录

第一章 绪论.....	1
1.1 研究背景及意义.....	1
1.2 国内外研究现状.....	2
1.2.1 容器化技术.....	2
1.2.2 深度学习云平台.....	4
1.3 论文主要工作内容.....	6
1.4 论文组织结构.....	7
第二章 深度学习平台相关技术概述	9
2.1 Docker 容器技术.....	9
2.2 Kubernetes 技术	11
2.2.1 Kubernetes 架构及核心组件	12
2.2.2 Kubernetes 调度策略	13
2.3 存储服务.....	14
2.3.1 对象存储.....	15
2.3.2 Ceph 分布式对象存储	16
2.4 Keycloak	17
2.5 Git.....	18
2.6 本章小结.....	19
第三章 深度学习平台系统需求分析	21
3.1 系统概述.....	21
3.2 系统功能性需求分析.....	22
3.2.1 用户管理需求分析.....	22
3.2.2 存储功能需求分析.....	23
3.2.3 主机管理需求分析.....	24
3.2.4 镜像管理需求分析.....	25
3.2.5 数据集管理需求分析.....	25
3.2.6 项目及开发环境管理需求分析.....	26
3.2.7 模型管理及模型托管需求分析.....	26
3.2.8 系统监控模块需求分析.....	27
3.3 系统非功能性需求分析.....	27
3.3.1 高可用性.....	27
3.3.2 安全性.....	27
3.3.3 易用性.....	28
3.4 本章小结.....	28

第四章 深度学习平台系统整体设计	29
4.1 系统架构设计	29
4.2 系统私有云环境搭建	31
4.3 系统功能设计	34
4.3.1 用户管理模块设计	35
4.3.2 主机管理模块设计	35
4.3.3 镜像管理模块设计	36
4.3.4 数据集管理模块设计	37
4.3.5 项目管理模块设计	39
4.3.6 开发环境管理模块设计	40
4.3.7 模型管理模块设计	42
4.3.8 模型托管模块设计	43
4.4 数据库设计	43
4.4.1 数据库实体关系	44
4.4.2 数据库表设计	44
4.5 本章小结	47
第五章 深度学习平台系统实现	49
5.1 系统基础功能实现	49
5.1.1 基于 Keycloak 的用户管理	49
5.1.2 基于 Ceph 的分布式对象存储	56
5.1.3 基于分布式对象存储的 Git 版本管理	60
5.1.4 监控模块实现	62
5.2 系统业务功能实现	63
5.2.1 主机管理模块	63
5.2.2 镜像管理模块	66
5.2.3 资源管理模块	67
5.2.4 开发环境管理模块	69
5.2.5 模型托管模块	73
5.3 本章小结	74
第六章 系统部署与测试	75
6.1 系统部署	75
6.1.1 系统软件环境	75
6.1.2 Docker 部署	76
6.1.3 Kubernetes 集群部署	77
6.1.4 Ceph 集群部署	78
6.2 系统功能测试	79
6.2.1 用户管理模块测试	79
6.2.2 主机管理模块测试	79

6.2.3 镜像管理模块测试.....	80
6.2.4 资源管理模块测试.....	81
6.2.5 开发环境管理模块测试.....	81
6.2.6 模型托管模块测试.....	82
6.3 系统性能测试.....	82
6.3.1 对象存储性能测试.....	82
6.3.2 接口压力测试.....	84
6.4 本章小结.....	84
第七章 总结与展望	87
7.1 工作总结.....	87
7.2 工作展望.....	87
参考文献.....	89

第一章 绪论

1.1 研究背景及意义

近年来,以深度学习为代表的人工智能浪潮汹涌澎湃,对当今社会的生产生活、公共服务乃至全球的竞争格局等都产生了广泛且深远的影响^[1]。随着产业变革和新科技革命的到来,人工智能技术和产业之间的融合愈发深入,这种融合正在推动新兴产业之间以及新兴产业与传统产业之间的跨界合作和发展。作为引领未来时代发展的新兴战略性技术,人工智能已然成为推动产业变革和科技革命的重要力量^[2]。习近平总书记多次作出重要指示,强调“要深入把握新一代人工智能发展的特点,加强人工智能和产业发展的融合,为高质量发展提供新动能”^[3]。

世界各国不断强化人工智能的战略地位,自2016年起,先后有40多个国家和地区将推动和促进人工智能的发展上升到国家战略的高度^[4,5]。我国在《中共中央关于制定国民经济和社会发展第十四个五年规划和2035远景目标纲要的建议》中指出,聚焦人工智能等高新前沿领域,开展一批具有前瞻性的重大战略科技项目,推动我国数字经济快速健康发展^[6]。美国成立了国家AI研究资源工作组等多家机构,陆续发布了一系列政策,将人工智能提高到“未来产业”和“未来技术”的高度,不断巩固自己在人工智能等高科技领域的地位以及全球竞争力。欧盟发布《2030数字化指南:欧洲数字十年》等指南,力求全面重塑欧洲在数字时代的全球影响力,其中将推动人工智能的发展列为重要工作^[7]。

虽然各国对以深度学习为代表的人工智能给予了高度的重视,并纷纷将人工智能提高到国家战略的高度,同时给予了大量的政策支持和资金投入,但相关从业人员在实际的深度学习开发过程中人面临不小的困难。如:

(1)深度学习开发对开发者所使用的服务器硬件要求高,需要大量的计算资源,然而计算资源价格昂贵,往往需要多人同时使用一台高性能计算服务器,这就会造成计算资源分配不合理,产生严重的资源竞争问题,计算资源利用率低;

(2)深度学习开发过程步骤繁琐,从环境搭建与维护、数据资源管理、服务部署到服务管理等对于初学者来说都是不小的挑战。深度学习开发环境搭建流程复杂繁琐,即便是资深从业者在搭建过程中也会由于服务器环境、软件版本等问题在搭建环境过程中产生一系列的问题,导致开发环境搭建耗费大量的时间精力;深度学习开发前期需要进行大量调研,查看大量文献资料,查找或重新标注符合要求的数据集等,由于各个企业和个人资源不共享,资源相对封闭,从业人员进行了大量的重复性工作,难以获取有效的资源,具有较高的学习和准入门槛;数据安全问题,对于一些涉密或

私密性要求较高的数据存储在各大云服务商提供的公有云存储服务中是非常不现实的，存储在本地文件系统中又存在数据管理困难，一旦本地文件系统受损则会产生毁灭性打击等；模型部署困难、应用管理和后期维护复杂繁琐，深度学习应用服务没有便捷的服务访问、状态监控和异常处理机制；

(3) 一个大型的深度学习项目往往都是由一个团队进行开发，团队成员间能否实现高效的分工协作，成员间项目开发是否采用相同的依赖或软件版本，项目文件能否实现有效的版本管理，成员权限能否进行有效的划分等都会对项目的开发效率产生重要的影响。

针对以上深度学习开发应用过程中所遇到的问题，将 Docker 及 Kubernetes 技术与深度学习环境构建和应用部署等环节相结合，解决开发过程中开发环境构建、服务器资源调度以及资源隔离等问题。Docker 作为一种在操作系统层面实现的虚拟化技术，与传统的虚拟化技术相比，Docker 具有高效的资源利用率、快速的启动时间、一致的运行环境、进程和资源隔离等特点^[8,9]。基于这些特点，可以轻松地为开发者搭建开发环境，也可以快速实现开发环境的迁移，同时确保环境隔离。针对深度学习开发过程中，不同用户对于开发环境具有不同的要求，结合 Docker 镜像自定义构建功能，基于深度学习当前主流框架，例如：PyTorch、Tensorflow 等构建符合开发者要求的基础镜像，满足开发者需求，提高开发效率。Kubernetes 作为当前最流行的容器编排工具，它能够在物理机或虚拟机集群上轻松调度和运行容器，有效管理集群中的容器，提高集群资源利用率^[10]。

本文深入了解 Docker 和 Kubernetes 的特性，并结合深度学习开发过程中的所遇到关键问题，在私有云环境下，搭建一个集项目管理、数据存储、开发环境一键搭建、模型管理、模型快速部署的一站式深度学习平台。

1.2 国内外研究现状

1.2.1 容器化技术

随着全球数字化浪潮的逐步深入，以容器技术、微服务、服务网格等为代表的云原生技术得到广泛应用，云原生的作用也逐渐增强，并成为推动数字化浪潮下业务创新的重要动力^[11,12]。云原生技术便于各个组织机构在公有云、私有云以及混合云等不同新型动态环境下，自动构建并运行可弹性扩展的应用服务^[13]。利用以容器技术为代表的云原生技术来构建私有云平台，可以显著提高私有云平台中的资源的利用率、服务的效率和可用性。容器技术和云原生犹如一对双螺旋体，容器技术的出现催生了云原生思潮，云原生思潮则又推动了容器技术的不断发展。容器技术所要解决的核心问题之一就是运行时环境隔离，通过运行时环境隔离给不同容器构造一个互不干扰的无

差别运行时环境，使容器可以在任意时间任意位置构建运行。

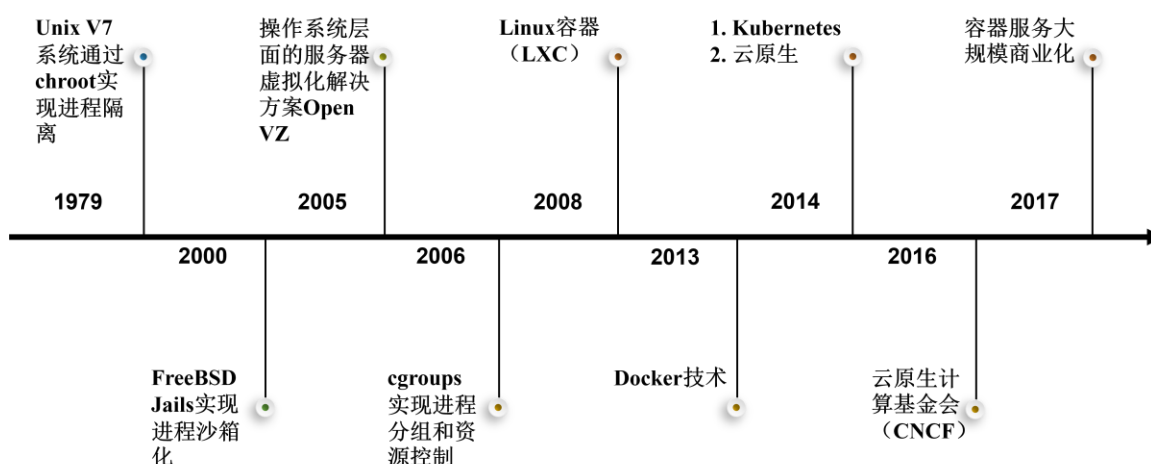


图1.1 容器化技术发展历程

容器技术的发展开端最早可追溯到二十世纪七十年代，如图 1.1 所示。1979 年 Unix V7 系统引入了 chroot^[14]系统调用，为不同进程提供一个隔离的虚拟文件系统，这标志着进程隔离的开始，chroot 也被认为是容器技术的鼻祖。2000 年，操作系统 FreeBSD 4.0 通过 FreeBSD jails 隔离环境，实现了真正意义上的进程沙箱化^[15,16]。2001 年 Linux VServer 操作系统虚拟化机制出现，他通过修补 Linux 内核来实现，他可以对操作系统上的资源进行隔离划分。Oracle 于 2004 年发布的 Oracle Solaris Containers 采用了 zone 技术，通过此技术在操作系统中隔离出具有独自进程空间、文件系统和网络配置等的完全隔离的虚拟服务器^[17]。2005 年，SWsoft 推出了基于 Linux 的操作系统层面的服务器虚拟化解决方案 Open VZ，它是 Linux OS 虚拟化技术的先行者^[18]。Process Containers 由 Google 于 2006 年发布，用于控制容器中 CPU、内存、I/O 等资源的配额，随后更名为 cgroups，并于 2008 年整合至 Linux 内核中^[19]。2008 年，集众家之所长的 Linux Container（LXC）^[20]横空出世，它是第一个完整的 Linux 容器管理器实现方案，依托于 namespace 的资源隔离能力和 cgroups 的资源控制能力^[21]。

2013 年随着 Docker 的出现，容器技术的发展迎来了重要的转折点，从萌芽期转入迸发期，容器技术开始席卷全球。Docker 形成了完整的容器管理生态系统，具有中央和本地容器注册库、分层高效的容器镜像模型、命令行、清晰便捷的 REST API 等，随着 Docker 技术的不断发展，其实现方式也逐渐由 LXC 过渡到自行开发的 libcontainer，最后演变为使用 runC 和 containerd^[22]。虽然 Docker 极大的促进了容器技术的发展，但在工业化应用过程中依然存在诸如网络问题、负载均衡问题、监控等问题，2014 年 Kubernetes 容器编排应运而生，容器技术、微服务、容器编排三者相辅相成，云原生的概念也在此被提出。2016 年，Google、Redhat、Microsoft 等巨头成立了 Cloud Native Computing Foundation（CNCF）组织，意在推动云原生生态的建设。

2017年后,容器服务商业化,基于 Docker 和 Kubernetes 的云平台如雨后春笋般出现,如国外 Google 的 GKS、Amazon 的 EKS、Microsoft 的 AKS,国内公司 Alibaba 的 ACK、华为的 CCE 等。

1.2.2 深度学习云平台

虽然以容器技术为代表的云原生技术得到快速发展和商用化,基于 Docker 和 Kubernetes 的云平台服务众多,但都不是专用于深度学习开发学习的云平台,只是提供了深度学习应用部署管理的能力。为了满足相关从业人员对深度学习开发及应用需求,提供一套完整的深度学习开发生产迭代平台,国内外各大厂商抓住机遇相继推出了各具特色的深度学习云平台。

Google 于 2021 年全面上线 Vertex AI^[23]全托管式机器学习平台,它是一个包含深度学习开发所需工具的单一平台,集成了 TensorFlow、PyTorch、XGBoost 等常见的机器学习框架,允许开发者管理数据、蓝本、实验、部署模型、解析模型并在生产中对部署的应用进行监控。它依赖于 Google Cloud,对于一些存在特殊要求的数据需要考虑数据隐私和安全问题。

Microsoft 在自己公有云 Azure 上推出了基于 Web 的深度学习平台 Azure Machine Learning,它意在加速和管理机器学习项目的生命周期,简化开发和部署,以加速开发-测试循环,帮助开发者快速进行训练、模型部署和应用管理,它支持在 Azure Machine Learning 中创建模型,也支持使用从开源平台上构建的模型,例如:PyTorch、TensorFlow 或 scikit-learn,通过 MLOps 工具监视、重新训练和重新部署模型等。

Amazon 也有自己的深度学习平台 Amazon SageMaker,它是一个完全托管的机器学习服务,通过集成 Jupyter 为开发者提供便捷的数据访问、算法开发等功能,它可以直接连接到存储在 Amazon S3 中的数据,也支持通过 AWS Glue 将数据从 Amazon RDS、Amazon DynamoDB 和 Amazon Redshift 移动到 S3 以在 Notebook 中进行分析和使用,它预装并优化了目前最常用的机器学习算法,同时预先配置并运行 TensorFlow、PyTorch 和 Apache MXNet 开源框架;但是它与 python SDK 和 AWS cloud 绑定太深,易用性不好。

IBM 公司推出了 IBM Watson Studio,力求确保分析人员和开发者能够在 IBM Cloud Park for Data 上随时随地构建、运行、管理 AI 模型并优化决策,实现深度学习开发生命周期自动化,加快应用价值实现,它将 PyTorch、TensorFlow 和 scikit-learn 等开源框架与自身生态系统工具结合,支持 Jupyter Notebook、JupyterLab 等开发环境,支持 Python、R 和 Scala 等语言。

国内各大厂商也推出了具有自身特点的深度学习云平台,用于简化、加速深度学习开发生命周期。

腾讯的 TencentCloud TI Platform 基于腾讯云强大的大数据存储和处理能力,支持 Tensorflow、Torch、PySpark 等主流训练框架,并支持一机多卡、多机多卡模式的 GPU 分布式计算,通过简便易操作的可视化拖拽布局,用户可以快速组合不同的数据源、深度学习算法、模型和评估模块,可以实现无需大量编码即可完成复杂的机器学习任务;同时它可以无缝衔接腾讯云的其他服务资源,如:对象存储(COS)、计算(GPU 计算平台)。

阿里云也构建了基于云原生的轻量化、高性价比机器学习平台 Platform of Artificial Intelligence (PAI),它涵盖了从交互式建模、拖拽式可视化建模、分布式训练到模型在线部署的机器学习开发全流程服务。PAI 底层整合并支持多种计算框架,例如:流式计算框架 Flink、深度优化的 TensorFlow、大规模计算框架 Parameter Server、Spark 等业内主流开源框架,支持分布式训练、基于 Notebook 的开发环境、可视化自动化建模,基于阿里云强大的生态环境,衔接大数据开发治理平台 DataWorks,同时可将数据存储至 OSS 或 MaxCompute 中。

百度推出了自研的 PaddlePaddle 深度学习平台,集成了深度学习核心框架,同时提供基础模型库、端到端开发套件和各种深度学习工具组件,它基于自研的 Paddle 框架,提供支持低代码开发的高阶 API,支持超大规模的分布式训练和不同组合形式的加速策略,支持 Python 和 C++ 语言。

ModelArts 作为由华为云推出一站式 AI 平台,它为深度学习开发者提供交互式半自动化辅助标注、海量的数据预处理、高效的算法开发和分布式训练、模型自动化生成以及端-边-云模型部署等功能,支持 TensorFlow、PyTorch、MindSpore 等主流框架;为了进一步提升算法开发的效率和模型训练的速度,ModelArts 采用自行研发的构建于主流深度学习框架 TensorFlow、PyTorch 和 Keras 之上的分布式训练加速框架 MoXing;另外,借助华为海思 Ascend 芯片实现高效边端推理。

国内七牛云自主研发了分布式深度学习平台 AVA,它支持 JupyterLab 开发环境,支持 Caffe 和 MXNet 等深度学习框架,但暂不支持 TensorFlow 和 PyTorch 深度学习框架,支持集成 LabelX 数据标注系统数据集管理、模型在线训练等功能,但暂不支持在线模型部署功能。

通过对以上国内外主流的深度学习平台的分析及调研,各大云服务厂商及大型科技公司提供的深度学习云平台服务普遍存在服务收费价格昂贵,对于中小型团队和个人不友好,且存在产品技术闭源对特定平台的依赖程度过高,无法直接迁移到其他平台或本地环境,部分平台功能不完善无法真正实现一站式深度学习开发部署流程;对于国外厂商提供的深度学习云平台国内用户在访问使用上也存在诸多不便无法直接访问或网络环境不佳;另外,不支持私有化部署,对于一些涉密或私密性要求高的数据存储存在公有云环境中存在重大隐患。本文在私有云环境下采用 Docker、

Kubernetes 等技术设计实现一个基于微服务架构的深度学习平台，为中小型团队或高校师生提供一个一站式深度学习开发生产迭代平台。

1.3 论文主要工作内容

本文的主要工作是面向中小型团队和个人，针对当前深度学习开发过程中所存在的痛点，如：环境搭建困难，开发环境部署繁琐维护困难，计算、存储资源管理分配不合理等，围绕项目管理-数据集管理-模型训练-模型部署等环节，在私有云环境下基于微服务架构设计实现一个一站式深度学习平台，为用户提供计算资源统一调配管理和安全可靠的数据存储服务，支持 PyTorch、TensorFlow 主流深度学习框架，帮助开发者快速搭建深度学习开发环境，提供可视化的 Web 操作页面，提高用户的开发效率，减少深度学习开发项目生命周期，加快项目落地上线。综上所述，本文的主要工作和研究内容如下：

（1）以微服务架构设计整个平台系统

针对深度学习开发、训练、部署流程，将整个系统划分为基础功能模块和业务功能模块，基础功能模块包括用户认证和授权、数据存储及版本管理、系统监控等，上层业务模块为了满足传统开发流程可划分为主机管理、镜像管理、数据集管理、项目管理、开发环境管理、模型管理以及模型托管七大业务模块；系统底层服务通过 Docker 容器化技术分别构建部署身份认证与授权服务、分布式对象存储服务、镜像存储与分发服务、基于对象存储的 Git 版本管理服务、系统监控服务以及 Java 后台核心服务，以支持上层功能模块的实现。将整个平台系统拆分为小型服务单元，采用微服务架构设计系统，可以增加整个平台的灵活性、可靠性、可维护性和技术异构性。

主机管理模块通过管理部署开发环境容器的宿主机，合理调度部署容器，提高计算资源的使用效率，解决计算资源分配不均衡、利用率低的问题；项目管理、数据集管理、模型管理模块依托于分布式对象存储或基于分布式对象存储的 Git 版本管理服务以及自行设计的访问控制机制，管理不同类型的数据资源，解决数据壁垒和数据版本管理等问题；开发环境管理模块依托于镜像管理模块提供的集成不同深度学习框架的 JupyterLab 镜像快速构建并管理开发环境镜像，解决深度学习开发环境搭建困难的问题；模型托管模块则基于 Kubernetes 集群一键部署模型，对外提供推理服务，简化深度学习模型部署上线步骤。

（2）细粒度的资源权限管理与授权机制

针对不同资源的成员权限管理、确保团队的资源和信息安全问题，本文通过整合 Keycloak 用户认证和授权服务，结合不同场景，采用不同的访问控制机制，自行设计并实现了细粒度的资源权限管理与授权机制，以针对不同资源、模块或操作给不同用

户设置不同的权限；与传统的后台服务相比，本文将认证与授权服务从 Java 后台核心业务中抽离，通过 HTTP/REST 协议实现后台服务与认证授权服务通信，进行用户身份认证和授权，更加贴合微服务架构的设计思想。

（3）基于分布式对象存储的 Git 版本管理

针对资源存储与访问、资源版本管理以及团队协作等问题，本文以分布式对象存储为载体，将 Git 版本库中的所有文件以对象的形式存储在分布式对象存储服务中，为了实现资源 Git 版本库服务端与客户的数据交互，依托于 Git 实现库 JGit，结合对象存储的挂载机制，自行设计并实现支持 HTTP 协议的后端服务，以支持团队协作开发、资源版本的高效管理。虽然分布式对象存储服务支持数据文件的版本管理，但这种版本管理更适合管理单文件数据，对于具有多个文件的资源进行版本管理十分不便，因此这里引入 Git 版本管理。

1.4 论文组织结构

本论文的组织结构将分为七个章节，每章节的安排如图 1.2 所示。

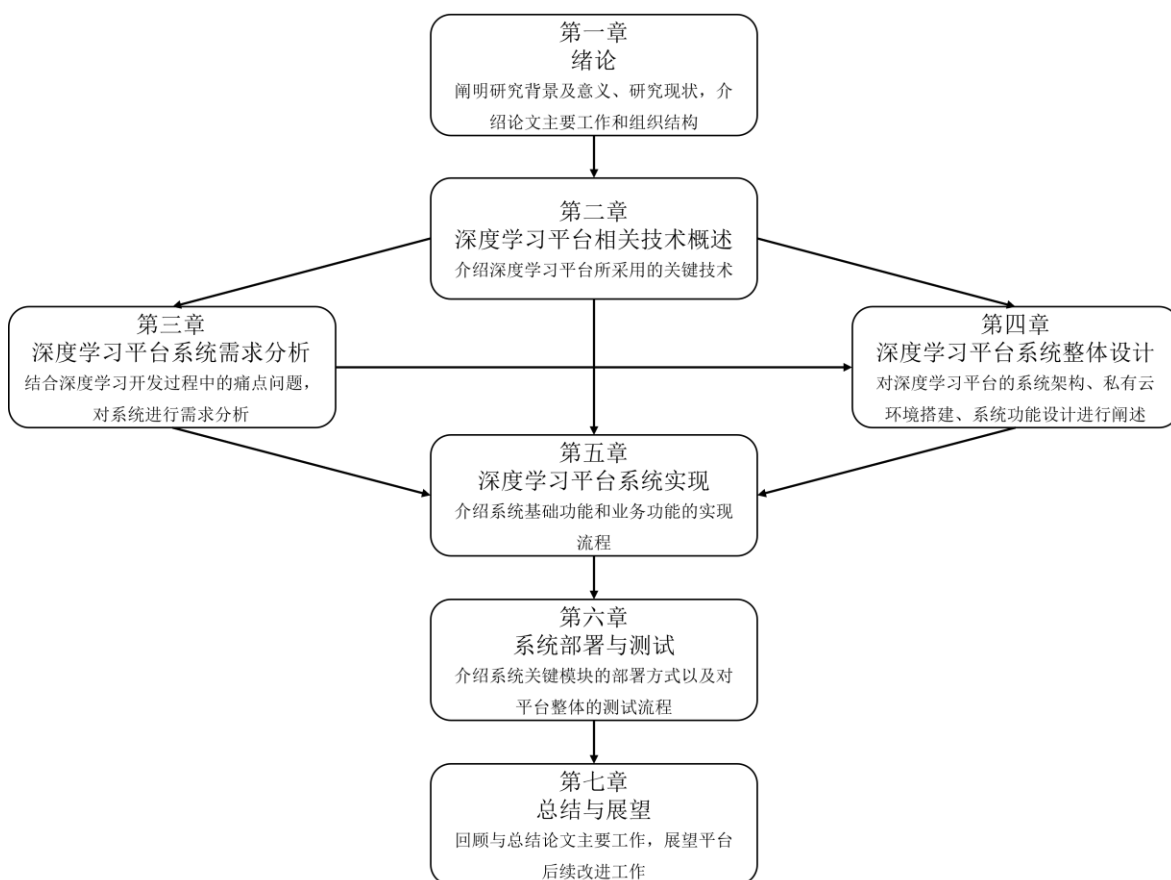


图1.2 论文组织结构图

第一章：绪论。本章主要介绍设计与实现私有云环境下基于微服务架构的深度学习

习容器化平台的研发背景及意义，总结关键技术以及当前国内外深度学习平台的研究发展现状，并系统性的介绍了本文的主要工作内容和组织结构。

第二章：相关技术概述。本章主要介绍了深度学习平台设计与开发过程中所采用的相关核心技术，核心技术包括：Docker 容器技术、Kuberbetes、Ceph 分布式对象存储、Keycloak 认证与授权解决方案、Git 分布式版本管理。

第三章：深度学习平台系统需求分析。本章主要结合深度学习开发过程中的痛点、项目研究背景以及采用的相关技术，针对私有云环境下基于微服务架构的深度学习容器化平台分别从系统功能性需求和非功能性需求进行分析。

第四章：深度学习平台系统整体设计。本章主要基于相关技术选型和系统需求进行系统整体设计，包括系统整体架构、私有云环境搭建、系统功能设计等；系统功能部分结合深度学习开发、训练和部署等流程将功能细化为主机、镜像、数据集、项目、开发环境、模型及模型托管等功能模块。

第五章：深度学习平台系统实现。本章主要包括系统基础功能实现和系统业务功能实现两部分，其中系统基础功能包括用户认证与授权模块、基于 Ceph 的分布式对象存储、基于分布式对象存储的 Git 版本管理和系统监控，业务功能则基于设计部分实现直接面向用户的功能模块。

第六章：系统部署与测试。本章主要介绍了私有云环境下基于微服务架构的深度学习容器化平台关键服务的部署以及对平台进行整体测试。

第七章：总结与展望。本章主要回顾与总结了论文的研究工作，探讨平台存在的不足之处，并列举之后可改进部分。

第二章 深度学习平台相关技术概述

根据绪论中所介绍的深度学习容器化平台的研究背景和意义,以及国内外的研究现状等,本章主要从基于微服务架构的深度学习容器化平台设计与实现过程中所使用的关键技术展开介绍,了解相关技术的原理和架构,进而分析其特点。相关技术如下: Docker 容器技术、Kubernetes、对象存储、Keycloak 认证与授权、Git 版本管理。

2.1 Docker 容器技术

随着云原生技术的发展,容器技术的使用越来越广泛,越来越多的企业采用容器作为新的 IT 基础设施。基础设施架构总是伴随着软件架构而演变,随着应用业务复杂度的不断飙升,单个功能模块也随之越来越复杂和庞大,以往的单体架构不再能满足开发部署的要求。因此,具有业务功能解耦、不同模块可以并行开发和部署特点的微服务架构逐渐取代原有架构,得到快速发展。虚拟化技术克服了物理资源隔阂壁垒,提升了资源的利用效率,将用户从管理基础架构的工作负担中抽离。而容器平台则进一步抽象,为应用提供服务依赖、运行环境以及底层所需的计算和存储资源,从而极大地提升了应用开发、部署、运维效率。

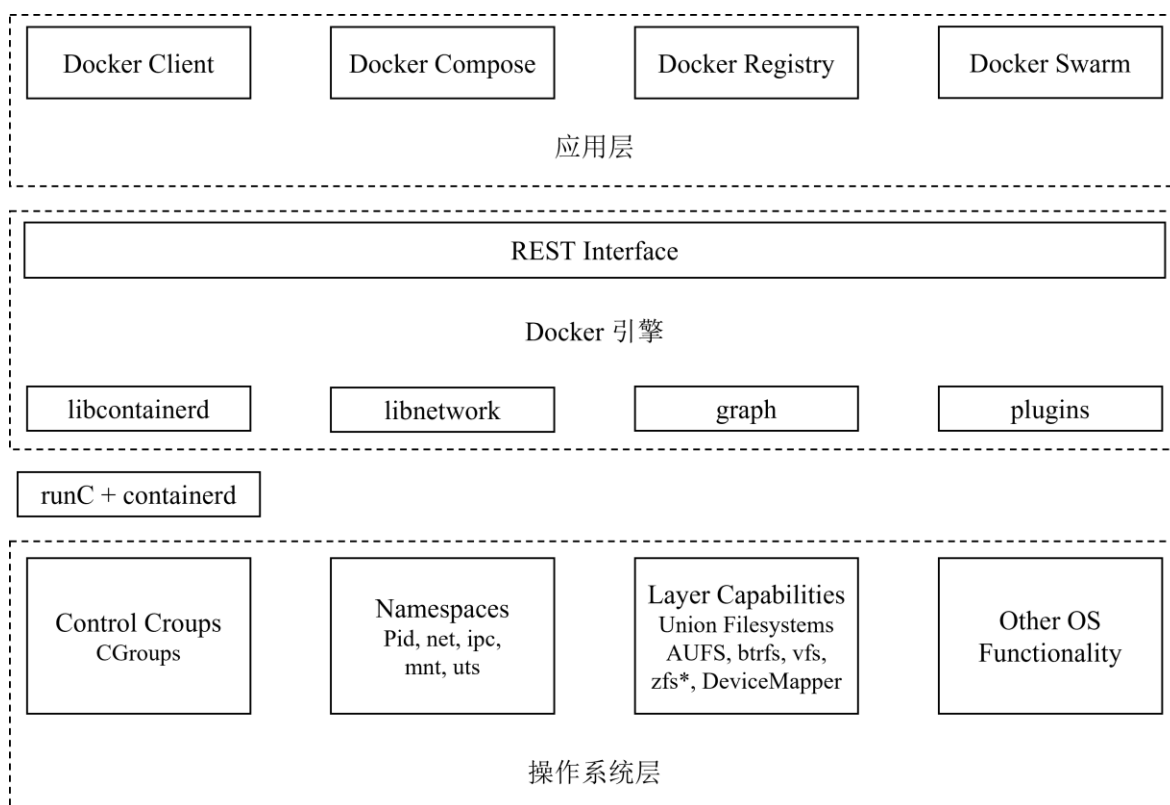


图2.1 Docker 架构图

Docker 基于 Linux 内核的 cgroup、namespace 以及 Layer Capabilities 等技术使用 Go 语言进行开发和实现，对进程进行封装隔离。Docker 架构如图 2.1 所示，runC 为轻量级的 Linux 命令行工具，基于 Open Container Initiative (OCI) 的标准，在指定的 namespace 和 cgroup 中构建并运行容器。Containerd 为守护进程，用于管理容器生命周期。Docker 通过 namespace 实现容器的资源和进程隔离，使容器具有自己独立的系统空间，避免不同容器间进程和资源相互干扰；通过 cgroup 控制进程的资源访问。Docker 引擎通过 libcontainerd 核心库访问 Linux 中的 namespace 和 cgroup，通过 libnetwork 网络插件管理 Docker 容器的网络，实现容器与不同目标实体的通信。

Docker 采用客户端-服务端架构，如图 2.2 所示。用户通过 Docker 客户端将消息指令发送给 Docker 守护进程，通过 Docker 守护进程来实现 Docker 容器的构建、运行和分发工作。Docker 客户端与 Docker 守护进程之间的通信则是在 UNIX 套接字和网络接口上使用 REST API 实现。Docker 守护进程的作用主要包括三部分：一是负责监听 Docker API 请求；二是管理 Docker 镜像、容器、网络和卷等对象；三是与其他守护进程通信进而管理 Docker 服务。Docker 注册表负责存储和分发 Docker 镜像。

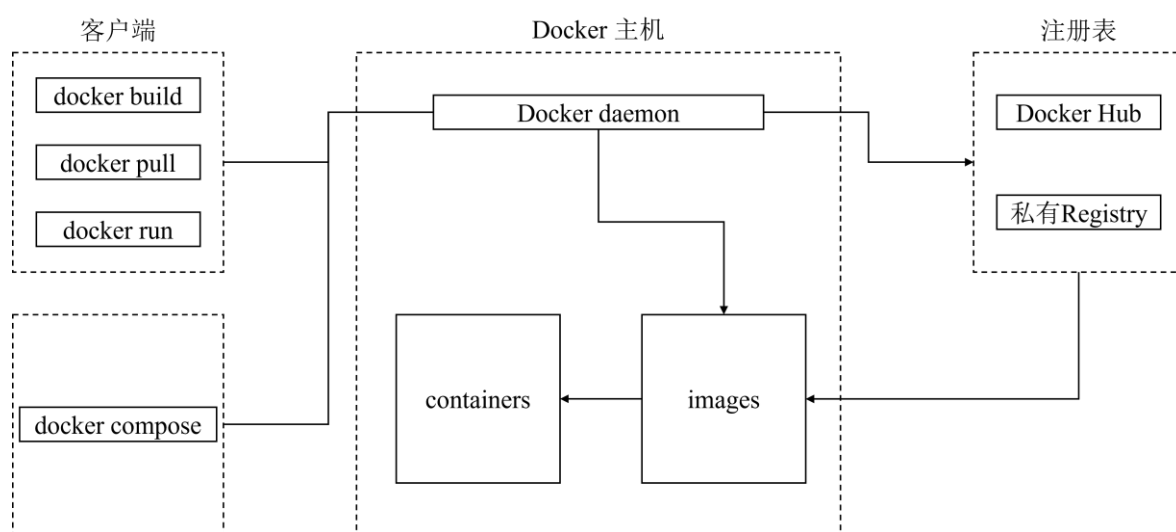


图2.2 Docker 调用流程图

传统的虚拟机技术是在物理资源层面实现的隔离，首先虚拟机的 Hypervisor 对硬件资源进行虚拟化，虚拟出一套硬件并在其上运行一个完整的操作系统，然后再运行所部署的应用程序，虚拟机技术比 Docker 技术多了一层 Guest OS 层^[24]。Docker 技术是面向操作系统层面的虚拟化技术，它直接将本身的应用运行于宿主机的内核上，无需向虚拟机一样需要安装从操作系统，因此可以节省大量的系统资源，如图 2.3 所示。

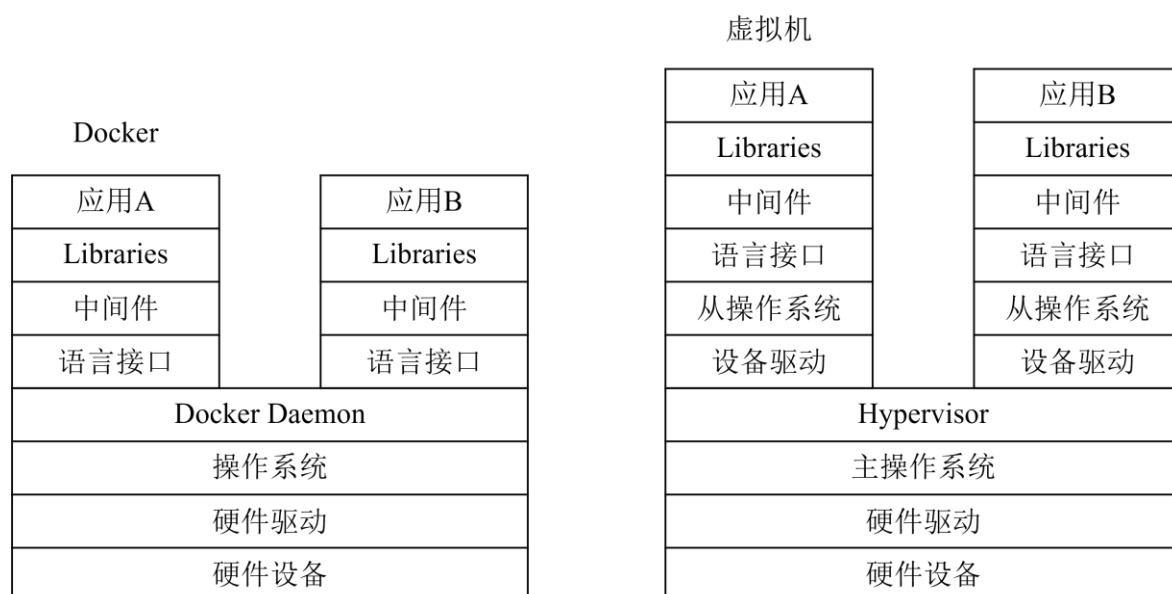


图2.3 虚拟机与 Docker 技术对比

Docker 作为新兴的虚拟化技术，具有传统的虚拟化技术无可比拟的优势，如：接近原生的系统性能、更快的启动时间、更高效的资源利用率、一致的运行环境、更加易于维护、扩展以及应用迁移。表 2.1 对 Docker 与虚拟机技术部分特性进行对比。

表2.1 Docker 容器与虚拟机对照表

特性	Docker 容器	虚拟机
启动时间	秒级	分钟级
硬盘使用	一般为 MB	一般为 GB
性能	接近原生	弱于
系统支持量	支持部署上千个容器	一般为几十个

2.2 Kubernetes 技术

Kubernetes 也称 k8s，是一个容器编排平台，它可以自动化完成容器化应用的部署、管理和扩展等过程，提供了一个可弹性运行的分布式系统框架^[25]。Kubernetes 并不是传统的 PaaS 系统，它面向容器级别运行管理应用，它不仅包含传统 PaaS 系统的共有特性，还具有可插拔的特性，可以为构建开发平台提供基础，同时在重要的地方保留了用户选择权，具有高灵活性，广泛应用于公有云、私有云、混合云等应用场景中。Kubernetes 具有如下特点^[26]：

(1) 服务发现和负载均衡：它可以使用自身部署的 DNS 服务，解析分配给每个服务的 DNS 名称，从而暴露服务的虚拟 IP 地址；若进入某个容器的流量过大，可通过负载均衡分配网络流量；

(2) 自动部署和回滚：它可以自动化部署创建新容器，删除现有容器并将其资源应用于新容器，以受控制的方式将容器实际状态更改为期望状态；

(3) 存储编排：允许挂载不同的存储系统，包括本地存储系统、云存储服务等；

(4) 自动封箱计算：可以根据 Pod 需求按实际情况自动调度到合适节点部署容器，以最佳方式最大化利用节点资源；

(5) 自我修复：自动检测 Pod 的运行状态，重启失败的 Pod，节点修复将故障节点的 Pod 迁移到健康节点。

2.2.1 Kubernetes 架构及核心组件

Kubernetes 架构如图 2.4 所示，Kubernetes 集群包含节点代理 Kubelet 和 Master 组件，每个 Kubernetes 集群至少包含一个工作节点，但每个集群一般会运行多个节点，用于提供容错性和高可用性。Pod 是 Kubernetes 集群中用于运行和部署应用服务的最小单元，它包含一个或多个容器，并且允许自身内部容器共享网络地址和文件系统^[27]。Pod 作为应用负载组件托管于工作节点之上，控制平面则统一管理集群中的所有工作节点和 Pod。

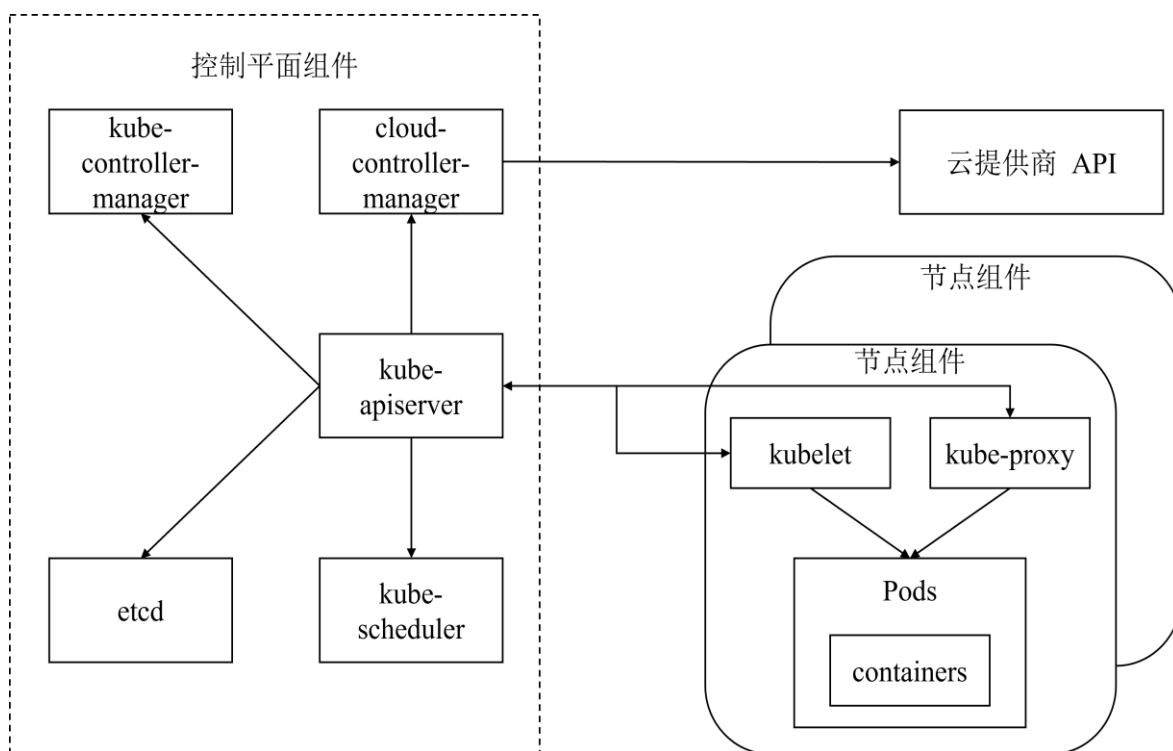


图2.4 Kubernetes 集群架构图

Kubernetes 集群组件主要分为两部分：控制平面组件和节点组件。

控制平面组件主要的作用是为集群做出全局决策，合理调度集群资源，检测和响应集群事件等，它可以运行于集群的任意节点上，但为了方便起见一般会将所有控制

平面组件运行于同一台计算机。控制平面组件如下^[28]：

(1) kube-apiserver 组件：提供 Kubernetes API 服务，接收并转发处理 API 请求，实现各组件间通信，并提供认证授权、访问控制等机制；

(2) kube-controller-manager 组件：一组控制器集合，用于维护集群状态，保证系统正常运行，自动进行扩展、修复和滚动更新等；

(3) cloud-controller-manager 组件：云控制管理器，通过 API 与云提供商进行通信，集成云平台资源；

(4) etcd 组件：作为集群的后台数据库，保存整个集群的状态，具有一致且高可用的键值存储特性；

(5) kube-scheduler 组件：负责资源调度，监视新创建的、未指定运行节点的 Pod，按照预定调度策略将 Pod 调度到满足 Pod 资源请求条件的最优节点上。

节点组件会在每个节点上运行，它主要负责提供 Kubernetes 的运行环境，维护本节点上正在运行的 Pod。节点组件如下：

(1) kubelet 组件：负责维护集群节点中容器的生命周期，它会接收一组描述 Pod 属性和规格信息的 PodSpecs，确保 Pod 中的容器处于健康的运行状态；

(2) kube-proxy 组件：负责网络代理和负载均衡，将流量转发至正确的 Pod；

(3) container-runtime 组件：负责节点上 Pod 以及容器的实际运行。

2.2.2 Kubernetes 调度策略

kube-scheduler 调度器通过 Kubernetes 的监测机制获取集群中新创建且未被调度到节点上的 Pod，根据一定的策略将 Pod 调度到合适的节点上。

kube-scheduler 调度 Pod 的流程如图 2.5 所示。kube-scheduler 会根据 Pod 本身和内部容器的要求，调用 PodFitsResources 过滤函数检查所有节点的空闲可用资源是否满足 Pod 对资源需求情况^[29]，从而过滤掉不满足 Pod 特定调度需求的节点，符合条件的节点被称为可调度节点。假如没有任何节点满足 Pod 资源请求，那么这个 Pod 将一直处于未调度状态直到调度器找到合适的节点。调度器会对满足 Pod 资源请求的可调度节点进行打分，获取最优的可调度节点，若最高分节点存在多个调度器则会随机选取一个，最后 kube-scheduler 会将这个调度决定通知给 kube-apiserver，将 Pod 和节点进行绑定。

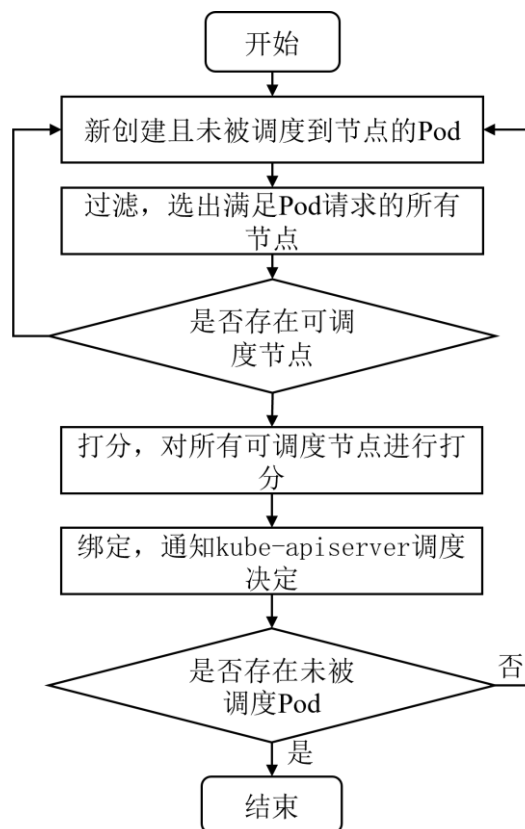


图2.5 kube-scheduler 调度流程

污点和容忍度会对 Pod 的调度产生一定的影响, 这种影响通过节点亲和性进行体现, 它使 Pod 具有被某种属性或特性的一类节点所吸引。污点应用于节点之上, 它使 Pod 和某类节点相互排斥, 使得不能容忍某些污点的 Pod 不会被这类节点所接受。容忍度是应用于 Pod 之上的, 它指的是 Pod 对具有某种污点的节点容忍限度, 它允许调度器将 Pod 调度到具有特定污点的节点, 允许调度但不保证调度。污点和容忍度相互配合, 从而避免将 Pod 调度到不合适的节点之上^[30]。

Pod 的优先级也会对 Pod 的调度产生影响, 它表示一个 Pod 相对于其他 Pod 的重要性。当启用 Pod 优先级后, 调度器会按优先级对悬决 Pod 进行排序, 较高优先级的 Pod 会更早的被调度, 若未找到满足 Pod 要求的节点, 则会触发对悬决 Pod 的抢占逻辑, 它会试图找到一个节点, 删除该节点中一个或多个优先级较低的 Pod 并将其驱逐, 释放资源, 从而将高优先级的 Pod 调度到该节点。

2.3 存储服务

随着业务的发展, 需要管理来自应用程序、用户等急剧增长的大量孤立数据, 这些数据大部分是非结构化的, 若采取多种不同的格式或存储介质进行存储, 则不容易进行管理, 增加了存储系统的复杂性, 无法以相同形式轻松访问数据用于数据分析、深度学习以及云原生应用。对象存储^[31,32]的出现消除了传统存储系统的结构复杂、容

量限制等困扰，具有更高的数据吞吐量，为深度学习、大数据、云计算等提供了更好更快速的数据存储、数据读写支持。

2.3.1 对象存储

对象存储又称基于对象的存储，是一种以非结构化格式存储和管理数据的技术，它将数据指定为不同的单元，并捆绑元数据和唯一标识符，用于查找并访问每个数据单元。对象存储会为文件添加全面的元数据，取消了传统文件存储过程中使用的文件分层结构，它将所有数据存放于一个统称为存储池的扁平地址空间中，而不是作为文件夹中的文件保存。当访问数据时，对象存储系统使用唯一标识符和元数据来查找所需的对象，对象存储支持使用 RESTful API、HTTPS、HTTP 查询对象元数据以查找和访问对象^[33]。

对象存储具有文件存储和块存储所不具备的优势：

（1）可伸缩性强：由于对象存储采用扁平化的存储架构，可轻松快速实现横向扩容，而不会受到文件存储和块存储那样的限制，对象存储基本没有大小限制，只需要添加新设备即可将数据负载扩大至 EB 级别；

（2）复杂性低易于搜索：对象存储没有文件夹或目录也就不存在具有层次结构的系统所存在的大多数复杂性，没有复杂的树或分区，扁平化的结构使得不需要知道数据的确切位置即可检索文件；

（3）高可用性和可靠性：对象存储采用冗余备份技术，可以自动复制数据并存储到多个地理位置的多个设备上，这可以在很大程度上避免服务中断和数据丢失产生的负面影响。

表 2.2 对不同类型的存储方式从数据结构、性能等几个维度进行比较分析。

表2.2 对象存储、块存储和文件存储特点对照表

特点	对象存储	块存储	文件存储
数据结构	以对象为存储单位	以固定大小的块为单位	以层次化目录和文件结构为单位
性能	适合大规模数据，吞吐量较高	高 IOPS，低延时，适合数据库、虚拟机等	适合文件共享，延迟适中
扩展性	高扩展性，支持 PB 级别存储	可扩展性较高，但受存储设备物理容量限制	存储容量扩展相对困难
适用场景	大量非结构化数据，如图片、音频、日志等	数据库、虚拟机等	文件共享、备份、归档等

2.3.2 Ceph 分布式对象存储

Ceph^[34]是一个可靠的、具有自动重均衡和自动恢复特性的去中心化分布式存储系统，它提供分布式文件存储、块存储、对象存储服务。

如图 2.6 所示，Ceph 底层实现为 RADOS 分布式存储系统，Librados 作为 Ceph 的核心库，一方面实现了对底层 RADOS 存储系统的进一步抽象封装，对外提供可以直接操作 RADOS 存储系统的 API；另一方面 Librados 则通过调用 socket 实现与 RADOS 集群其他节点的通信接收不同指令完成数据存储操作等^[35,36]。高层应用接口则是在 Librados 库的基础上进行更高层次的抽象从而实现更符合客户端要求的接口。Ceph 通过 radosgw 提供与 Amazon S3^[37]和 Swift 兼容的 RESTful API 从而对外提供对象存储服务，通过 RBD 对外提供块存储服务，通过 CephFS 实现任意大小并且兼容 POSIX 的分布式文件系统，对外提供文件存储服务和文件挂载服务。

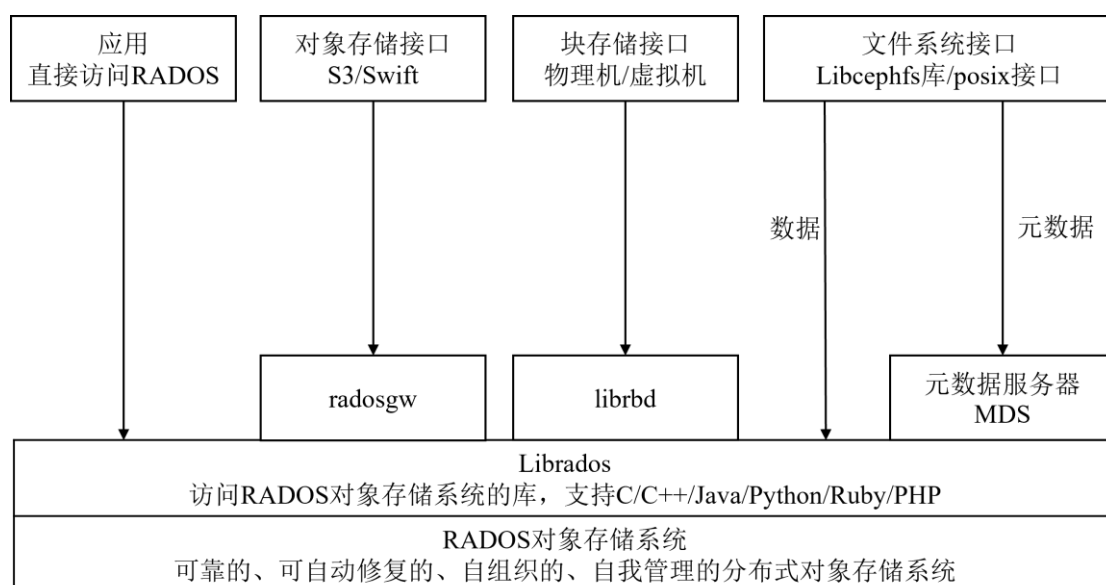


图2.6 Ceph 架构图

一个 Ceph 存储集群主要由以下三种守护进程组成：

- （1）Ceph Monitor：监控整个集群的状态，保证集群数据的一致性；
- （2）Ceph OSD：OSD 进程负责管理一组物理磁盘、存储和管理集群中的对象数据，接收并处理数据的复制、恢复、回填和再均衡等请求，向其它 OSD 发送心跳包同时监测其他 OSD 进程发来的心跳包，从而向 Monitor 提供监控信息；
- （3）Ceph MDS：MDS 进程为 Ceph 文件系统提供元数据管理与存储，实现元数据计算、缓存，从而提高文件系统的性能。

图 2.7 表示 Ceph 数据存储过程，无论是文件存储、块存储还是对象存储，Ceph 都会将数据切分为对象，以对象的形式存储在底层 RADOS 系统上。每个对象都会有唯一的对象 ID，通过文件 ID 和分片编号组合生成。对象 ID 的作用是唯一标识每一

个对象，并且标明了此对象是由哪个文件切分而成。但对象并不会直接存储在 OSD 进程管理的磁盘上，因为在集群中可能存在上千万甚至上亿的存储对象，如果对存储对象进行直接遍历寻址是一种十分低效的方式。Ceph 通过引入归置组 PG 的概念解决数据分配和数据定位的问题。placement groups (PG) 是一个逻辑概念，类似于数据库寻址时数据库中的索引，在数据迁移过程中以 PG 作为基本单位进行迁移。Ceph 会将对象映射进 PG，然后根据管理员设置的副本数进行数据复制，根据 CRUSH 算法将数据存储到不同的 OSD 节点上^[38]。

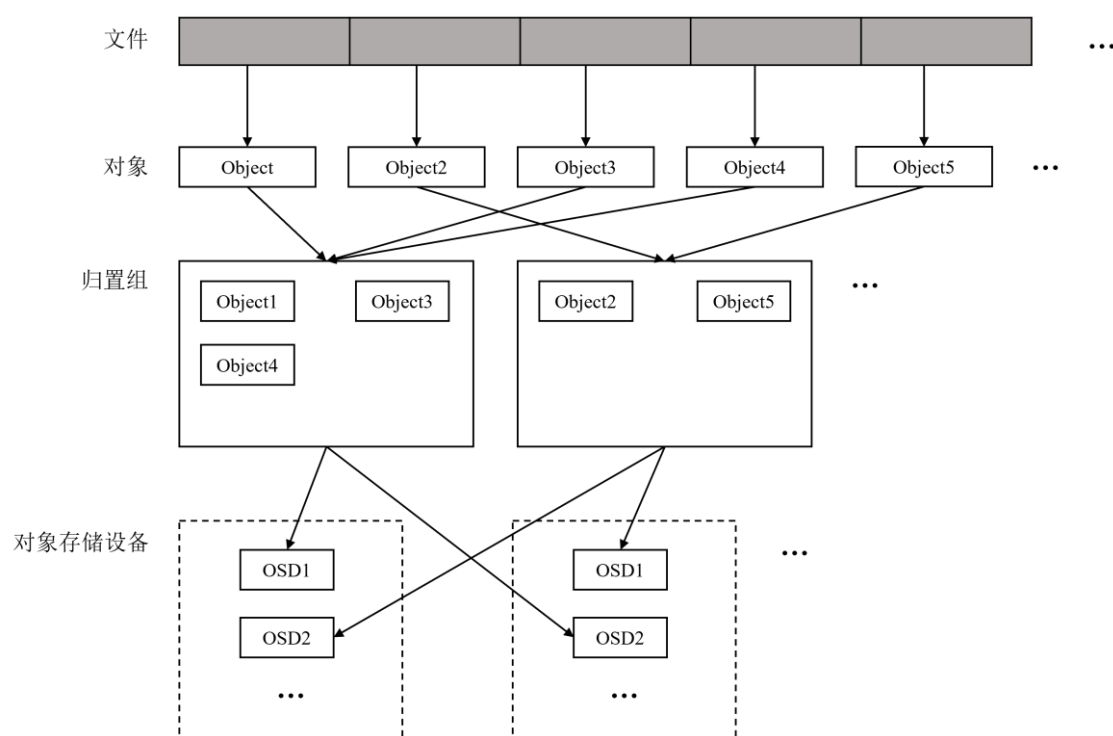


图2.7 Ceph 数据存储过程

2.4 Keycloak

Keycloak 是一个开源的身份认证和访问控制解决方案，支持单点登录、OpenID Connect、OAuth 2.0 等协议，支持使用第三方身份提供商提供的账号进行登陆认证，例如：微信、Github 等^[39,40]。Keycloak 支持细粒度的授权策略，可以通过组合不同的访问控制机制实现对不同资源的访问控制，例如：基于角色的访问控制 RBAC、基于用户的访问控制 UBAC、基于属性的访问控制 ABAC 等^[41]。

为了实现对平台系统中资源权限的访问控制，基于 Keycloak 中资源服务器（Resource Server）、资源（Resource）、范围（Scope）、策略（Policy）、权限（Permission）等抽象概念管理平台资源。资源服务器主要用于托管受保护的资源，并能够接受和响应访问受保护资源的请求，然后基于某种条件信息最终决定是否授予请求者对受保护

资源的访问权限。范围表示可以在资源上执行的操作。策略定义一系列规则，用于规定授予访问权限必须满足的条件。权限将受保护资源与评估策略进行绑定关联，从而确定是否授予访问权限。

2.5 Git

Git 是基于 Linux 内核开发的目前最流行的分布式版本控制系统。近几年由于 DevOps^[42]的迅速发展，版本控制作为 DevOps 的重要环节，它提供了一个数据或代码统一管理配置中心，帮助团队更好地管理和协作，而 Git 作为最优秀的版本控制系统之一，毫无疑问成为这方面的宠儿。本文设计实现的深度学习平台基于 Git HTTP 协议的底层原理，结合 Ceph 分布式对象存储，将 Bucket 作为服务端 Git 仓库的存储地址，结合深度学习平台系统需求实现自定义的 Git HTTP Server 后端服务。

Git HTTP 智能协议主要分为上传和下载两个交互过程，每个交互过程主要包括引用发现和数据传输两部分，对应于两次 HTTP 请求。上传交互过程对应于 push 操作，Git 通过客户端进程 send-pack 和服务端进程 receive-pack 进行数据交互；下载交互过程对应于 clone、fetch、pull 操作，Git 通过客户端进程 fetch-pack 和服务端进程 upload-pack 进行数据交互。交互过程如下：

(1) 引用发现：客户端会向服务端发送一次 GET 请求，主要作用是进行握手并获取服务端仓库的引用信息，用于分析客户端和服务端双方数据差异；

(2) 数据传输：客户端会向服务端发送一次 POST 请求。数据传输分为两部分，一种为客户端向服务端传输，即上传数据；一种为服务端向客户端传输数据，即下载数据。上传过程在通过引用发现获取服务端仓库引用信息后，客户端会自行计算出服务端所缺失的最小数据集，将最新数据打包发送给服务端，服务端接收到数据后进行解压，并更新版本仓库的引用信息。下载过程通过引用发现获取数据后，客户端通过此次 POST 请求告诉服务端“我”有哪些数据，“我”需要哪些数据，服务端通过这些信息计算出客户端所需的最小数据集，然后将数据打包通过请求响应返回给客户端，客户端接收数据进行解压和引用更新。

图 2.8 以 Git HTTP 智能协议的数据下载交互过程为例，展示其具体交互过程。客户端首先会通过 GET 请求向服务端发起第一次请求，通过第一次请求获取服务端仓库的引用信息。客户端在获取到服务端仓库的引用信息后，与本地仓库进行比较得出本地仓库需要更新的数据；客户端通过 POST 请求向服务端发起第二次请求，将本地仓库需要更新数据信息发送给服务端，服务端通过此信息计算出客户端所需最小数据集，并打包发送给客户端；客户端接收数据并解压，更新本地仓库引用信息。



图2.8 Git HTTP 下载交互过程

表 2.3 对现有部分主流的版本控制解决方案进行比较分析，列举不同版本控制解决方案采用的是分布式还是集中式、是否开源以及其特点。

表2.3 主流版本控制解决方案对照表

名称	分布式/集中式	开源/商业	特点
Git	分布式	开源	支持离线工作，具有强大的分支管理和分支合并功能，广泛应用于开源项目和商业开发
SVN	集中式	开源	简单易用，对二进制文件和大型文件的支持较好，但在无网络环境下，部分功能会受到限制
Mercurial	分布式	开源	简单易用、跨平台支持好，但缺少对高级功能的支持，可扩展性不如 Git
Perforce	集中式	商业	大型项目管理、高性能，但需付费使用

2.6 本章小结

本章主要针对私有云环境下基于微服务架构的深度学习容器化平台的设计和实验过程中所采用的关键技术进行简要概述，并列举技术的优势和特点，及采用这些技术的原因。如：Docker 作为一种容器化技术，可以快速打包部署应用程序，提高应用程序的可移植性、可伸缩性和安全性；Kubernetes 作为一个自动部署和管理容器化应用的开源平台，可以轻松管理大规模的容器化应用程序；对象存储作为一种分布式对象存储技术，提高了数据存储的可靠性和可扩展性；Keycloak 作为一个身份验证和访

问控制解决方案，实现身份验证和授权功能，确保应用程序安全；Git 作为一个分布式版本控制系统，简化数据版本管理流程。正是因为这些底层核心技术的支持，才能搭建一个性能优良、功能全面的一站式的深度学习平台，为开发者提供优质的服务。

第三章 深度学习平台系统需求分析

结合深度学习开发过程中开发者所面临的问题，本章主要对私有云环境下基于微服务架构的深度学习容器化平台的需求进行分析论述和详细说明。本章主要包含以下几个方面：第一，系统概述，简要概述整个平台系统，分析深度学习平台的层次架构；第二，系统功能性需求分析，即为满足深度学习开发、训练和部署等要求，解决传统深度学习项目开发过程中存在的痛点问题，本深度学习平台应具有的功能模块；第三，系统非功能性需求分析，即对深度学习平台的各方面性能等需求进行分析，为平台平稳健康运行进行全面把控。

3.1 系统概述

通过第一章对深度学习平台的研究背景及意义的分析，以开源开发框架为核心的深度学习平台应该具有集数据存储、算法开发、模型训练到部署的完整服务体系，囊括开发框架、算法模型、开发工具及能力平台三大核心层级，如图 3.1 所示。

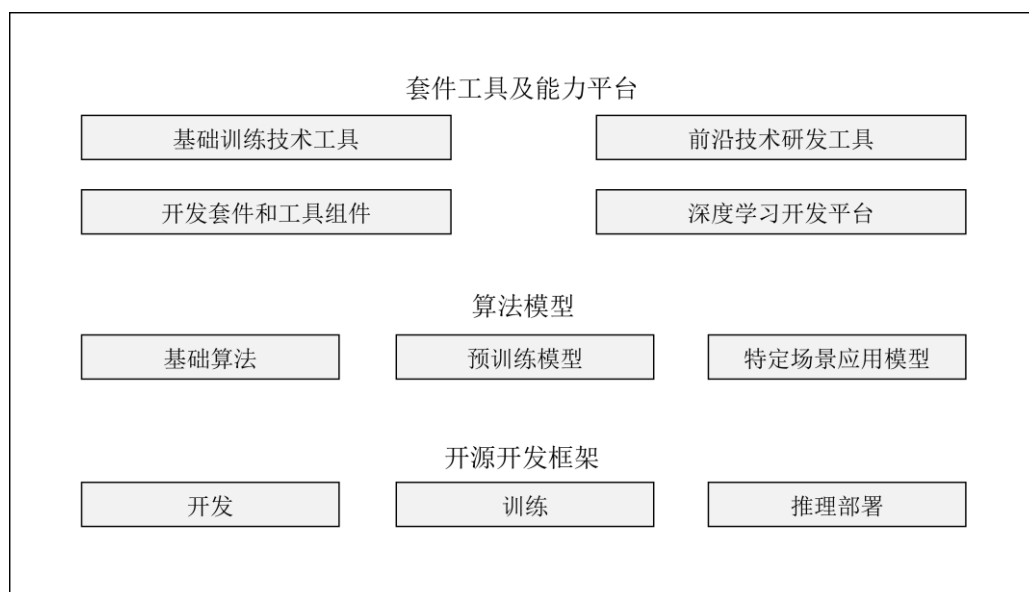


图3.1 深度学习平台层次架构

本深度学习平台系统主要面向中小型团队及高校师生，为其提供私有化的平台服务，深度学习平台底层依托于开源开发框架，例如 PyTorch、TensorFlow，开发框架作为深度学习平台的核心枢纽，在开源深度学习框架下将上层应用与下层 GPU 等计算资源相互串联，打通深度学习开发、训练、部署全流程。中间层为算法模型，深度学习平台为用户提供现有主流算法、用户自开发算法管理以便用户在现有算法的基础上进行改进和二次开发，提供模型存储及管理功能，方便用户借助预训练模型减少数

据采集、标注所耗费的时间及人力成本，缩短深度学习开发流程。上层为套件工具及能力平台，支持不同层级的模型开发和部署，降低深度学习技术准入门槛，提供全生命周期管理、搭建一体化一站式深度学习平台。

3.2 系统功能性需求分析

系统功能性需求分析主要站在用户及系统管理员的角度分析一个一站式深度学习平台应该具有哪些功能模块，从而帮助管理员方便有效的管理整个平台，为开发者提供从项目立项、数据处理、算法开发、模型训练、模型文件存储到模型托管的全周期服务。传统的深度学习开发流程如图 3.2 所示，基于业务问题，选择合适的深度学习框架，收集符合业务开发的原始数据，构建高质量数据集进行模型训练；然后对训练结果进行评估，确定其性能和准确性是否可以解决业务问题。

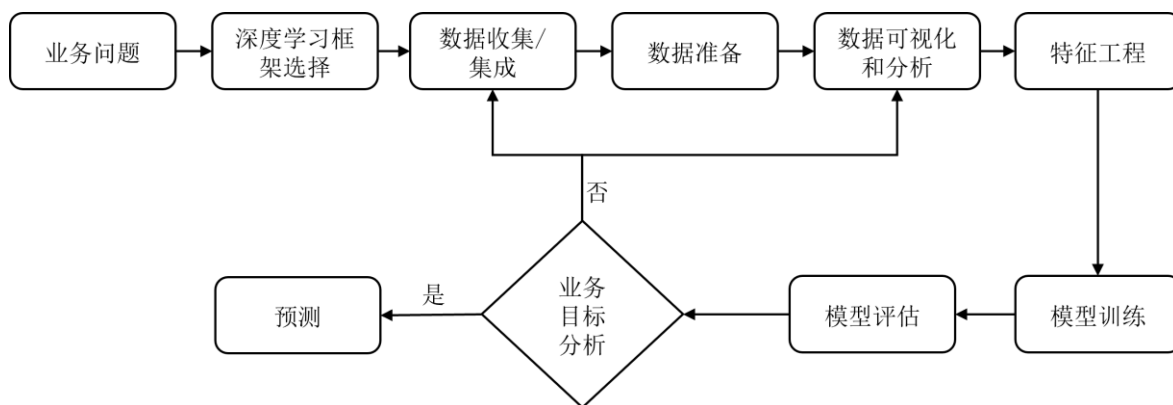


图3.2 深度学习开发流程图

3.2.1 用户管理需求分析

用户对于平台来说是一项十分宝贵的财富，二者彼此依赖、深度互动，合理管理平台中的用户对于平台的可持续发展来说十分重要。为了确保平台资源和信息安全，合理管理平台中不同身份的用户，形成细粒度的资源权限管理和访问控制机制是十分必要的。对于系统层面用户主要分为系统管理员和平台普通用户，如图 3.3 所示。

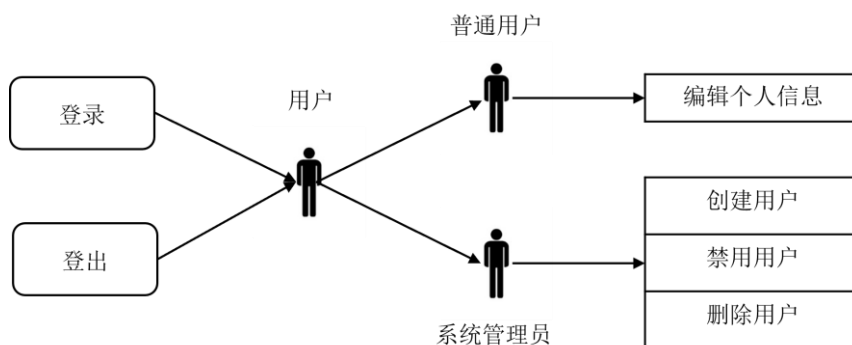


图3.3 用户管理用例图

系统管理员作为深度学习平台的主要管理和维护人员,具有整个平台最大系统权限,主要负责平台用户的管理、平台资源的管理、Kubernetes 集群的管理以及平台整体运行环境的监测等工作,如图 3.4 所示。用户的管理主要是禁用封存僵尸用户、删除非法用户等。平台资源的管理主要包括普通资源(项目、数据集、模型)的禁用、镜像资源的创建、发布等。Kubernetes 集群的管理主要包括集群的创建与删除、集群节点的添加和删除、以及整个集群日志的获取和集群状态的监测。

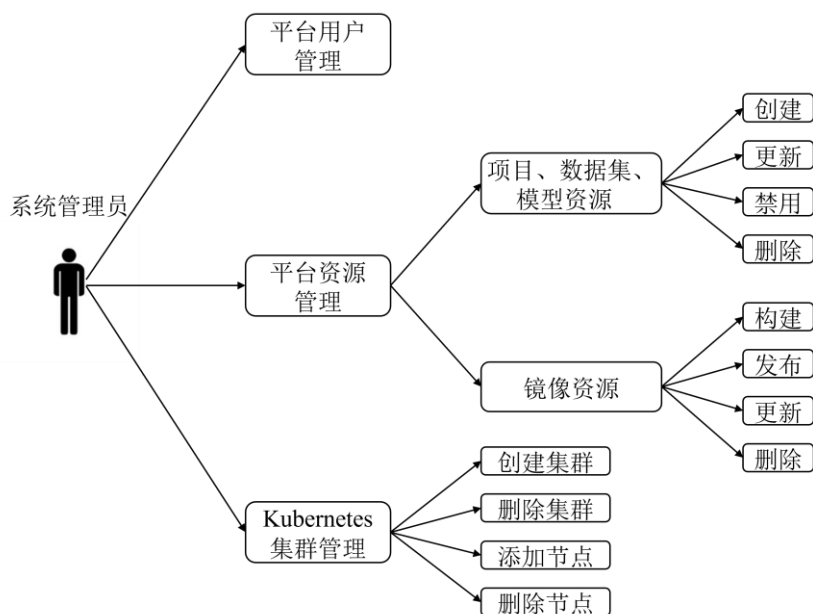


图3.4 系统管理员用例图

平台普通用户则是深度学习平台的主要使用者,使用平台的各项功能进行深度学习开发,存储深度学习相关资源和成果等。

3.2.2 存储功能需求分析

存储功能是深度学习平台所必须具备的基础功能,存储的数据主要包括关系型数据和非关系型数据。关系型数据主要采用开源的关系型数据库如 MySQL 等进行存储;其他数据为了更加匹配深度学习云平台这种需要存储大量各类型数据的应用场景,采用对象存储的方式来存储各类型文件数据,构建数据湖。

关系型数据存储需求如下:

(1) 建表存储深度学习平台所需的的关系型数据,用于支持平台后台代码逻辑的运行以及前端页面信息的展示;

(2) 支持数据可视化展示;

(3) 支持数据备份和数据恢复。

文件数据存储需求如下:

(1) 文件上传与下载:采用对象存储存储文件数据,需支持用户在 Linux 环境

下以执行 shell 命令的形式上传和下载数据，同时用户也可以通过调用 API 接口的形式上传和下载数据，支持文件批量操作、文件断点续传等；

(2) 文件在线查看：用户无需下载文件即可查看文件内容；

(3) 存储系统快速扩容：基于深度学习庞大的数据使用量，平台数据存储系统应具有快速扩容能力以满足庞大的数据存储需求，对象存储由于采取扁平的数据存储结构，可以以低成本快速实现存储容量扩容；

(4) 文件分享：在深度学习平台对象存储服务内部支持用户间数据快速传输分享，实现用户间数据共享；

(5) 文件数据访问权限控制：细化文件数据读写操作的权限，公开的文件数据平台内用户皆可访问读取，写操作则只允许具有特定权限的用户执行，私有的文件数据只允许部分成员访问（文件所有者、某类资源组的成员）。

3.2.3 主机管理需求分析

主机管理模块的服务器主要分为两部分，一部分为开发环境容器宿主机，一部分为用于部署模型实现深度学习应用落地的 Kubernetes 集群。

开发环境容器宿主机主要用于部署用户启动的深度学习开发环境容器，为开发环境容器提供 GPU、内存等资源，结合容器秒级的启动速度，通过容器的暂停和重启等释放宿主机资源，进而提高资源的利用率，解决深度学习开发过程中缺少服务器或计算资源的情况。开发环境容器宿主机管理应满足一下需求：

(1) 添加主机：将服务器加入到系统中，实现计算、存储等资源的跨域整合，进而提供计算、存储资源，为开发环境容器部署提供基础环境；

(2) 删除主机；

(3) 主机状态监控：实时获取服务器各种资源的时序用量信息。

Kubernetes 集群作为本深度学习平台中模型部署的主要承载者，用于解决模型部署困难、深度学习应用管理和后期维护复杂繁琐等问题，通过已有的调度策略合理利用集群中各个节点的 GPU、内存等资源将启动推理服务的 Pod 调度到合适的节点进行部署，管理集群中已部署的各项 AI 应用。因此，对于 Kubernetes 集群的管理十分重要，集群管理模块需要满足一下需求：

(1) 创建集群：集群为用户提供模型部署及应用发布的基础环境，合理调度利用集群资源；

(2) 删除集群；

(3) 添加集群节点：通过添加集群节点扩大集群规模，对集群资源进行横向扩展，以满足服务不断增加的需要；

(4) 删除集群节点：对于出现问题的节点或软硬件条件已经不再满足当前服务

部署需求的节点，可以将其踢出集群，避免出现不必要的麻烦；

(5) 更新节点信息：通过管理更新节点信息，赋予节点特殊标记；

(6) 集群状态监控：监控集群状态，并以可视化的形式展示给系统管理员，便于系统管理员管理集群。

3.2.4 镜像管理需求分析

在深度学习开发过程中存在不同的深度学习开发框架，不同用户由于自身先验经验、开发习惯、周围深度学习学术氛围等因素并结合各个深度学习框架的特点可能会选择不同的深度学习框架，为了满足不同用户对不同深度学习开发环境的需求，应当提供各种不同的自定义镜像，用以启动基于不同框架的深度学习开发环境。因此镜像管理模块应当满足一下需求：

(1) 提供镜像存储仓库服务，便于系统管理员发布基于本平台特色的镜像，以及平台用户拉取镜像，构建自己的深度学习开发环境；

(2) 镜像仓库权限管理及访问控制服务，镜像的推送发布应只能由系统管理员执行，从而避免镜像仓库存储无效或非法不安全的镜像，平台普通用户仅能从镜像仓库中拉取镜像，构建运行容器；

(3) 海量的镜像集，用以支持 PyTorch、TensorFlow 等深度学习框架，以及对同种框架不同版本的支持，同时还可选择是否支持 GPU 等；

(4) 提供检索查询镜像仓库中存储镜像信息的 API 接口集，便于显示镜像仓库中的镜像列表以及镜像的基本信息。

3.2.5 数据集管理需求分析

数据集作为深度学习开发过程中的重要一环，数据集质量的好坏直接决定了深度学习训练结果的好坏，一个高质量的数据集往往能够提高模型训练的质量和预测的准确性。数据是基础，任何深度学习研究都离不开数据。当我们根据实际问题确定业务目标后，就进入数据收集阶段。数据收集阶段开发者需要根据业务目标收集合适的数据，可以直接使用符合要求的公开数据集或在已有数据集的基础上进行数据增强等操作以满足对数据集使用的需求，对于没有符合要求的公开数据集开发者则需要获取相关类型的数据进行数据标注以建立符合要求的数据集。因此，如图 3.5 所示，数据集管理模块应满足以下需求：

(1) 已有数据集上传功能：用户创建数据集信息记录后，可直接上传现有数据集到深度学习平台，管理和使用自己的数据集；

(2) 原始数据文件导入：用户可以创建在线标注类型的数据集信息记录，导入原始未标记的数据文件；

(3) 数据标注：支持数据集在线标注；

(4) 数据集版本管理：支持基于 Git 的数据集版本管理，为数据集在线协作标注提供支持，对于已标注完成的数据集管理者可发布相应数据集版本提供给平台用户使用，用户可根据数据集的使用情况提供反馈意见，数据集开发管理者可对数据集进行优化，进而迭代更新。

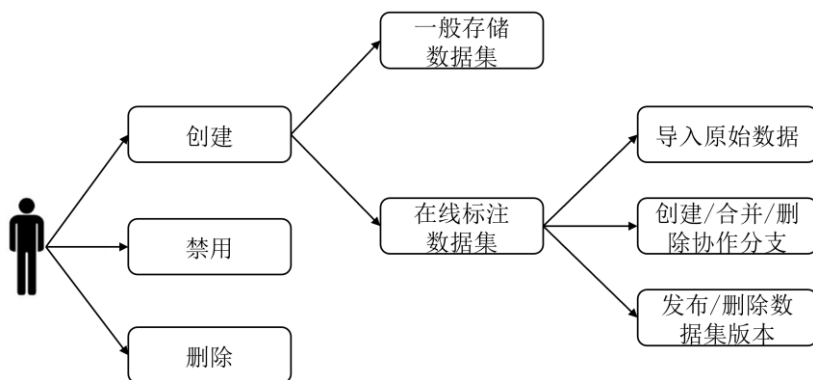


图3.5 数据集管理用例图

3.2.6 项目及开发环境管理需求分析

开发一个深度学习应用往往需要进行多人协作进行，通过以项目的形式管理深度学习开发的整个流程，项目成员相互沟通、相互协调，使得整个开发过程具有更好的可维护性、更容易的跟踪、更明确的版本控制、更容易的迭代和更可靠的架构。基于项目所面向的业务问题，配置项目所使用的数据集、深度学习框架等，进而启动深度学习开发环境容器。此模块应满足以下需求：

- (1) 支持基于 Git 的版本管理；
- (2) 可对项目进行配置，选择项目所采用的数据集、预训练模型，配置项目采用哪种深度学习框架的哪个版本，配置项目是否使用 GPU 等；
- (3) 支持项目成员管理，设置项目成员的权限；
- (4) 通过项目一键构建深度学习开发环境，并允许用户通过 Web 页面访问 JupyterLab 开发环境，并收集开发环境日志信息；
- (5) 监控开发环境容器运行状态和宿主机资源使用情况；
- (6) 自动化收集深度学习项目成果产出，获取训练产生的模型文件，通过对象存储上传至私有云。

3.2.7 模型管理及模型托管需求分析

此处模型是指已经训练好的神经网络模型的参数和结构，是模型训练的最终产物，平台模型的获取途径主要分为两部分：一部分模型来自平台用户通过开发环境进行模型训练产生的，一部分来自平台用户直接上传的已有模型。模型托管是指将深度学习

模型部署到生产环境，提供在线的推理服务。用户可基于深度学习平台，通过现有模型编写相应的数据预处理和后处理函数后，在 Kubernetes 集群中一键部署模型，对外向用户提供推理服务 API 接口。此模块需要满足以下需求：

- (1) 模型资源管理，支持上传已有模型、发布版本、编辑更新模型记录信息等；
- (2) 基于模型一键部署服务，并对外提供数据推理 API 接口；
- (3) 服务运行状态监测。

3.2.8 系统监控模块需求分析

为了维护平台各项服务稳定可靠的运行，实现系统监控模块是十分必要的，监控模块通过采集各项监控指标，配合合理的预警机制，可以提前发现平台中的问题，通过分析监控指标信息寻找问题产生的根源，从而对问题快速做出反应，解决问题，进而保证平台各项服务稳定运行，提升用户体验。

3.3 系统非功能性需求分析

系统的非功能性需求分析是指对系统的高可用性、安全性、易用性等性能的分析，它涉及识别应用程序中可能存在的性能、可靠性和安全性问题，以及在设计和开发过程中如何处理这些问题。

3.3.1 高可用性

高可用性是指系统无中断地执行其功能的能力。提高系统高可用性的方式可包括以下几种：

(1) 使用集群减少出现单点故障的可能性，本系统使用 Ceph 集群实现对象存储功能，它可以灵活控制副本数，支持故障域分隔和数据强一致性，多种故障场景自动修复；通过 Kubernetes 在集群中部署容器化应用，它通过主节点、节点和应用的多个副本实现高可用性，当集群中的某个服务出现错误时主动重新启动失败的容器，或某个节点出现故障会将此节点上的服务调度到其他节点部署；

(2) 使用缓存机制，缓存热点数据，减少数据库的访问请求；

(3) 采用微服务架构设计整个系统，将整个系统拆分为多个独立的服务，实现系统不同功能之间的解耦和或松耦合。

3.3.2 安全性

深度学习平台的用户登录认证系统基于整合 Keycloak 实现，它支持多种登录认证协议，并提供支持细粒度授权策略的访问控制机制，以确保平台对外提供的各项 API 接口的安全性。另外，平台各个服务器之间通过 WireGuard 组件虚拟内网，将各

个不同地理位置的服务器接入内网，服务器间的流量传输通过 WireGuard 进行加密，保证了数据传输的安全性^[43]。

3.3.3 易用性

平台的易用性是指平台的设计和功能是否能够让用户轻松地使用和操作。一个易用性高的平台能够提高用户的使用体验和效率，降低用户的学习成本和使用成本，从而增强用户的忠诚度和满意度。提高平台易用性的实现方式包括：

（1）用户研究：了解用户的需求和习惯，设计符合用户习惯的交互方式和界面设计，提高用户的使用体验；

（2）界面设计：平台通过为用户提供简洁干净、流程清晰的前端界面，减少冗余操作，简化深度学习开发流程，使用户可以快速轻松上手，减少深度学习学习成本，降低平台使用门槛；

（3）导航设计：设计直观、易懂的导航结构，让用户可以迅速找到相应的界面，减少用户的迷失感，提高用户使用的效率；

（4）反馈机制：为用户提供及时的反馈，让用户了解操作的结果和后续的步骤，避免用户的猜测和疑惑；

（5）帮助文档：提供详细的帮助文档和教程，让用户快速掌握平台的使用方法。

3.4 本章小结

本章对私有云环境下基于微服务架构的深度学习容器化平台的设计与实现进行需求分析，详细讨论了平台的各项功能性需求和非功能性需求，为后续系统整体架构、网络结构和系统功能架构的设计明确目标和推进方向。在系统功能性需求方面，我们主要考虑了用户管理、存储、主机管理、镜像管理、数据集管理、项目及开发环境管理、模型及模型托管和系统监控等功能。通过这些功能，提供一个全面的深度学习容器化平台，帮助用户轻松地进行深度学习开发、管理和部署深度学习应用程序。在系统非功能性需求方面，我们主要考虑了高可用性、安全性、易用性等方面。通过高可用性的设计，我们可以保证系统在面对异常情况时仍然能够保持正常的运行；通过安全性的设计，我们可以保护用户的数据和隐私，确保系统的安全性；通过易用性的设计，我们可以提高系统的易用性和用户体验，使用户能够更加方便地使用系统。

第四章 深度学习平台系统整体设计

根据第二章对实现深度学习容器化平台相关技术的研究以及各个技术的特点,结合第三章对系统需求的分析,本章对平台系统的总体设计进行进一步阐述,介绍深度学习平台的系统架构、私有云网络环境搭建、系统功能的设计方案。

4.1 系统架构设计

本文采用微服务思想设计整个深度学习平台的系统架构,结合微服务模块化、松耦合、支持多语言、各模块可独立部署等特点,可以使整个系统具有更好的容错性、可靠性和可维护性。如图 4.1 所示,系统架构主要分为视图层、代理层、业务层、基础服务层、基础设施层五部分。

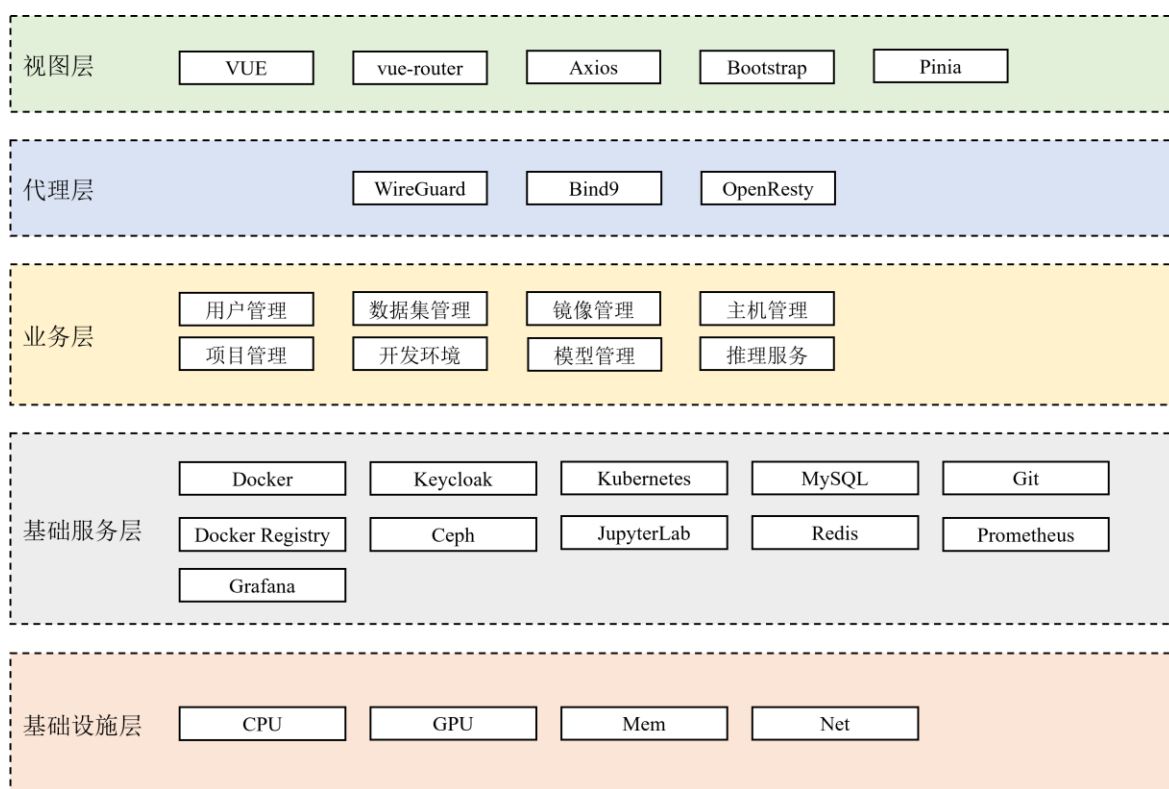


图4.1 深度学习容器化平台系统架构图

视图层: 即系统的前端部分,通过 Web 界面的形式展示给用户,基于 VUE 框架开发,使用 vue-router 插件管理前端路由构建单页面响应,使用 Bootstrap 所提供的可复用组件和前端工具包快速构建前端页面,使用 Axios 进行网络请求实现和后台的数据交互,使用 Pinia 实现 VUE 跨组件跨页面状态共享。

代理层: 即系统的整个网络流量的加密、转发层。系统通过 WireGuard 实现动态

组网构建虚拟内网环境,通过构建虚拟隧道将处于不同物理地址不同网段下的服务器加入到同一虚拟内网环境中,对虚拟内网中的流量转发进行加密以确保数据传输安全。OpenResty 是一个内部集成了大量精良 Lua 库、第三方模块以及大多数依赖项的基于 Nginx 和 Lua 的高性能 Web 平台,通过其实现高性能的 HTTP 和反向代理服务^[44]。OpenResty 作为代理服务器接收前端客户端的 HTTP 连接请求,代理服务器将请求转发给内部网络服务器的对应服务,然后将应用服务处理后的返回结果返回给前端客户端。通过 OpenResty + Lua + Redis 的组合形式,解析请求地址,实现动态路由转发。通过 Bind9 搭建部署私有的 DNS 服务器^[45],提供域名解析服务,将用来寻址的域名解析为对应的 IP 地址。平台基于私有的 DNS 服务加上 WireGuard 的配合实现测试环境和生产环境采用同一套域名,但可以通过 WireGuard 客户端配置不同的 DNS 实现通过同一域名访问不同环境服务的目的,最大限度地保证测试环境和生产环境配置信息保持一致,同时确保测试环境只有内部开发测试人员可以访问。

业务层:即系统主要功能模块的实现层,通过代码编写实现各个功能模块的功能,整合并调用基础服务层各项服务功能,对外提供 API 接口,处理前端的各个请求,实现前后端数据交互。

基础服务层:主要包含为实现各个功能模块所采用的底层技术和服务。如图 4.2 所示,基于微服务的设计思想,将系统的各项服务通过 Docker 以容器的形式部署到特定的服务器上,平台为用户提供的开发环境也是以容器的形式在平台提供的服务器上启动,模型托管则通过 Docker + Kubernetes 将模型部署到平台为用户提供的集群上,从而对外提供推理服务。平台也会为用户提供符合平台要求和规范的开发环境基础镜像,这些镜像则存储在 Docker Registry 私有镜像仓库中,最终以容器的形式启动开发环境,用户可以在 JupyterLab 的 Web 页面中编写代码进行深度学习开发。Keycloak 则会为整个深度学习平台提供身份认证和访问权限控制服务。平台的操作数据保存至 MySQL 数据库中,通过内存和 Redis 实现平台的数据缓存机制,缓存热点数据,减少数据库的访问请求,从而减少数据库压力降低请求响应时间;平台的文件数据(数据集文件、模型文件、项目仓库文件)则保存至 Ceph 提供的对象存储服务中。通过 Exporter + Prometheus + Grafana 收集监控数据,实现对平台各项基础服务(MySQL、Redis、OpenResty 等)和服务器的状态监控,并以 Web 页面的形式进行可视化展示。

基础设施层：即系统各个硬件设备如 CPU/GPU 服务器提供的计算资源、内存资源、网络带宽等资源，基础设施是整个深度学习平台的基础，一切服务都是在此基础上部署或设计实现。

本小节主要介绍深度学习平台私有云环境各个服务器硬件参数以及私有云网络环境的搭建过程。为了便于管理和环境统一，服务器均使用 Linux 服务器，系统版本均为 Ubuntu Server 20.04，服务器资源配置参数应大于等于表 4.1 所列参数。

31

发训练部署所需的服务器一般分为两种：一种为不包含 GPU 的，用于计算量较小的轻量级深度学习任务；一种为提供 GPU 的，使用 GPU 可以大大提高计算速度提高模型训练和推理服务预测效率。对于部署深度学习平台后端服务的服务器，为满足 Java 核心应用程序的运行应至少需要 8 CPU 和 16G 内存。

表4.1 深度学习容器化平台服务器参数表

服务器名称	资源配置	GPU 参数	硬盘参数	描述
代理服务器	4 CPU + 8G MEM	无	256G SSD	用于部署 OpenResty 和 WireGuard Server
DNS 服务器	4 CPU + 8G MEM	无	256G SSD	用于部署 Bind9 搭建私有的 DNS 服务器
Ceph 集群服务器	Ceph Monitor 节点： 4CPU + 64G MEM	无	256G SSD	部署 Ceph 对外提供对象存储服务
	Ceph OSD 节点： 8 CPU + 16G MEM	无	256G SSD + 3 * 1T HDD	
镜像仓库服务器	8 CPU + 16G MEM	无	256G SSD + 10T HDD	部署 Docker Auth 和 Docker Registry 对外提供镜像仓库服务
Kubernetes 集群服务器	64 CPU + 256G MEM	2 * Tesla P4 (8G MEM)	3 * 1.8T HDD	在 Kubernetes 集群上部署模型，GPU 型号包括但不限于表所述
开发环境部署服务器	64 CPU + 256G MEM	2 * Tesla P4 (8G MEM)	256G SSD	部署深度学习开发环境容器，GPU 型号包括但不限于表所述
其他应用部署服务器	8 CPU + 16G MEM	无	256G SSD	部署深度学习云平台后端应用、MySQL、Redis、Keycloak 等

为了便于管理不同局域网下服务器，实现不同局域网下服务器之间安全高效的通信和服务器资源跨域整合，本文通过 WireGuard 搭建虚拟专用网络，连接不同局域网下的服务器。WireGuard 支持目前最先进的加密技术，采用了安全可信的架构来保证虚拟网络的安全性，具有高速加密传输的特点。本文将代理服务器作为虚拟专用网络

的入口，即中心服务端，部署 WireGuard Server 服务，接受各个局域网下服务器的连接请求。其他各个服务器作为固定接入端，部署 WireGuard Client 服务。对于深度学习平台系统开发人员或系统运维测试人员作为移动接入端，通过配置 WireGuard Client 使用移动设备随时随地访问各个服务器。虚拟专用网络拓扑结构如图 4.3 所示：

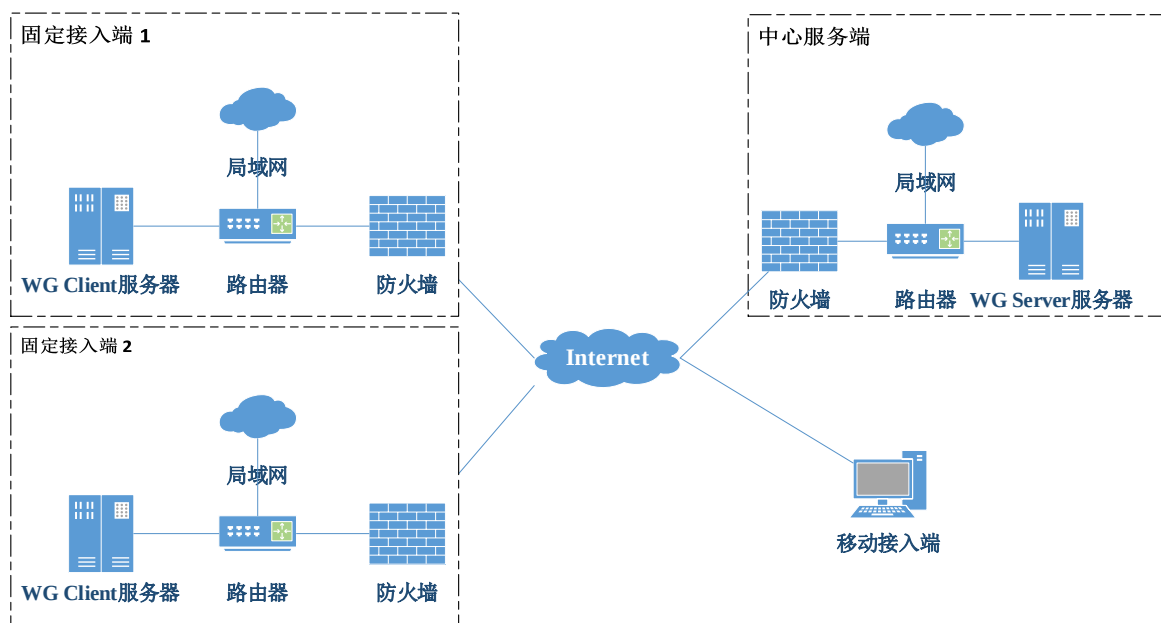


图4.3 虚拟专用网络拓扑图

虚拟专用网络中的各项应用服务之间可以通过 `wg ip + port` 的形式进行相互访问，既确保了数据传输的速度，又保证了数据传输的安全性，同时还可以使运维开发人员随时随地接入虚拟专用网络访问各个服务器。

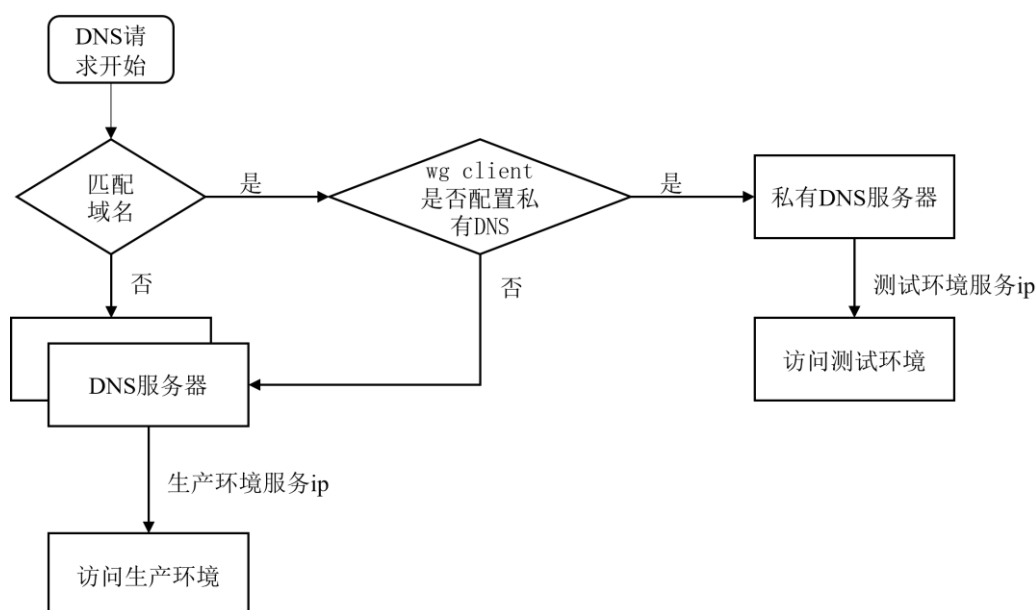


图4.4 DNS 域名解析流程图

如图 4.4 所示，本文还通过将 WireGuard 搭建的虚拟专用网络与 Bind9 搭建的私

有 DNS 服务相结合，配置 WireGuard Client 使用不同的 DNS 配置，实现使用同一域名访问不同环境下应用服务的功能，尽最大可能保证深度学习平台测试环境和生产环境的各项配置保持一致。

当开发测试人员访问测试环境深度学习开发平台时，需要通过 WireGuard Client 接入虚拟专有网络，使用私有 DNS 服务解析域名获取测试环境 IP，否则将访问生产环境的深度学习平台。这样做既可以保证不同环境配置基本一致，也可以确保非内部人员无法访问测试环境，因为只有内部人员才可以申请 WireGuard 公私钥并更新 WireGuard Server 配置以允许内部人员通过分配的 WireGuard IP 接入虚拟专用网络。

4.3 系统功能设计

为了设计实现一个一站式的深度学习平台，满足开发者对于深度学习资源管理、开发、训练以及模型部署的需求，根据上一章对系统功能性需求的分析，如图 4.5 所示，平台主要包括用户管理、主机管理、镜像管理、数据集管理、项目管理、开发环境管理、模型管理、模型托管等模块，通过不同模块相互配合解决计算资源利用率低、开发环境部署困难、数据存储安全性、模型部署及管理复杂、团队管理不合理等问题，简化深度学习开发流程，使用户聚焦于深度学习业务问题本身。

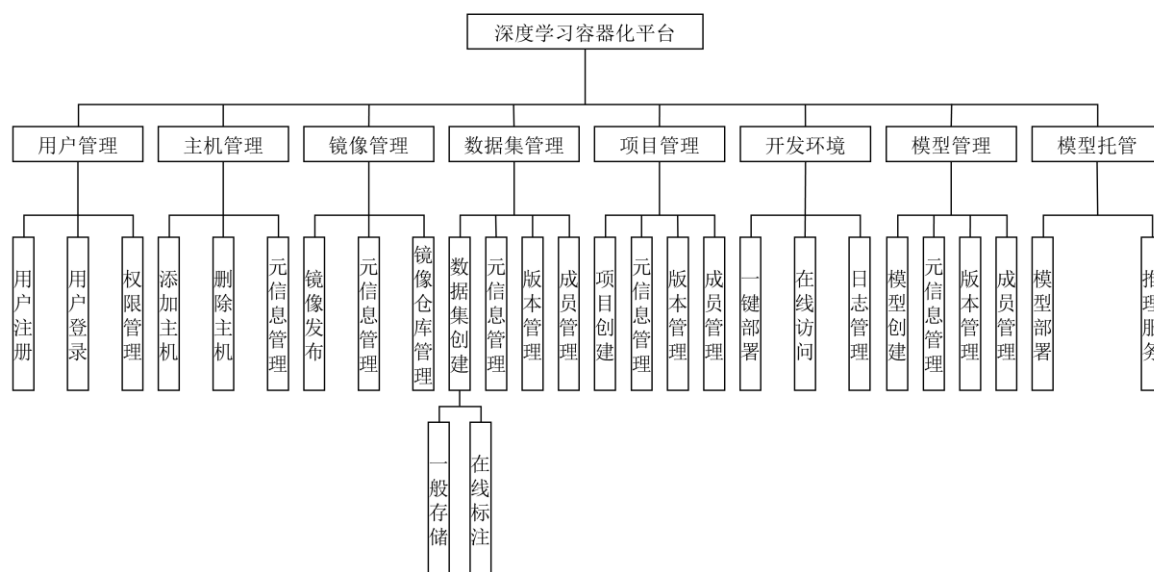


图4.5 深度学习容器化平台系统功能模块图

平台提供的各个功能模块并不是孤立存在的，每个模块之间都有着紧密的联系，如图 4.6 核心功能关系图所示，整个平台提供了一个完整的深度学习开发、训练、部署闭环。整个平台系统由一个中心节点和六个核心功能节点组成，中心节点即深度学习平台。数据集管理模块和模型管理模块分别用于管理和存储平台中的数据集和模型；项目管理模块用于管理深度学习开发训练过程中用户所编写的开发代码，同时配置项

目运行所需的开发环境信息以及所需的数据集和模型；开发环境容器基于项目配置信息创建并启动，支持多语言多开发框架，开发环境中生成的模型文件会转存至模型管理模块；模型托管模块则基于模型管理模块提供的模型部署推理服务应用。

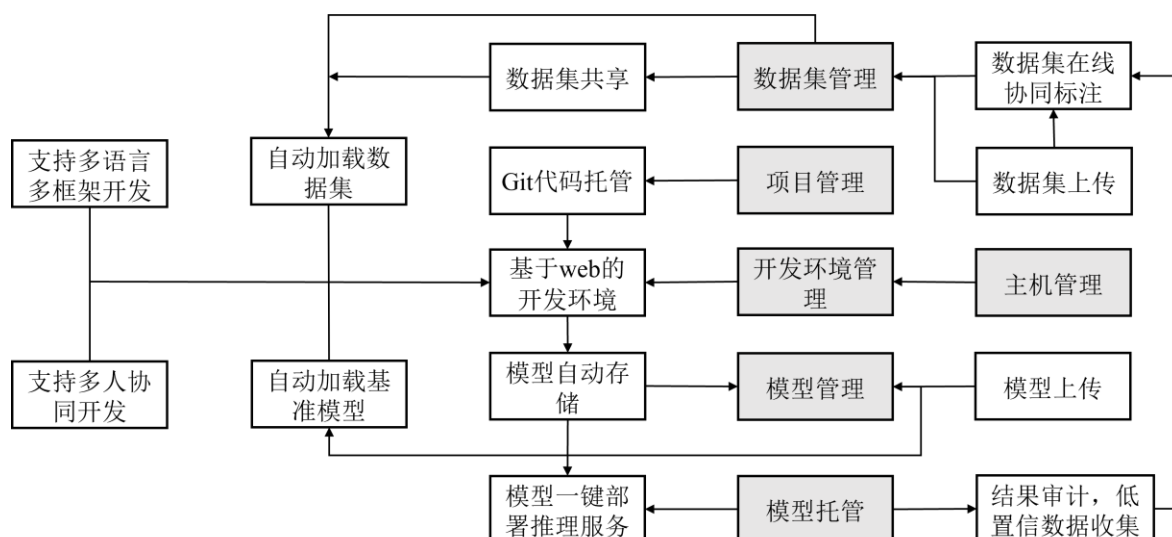


图4.6 深度学习容器化平台核心功能关系图

4.3.1 用户管理模块设计

在深度学习平台系统层面用户可分为系统管理员和平台普通用户。系统管理员可以创建、禁用、删除普通用户，修改普通用户信息，管理普通用户权限；平台普通用户可以在深度学习平台中创建管理自己的资源（如：数据集、项目等），或被设定为某项资源的特定角色，从而具有某项资源的特定权限。

4.3.2 主机管理模块设计

在主机管理模块中，系统管理员可以将新的服务器添加到主机管理界面，实现服务器资源的跨域整合，为平台用户提供部署开发环境的宿主机。对于主机管理模块，平台普通用户只有主机的使用权，在使用主机部署开发环境时，用户需申请主机资源；系统管理员管理平台提供的主机，添加新主机，删除旧主机，更新主机元信息。

添加主机流程如图 4.7 所示，管理员首先通过 Java 后台生成一对公私钥，将公钥添加至要添加主机的公钥管理文件中，本平台使用的均为 Linux 服务器，安装的系统均为 Ubuntu Server 20.04，即将公钥添加至/root/.ssh/authorized_keys 文件中，然后在平台主机管理界面输入主机 IP、用户名、密码等信息，生成主机对象，Java 后台自动将私钥添加至主机对象中，尝试与宿主机建立 SSH 连接，建立成功后获取主机 GPU、CPU、内存等信息，将主机对象添加至 MySQL 数据库中，定时收集主机资源时序信息用于实时显示主机状态，最后通知用户添加主机成功。

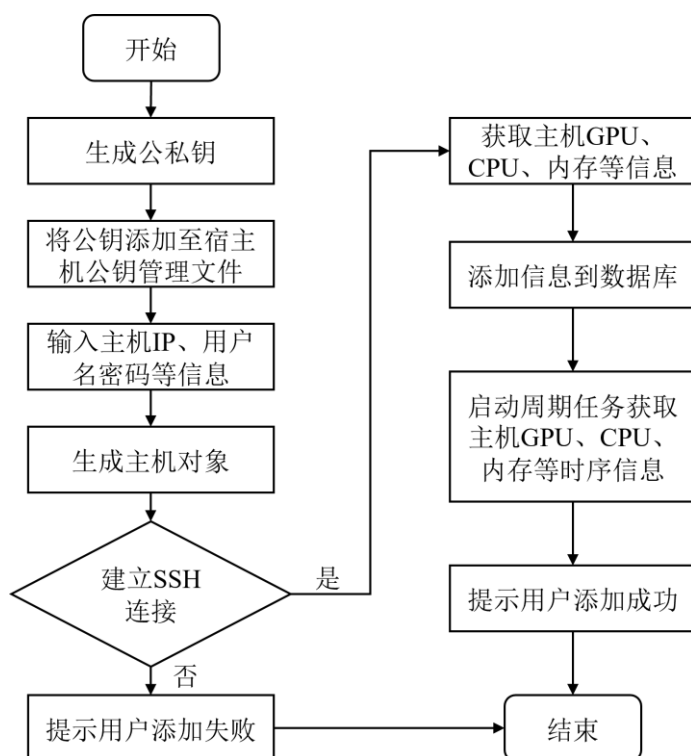


图4.7 添加主机流程图

4.3.3 镜像管理模块设计

对于镜像管理模块，平台通过部署 Docker Registry 搭建私有的镜像存储仓库，为用户提供镜像上传、下载服务。本模块的主要功能是为用户提供整合了不同版本的 PyTorch 或 TensorFlow 深度学习框架的 JupyterLab 镜像，为用户部署开发环境容器提供基础镜像。系统管理员会基于当前主流版本的 PyTorch 或 TensorFlow 深度学习框架构建基础镜像，并将新构建的镜像发布到深度学习平台，供平台用户使用这些基础镜像。对于镜像管理模块，平台普通用户只有镜像的使用权，在构建深度学习开发环境容器时，从平台私有镜像仓库中拉取镜像；系统管理员管理平台提供的镜像，发布新镜像，删除旧镜像，更新镜像元信息。

镜像管理模块主要的功能就是发布镜像以供用户使用，镜像发布流程如图 4.8 所示，系统管理员会根据用户需求以及各个深度学习框架的最新版本，编写构建自定义 JupyterLab 镜像的 Dockerfile 文件，通过执行 Docker 构建镜像命令“docker build”构建镜像，执行“docker login”命令输入用户名密码登录 Docker Registry 私有镜像仓库，执行“docker push”命令将镜像推送至私有镜像仓库，推送操作只有系统管理员可以执行；当镜像推送完成后，系统管理员可以通过 Docker Registry V2 提供的 API 获取私有仓库中镜像的元信息，然后管理员认领最新发布的镜像并补充镜像头图、描述信息等说明性信息，最后将镜像元信息保存至 MySQL 数据库，发布镜像。

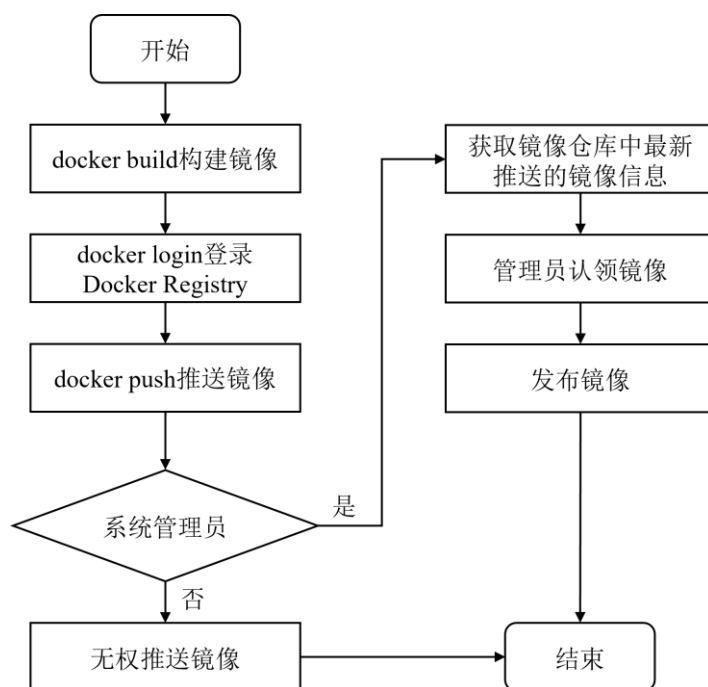


图4.8 镜像发布流程图

4.3.4 数据集管理模块设计

数据集管理模块主要为用户提供数据集在线存储和数据集在线标注服务，支持用户上传已经标注完成的数据集，也支持用户创建在线标注类型的数据集，上传数据集原始数据，从而构建数据集仓库，便于平台用户基于平台共享数据集信息。

表4.2 数据集角色权限表

类型	角色	权限说明
一般存储	Owner	1. 管理数据集元信息 2. 管理数据集成员
	Admin	数据集管理员，管理 Viewer 用户
	Viewer	查看使用数据集（公开数据集普通用户默认为 Viewer 角色）
在线标注	Owner	1. 管理数据集元信息 2. 管理数据集成员 3. 管理数据集 Git 版本库 4. 发布数据集版本
	Admin	1. 管理 Editor 和 Viewer 成员 2. 管理数据集 Git 版本库 3. 发布数据集版本
	Editor	数据集在线标注协作者，可创建自己的协作分支
	Viewer	查看使用数据集（公开数据集普通用户默认为 Viewer 角色）

数据集管理模块的数据集包含一般存储和在线标注两种类型,每种类型的数据集均可设置为公开或私有两种状态。在线标注类型的数据集支持 Git 版本管理,数据集成员可以创建自己的协作分支,进行数据集在线标注,标注完成后可以向 master 分支发起合并请求。当数据集标注完成后,数据集管理员可以通过对 master 分支执行“git tag”打标签的方式标识数据集版本信息,从而发布对应的数据集版本供成员或平台用户使用。同时数据集资源有自己的成员管理体系,不同角色的成员对于数据集资源有不同的权限,如表 4.2 所示。数据集文件存储在 Ceph 提供的对象存储服务中,每个数据集都由对应的 Bucket 进行存储,用户在使用数据集时可以通过 S3FS 将数据集文件挂载为本地文件系统进行使用。

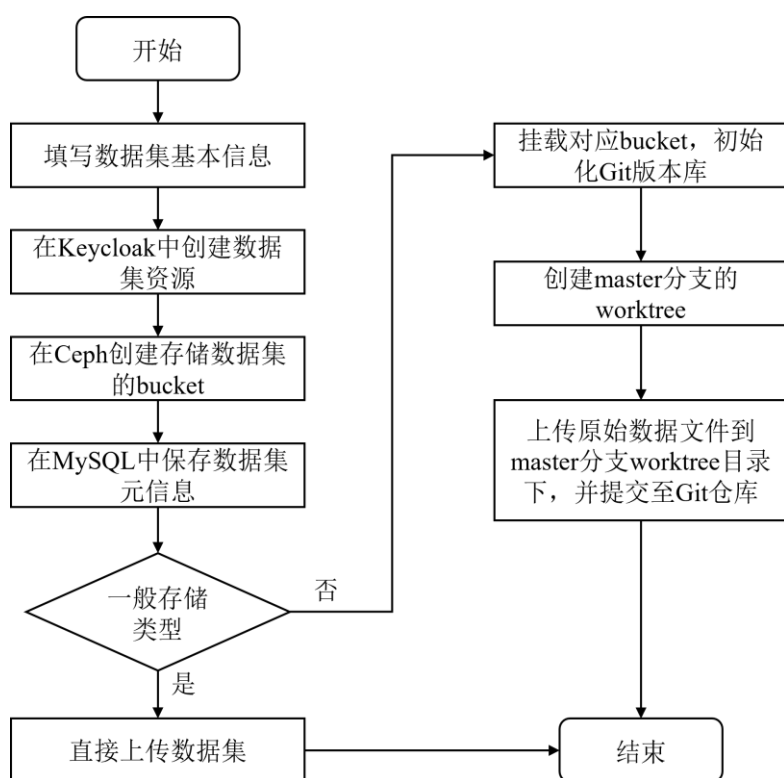


图4.9 数据集创建流程图

数据集创建流程如图 4.9 所示。一般存储类型的数据集仅仅是为用户提供了数据集的在线存储功能,在创建此类型的数据集时,首先在 Keycloak 中创建对应的资源并绑定相应的权限策略,为后续项目的权限管理做准备,然后在 Ceph 提供的对象存储服务中创建对应的 Bucket,用户将数据集上传至对应的 Bucket 中进行保存。在线标注类型的数据集为用户提供在线标注功能,此功能的 Java 后台的实现逻辑是结合 Git 版本管理和 Ceph 对象存储实现的。在线标注类型的数据集会首先将对应的 Bucket 挂载之 Java 后台服务所在的服务端,将其映射至本地文件系统的特定路径下;然后,在此 Bucket 中初始化管理在线标注数据集的 Git 版本库;其次,在 Git 版本库创建完成后,为默认存在的 master 分支创建对应的同名 worktree,用于通过对象存储服务支

持的 HTTP、HTTPS 协议直接在线查看不同分支下的文件内容；最后，可直接将文件上传至分支的 `worktree` 下，通过预留接口，将更新保存至 Git 版本库。

数据集数据通过 Ceph 提供的对象存储服务存储在对应的 Bucket 中，如图 4.10 所示，为不同类型数据集的 Bucket 存储结构。两种数据集 Bucket 存储结构的不同主要在于在线标注数据集使用 Git 版本仓库进行管理，因此此类型的数据集 Bucket 存储结构中额外存在数据集 Git 版本库的裸仓库文件夹（为了避免文件冲突，服务端 Git 仓库需为不包含分支文件目录裸仓库），以及各个分支工作目录文件夹。

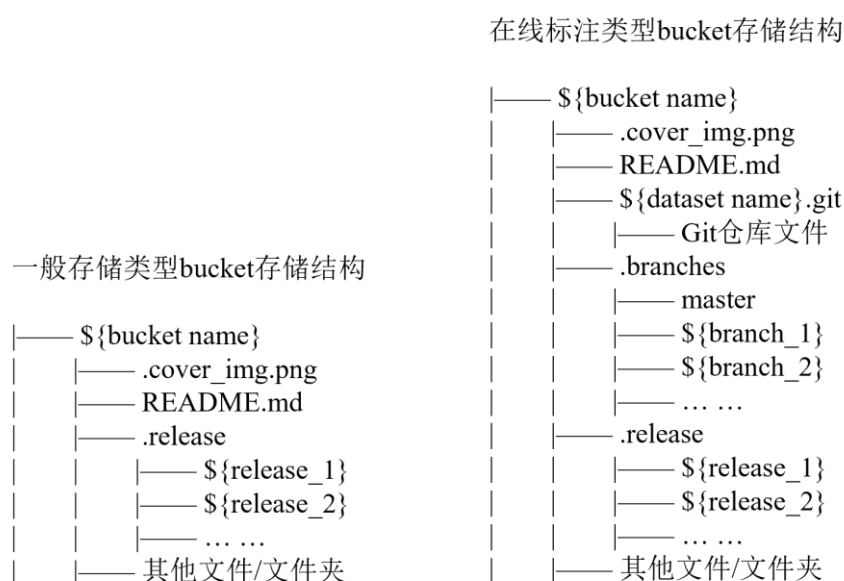


图4.10 数据集 Bucket 存储结构图

4.3.5 项目管理模块设计

项目管理模块主要为用户提供项目管理功能，用户可基于项目需要解决的问题对项目基本信息进行配置，选择项目所采用的深度学习框架的版本，项目训练所使用的数据集，开发环境中模型训练产生模型的存储位置。项目具有公开和私有两种状态。项目模块支持使用 Git 版本管理项目，项目成员可以在自己的分支上开发自己深度学习算法用于模型训练开发。对于项目中已经编码完成的比较成熟深度学习算法，项目所有者或管理员可以基于特定分支执行“`git tag`”打标签标识特定版本，从而发布项目代码的特定版本。项目是启动开发环境容器的基础，基于项目的配置信息生成启动开发环境容器的 `yaml` 文件。项目管理模块也有自己的成员管理体系，不同角色的成员对于项目具有不同的操作权限，如表 4.3 所示。

表4.3 项目角色权限表

角色	权限说明
Owner	1. 管理项目元信息 2. 管理项目成员 3. 管理项目 Git 版本库 4. 发布项目版本
Admin	1. 管理项目 Editor 和 Viewer 成员 2. 管理项目 Git 版本库 3. 发布项目版本
Editor	创建自己的项目分支，进行深度学习算法开发
Viewer	查看使用项目（对于公开项目，平台普通用户默认具有 Viewer 角色）

项目的创建流程与在线标注类型的数据集的创建流程类似，存储在 Bucket 中的项目的存储结构也与在线标注类型的数据集的存储结构类似，用户可以直接通过 Ceph 对象存储服务提供的 API 在线访问特定分支的文件内容。与在线标注类型的数据集不同的是，项目的 Git 版本库 master 分支中会包含默认文件，新创建的项目 clone 到本地后，默认的初始文件结构如图 4.11 所示，包含存储 Git 版本库信息 .git 文件夹、仓库配置信息的 config 文件、描述仓库的 README.md 文件、数据处理 dataprocess.py 文件、main.ipynb 文件和 .gitignore 文件。

项目默认结构

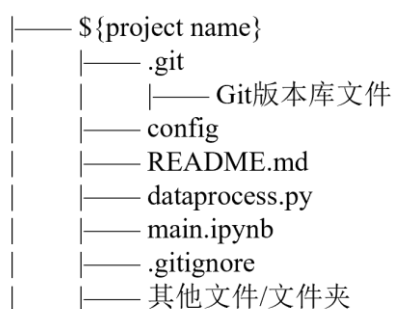


图4.11 项目默认文件结构

4.3.6 开发环境管理模块设计

开发环境管理模块管理用户基于特定项目启动的用于解决特定问题的开发环境容器，此模块基于项目的配置信息和用户申请的内存、CPU、GPU 等资源信息生成启动开发环境容器的 yaml 文件。

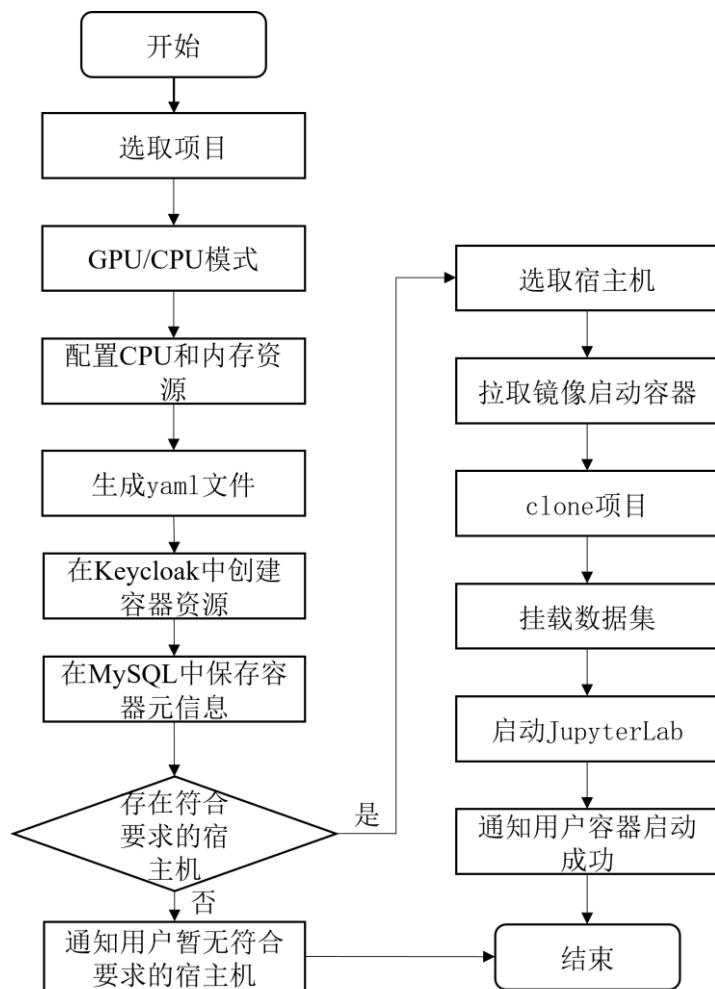


图4.12 开发环境容器创建启动流程图

开发环境容器创建启动流程如图 4.12 所示，用户在创建容器时，首先选取项目，然后再根据项目需求选取采用 GPU 模式还是 CPU 模式，配置容器所需的 CPU 和内存参数，基于项目信息和配置信息生成 yaml 配置文件，深度学习平台根据 yaml 文件中所规定的开发环境容器所需资源，选取满足请求资源要求的宿主机，拉取集成了 PyTorch 或 TensorFlow 不同版本深度学习框架的 JupyterLab 镜像，基于此镜像启动深度学习开发环境容器。开发环境容器会自动从项目的服务端仓库 clone 项目到容器中，并挂载所选数据集。用户可通过浏览器访问 JupyterLab 开发环境，在线编写深度学习算法，在线进行模型训练。JupyterLab 开发环境集成了 Git 版本管理插件，用户可将修改后的项目代码推送至远程项目 Git 版本库。当 JupyterLab 开发环境容器启动后，系统会收集各个 JupyterLab 开发环境容器的日志信息，根据容器日志信息分析容器状态并进行相应的处理。当用户未通过浏览器访问开发环境时，若开发环境后台无任务运行且容器空转运行时常超过一小时，则系统将自动暂停容器；当用户通过浏览器访问开发环境时，若用户虽打开浏览器页面但未进行任何操作且容器后台无任务运行时常超过两小时，系统也会自动暂停开发环境容器。由于 Docker 容器具有秒级的启动

速度，重启容器可以在几秒之内实现，因此可以将不活跃的 JupyterLab 开发环境容器暂停以释放宿主机 GPU、CPU、内存等资源。

4.3.7 模型管理模块设计

模型管理模块主要为用户提供模型文件在线存储和管理功能。模型管理模块的模型文件的来源主要分为两部分：一部分为 JupyterLab 开发环境容器中模型训练生成的模型，一部分为用户直接上传的已有模型。

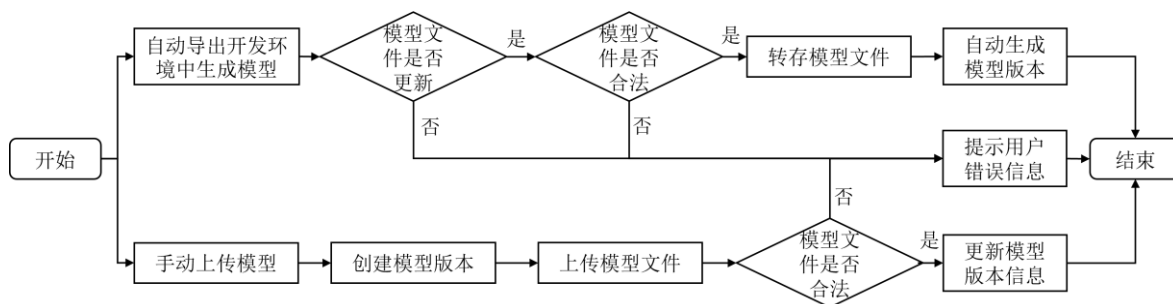


图4.13 模型存储流程图

平台模型存储流程如图 4.13 所示。当用户在配置 JupyterLab 开发环境容器时，若配置了关联的导出模型，则在启动 JupyterLab 开发环境后，Java 后台会自动检测容器中是否生成了模型文件。若检测到模型文件，首先会通过 SHA1 算法生成模型版本 id 检查此模型文件是否已经上传过，然后再检查模型文件格式是否合法，若合法则解析模型文件获取模型参数，将模型转存至对应的模型 Bucket 中，最后生成模型版本信息。用户也可以直接上传已有模型，在模型记录下创建特定的模型版本记录，上传模型文件，系统会自动检测模型文件是否合法并解析模型文件获取模型参数，最后更新模型版本记录。

模型管理模块各个模型记录的创建者也可使邀请其他用户成为本模型的成员，共同管理模型，不同角色成员权限如表 4.4 所示。

表4.4 模型角色权限表

角色	权限说明
Owner	1. 管理模型元信息 2. 管理模型成员 3. 管理模型版本
Admin	1. 管理模型 Viewer 成员 2. 管理模型版本
Viewer	查看使用模型（对于公开模型，平台普通用户默认为 Viewer 角色）

4.3.8 模型托管模块设计

推理服务是在深度学习训练好的模型上通过输入数据进行计算得到输出结果的过程，它是深度学习应用最终落地实现的重要一环，将深度学习模型应用到实际的生产环境中。模型托管模块的主要作用是基于模型管理模块提供的深度学习模型在 Kubernetes 集群中部署模型，通过模型推理 API 接口对外提供推理服务，供平台用户使用。模型托管模块实现深度学习应用从实验开发环境到实际生产环境的转换。

模型部署启动推理服务流程如图 4.14 所示，首先选取模型，根据模型输入输出参数编写数据前处理函数和数据后处理函数，生成服务配置启动 yaml 文件，在 Keycloak 中给推理服务创建对应的资源为后续推理服务 API 接口调用做准备，Java 后台调用特定方法在 Kubernetes 集群 master 节点上执行“kubectl create”命令部署模型启动推理服务；推理服务启动过程需要花费几分钟的时间，在此期间需要监控推理服务的启动状态查看是否启动成功，启动成功后更新 MySQL 数据库中对应推理服务的状态为 Successful，通知用户服务启动成功。

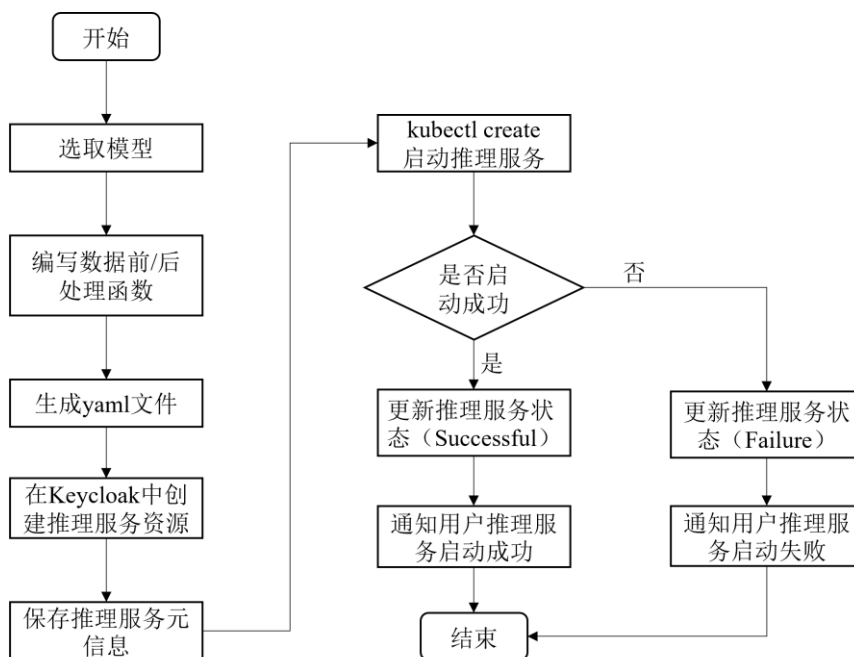


图4.14 推理服务启动流程图

4.4 数据库设计

数据库表设计是软件开发过程的重要环节，它直接影响数据库系统的效率、可靠性和扩展性。通过合理的数据库设计可以降低查询和更新操作的时间复杂度，减少不必要的数据冗余和数据冗余更新，优化索引和查询语句，提高数据库系统的读写效率；合理的数据库设计还可以提高数据的一致性和完整性。

4.4.1 数据库实体关系

数据库设计主要围绕深度学习平台系统主要功能模块进行,所涉及的实体主要包括用户、数据集、模型、项目、开发环境容器、推理服务、镜像和主机。深度学习平台各个实体之间的抽象关系如图 4.15 所示,通过实体关系图,将用户、数据集、模型、项目等各个实体之间的关系列举清楚,方便后续数据库设计和开发过程中,理解数据之间的关系,并确保数据库的设计符合应用程序的需求;其次,实体关系图可以帮助我们快速地找到并修复数据库中的问题,从而减少系统故障的发生;实体关系图还可以帮助我们更好地理解业务中涉及的数据和实体之间的关系,从而更好地制定业务策略和规划。

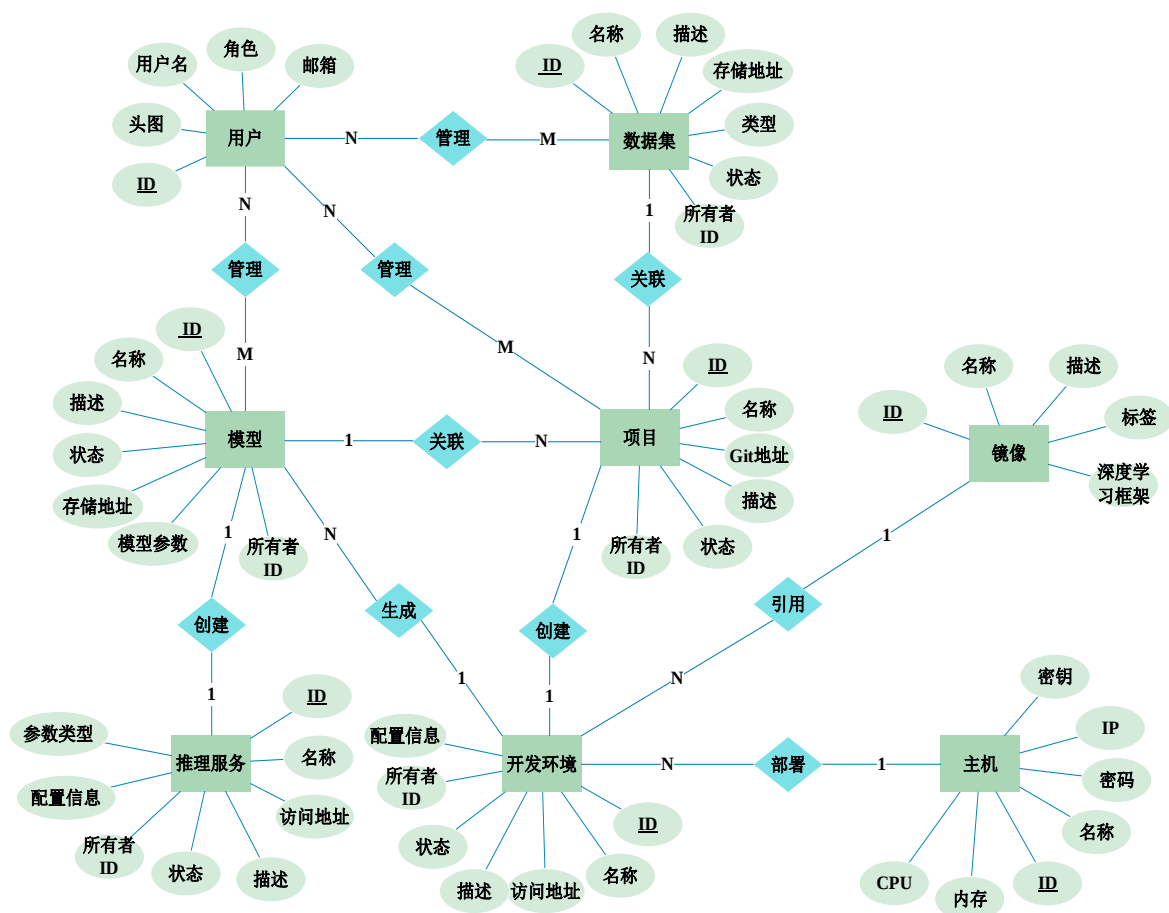


图4.15 深度学习平台 E-R 图

4.4.2 数据库表设计

本部分根据深度学习平台的需求分析及业务流程设计,并借助图 4.15 所示的深度学习平台 E-R 图,在满足业务开发需求的前提下,设计出合理、高效的数据库结构,通过各个数据表保存不同实体数据。下面将对数据表进行简要说明。

(1) 用户表 (users): 该表记录用户的个人信息,主要包含用户 ID、用户名、角

色、邮箱等用户基本属性。

(2) 主机表 (hosts): 该表记录平台提供主机信息, 主要包含主机 ID、用户名、密码、私钥、IP 以及主机 CPU、GPU、内存等主机基本信息。

(3) 镜像表 (images): 该表记录平台提供的镜像信息, 主要包含镜像 ID、镜像仓库名称、镜像名称、镜像描述信息、镜像标签等。

(4) 数据集表 (datasets): 该表记录数据集信息, 主要包含数据集 ID、名称、创建者 ID、存储 Bucket 名称、存储地址、头图地址、服务端 Git 仓库地址、数据集状态等信息。

(5) 数据集分支表 (dataset-branches): 该表记录数据集所包含的各个分支的基本信息, 在线标注的数据集采用 Git 进行版本管理, 成员通过创建自己分支的方式在自己分支进行数据标注。此表主要包含分支 ID、分支名称、分支创建者 ID、分支所属数据集 ID 等信息。

(6) 数据集版本表 (dataset-versions): 该表记录数据集已经发布的版本信息, 用户可以直接使用已经发布的数据集版本进行深度学习训练, 此表主要包含版本 ID、版本别名、版本发布者 ID、版本所属数据集 ID 等信息。

(7) 数据集成员表 (dataset-members): 该表记录数据集的成员信息, 主要包含成员 ID、成员所对应的用户 ID、成员所属数据集 ID、成员角色等信息。

(8) 项目表 (projects): 该表记录项目信息, 主要包括项目 ID、名称、创建者 ID、存储 Bucket 名称、存储地址、头图地址、服务端 Git 仓库地址、项目状态、项目所采用的框架镜像 ID、项目引用数据集 ID 等信息。

(9) 项目分支表 (project-branches): 该表记录项目所包含的各个分支的基本信息, 主要包含分支 ID、分支名称、分支创建者 ID、分支所属项目 ID 等。

(10) 项目版本表 (project-versions): 该表记录项目的版本信息, 主要包含版本 ID、版本别名、版本发布者 ID、版本所属项目 ID 等信息。

(11) 项目成员表 (project-members): 该表记录项目成员信息, 主要包含成员 ID、成员所对应的用户 ID、成员所属项目 ID、成员角色等信息。

(12) 开发环境容器表 (containers): 该表记录开发环境容器信息, 主要包含容器 ID、容器别名、容器 Docker Compose 配置信息、运行容器主机 ID、容器访问地址、容器所依附的项目 ID 等信息。

(13) 模型表 (models): 该表记录模型信息, 主要包括模型 ID、名称、创建者 ID、存储 Bucket 名称、存储地址、状态等信息。

(14) 模型版本表 (model-versions): 该表记录模型的各个版本的信息, 主要包括模型版本 ID、别名、版本创建者 ID、版本所属模型 ID、模型类型、模型的输入输出参数信息、模型文件大小等信息。

(15)模型成员表(model-members):该表记录模型成员信息,主要包含成员 ID、成员所对应的用户 ID、成员所属项目 ID、成员角色等信息。

(16)推理服务表(inferences):该表记录推理服务信息,表明已经落地提供服务的深度学习应用,主要包括推理服务 ID、创建者 ID、使用的模型版本信息、推理服务访问地址、推理服务配置信息等。

下面以项目表、开发环境容器表为例展示部署数据库表的详细内容,如表 4.5、表 4.6 所示。项目表主要根据实际要解决的问题记录项目的一些基本配置信息,包括所采用的数据集、集成特定深度学习框架的镜像等。

表4.5 项目表

字段名称	数据类型	约束	是否可为空	说明
id	VARCHAR (64)	主键	否	唯一标识
name	VARCHAR (128)	索引	否	项目名称
uid	VARCHAR (64)	外键	否	创建者
image_id	VARCHAR (64)	外键	否	项目所采用深度学习框架镜像
model_id	VARCHAR (64)	外键	是	项目输出模型保存位置
dataset_id	VARCHAR (64)	外键	是	项目引用数据集
type	VARCHAR (16)		否	PUBLIC/PRIVATE
bucket_name	VARCHAR (128)		否	存储项目的 Bucket 名称
url	VARCHAR (1024)		否	Ceph 对象存储服务数据访问地址
gti_url	VARCHAR (1024)		否	Git 服务仓库访问地址
cover_img_url	VARCHAR (1024)		是	头图地址
desc	VARCHAR (2048)		是	描述
disabled	TINYINT (1)		否	是否禁用
invalid	TINYINT (1)		否	是否失效
create_time	DATETIME		否	创建时间
update_time	DATETIME		否	更新时间

开发环境表主要记录了开发环境的基本参数,包括是否启用 GPU 以及 GPU 设备的型号、申请的 CPU 和内存资源的额度、启动开发环境 yaml 配置文件的详细信息、开发环境的访问地址等。

表4.6 开发环境容器表

字段名称	数据类型	约束	是否可为空	说明
id	VARCHAR (64)	主键	否	唯一标识
name	VARCHAR (128)	索引	否	名称
uid	VARCHAR (64)	外键	否	创建者
project_id	VARCHAR (64)	外键	否	容器所依托的项目
cpus	FLOAT		否	cpu 核数
mem	FLOAT		否	内存大小, 单位 GB
gpu_enabled	TINYINT (1)		否	是否启用 GPU
gpu_devices	TEXT		是	使用 GPU 信息
jupyter_url	VARCHAR (1024)		否	开发环境访问地址
port	INT (10)		否	开发环境端口号
docker_compose_config	TEXT		是	容器配置信息
cpu_usage	FLOAT		否	CPU 用量
mem_usage	FLOAT		否	内存用量
proc_num	FLOAT		否	进程数
status	VARCHAR (32)		否	容器状态: New、Paused、Failure、Running 等
create_time	DATETIME		否	创建时间
update_time	DATETIME		否	更新时间

4.5 本章小结

本章主要介绍了私有云环境下基于微服务架构的深度学习容器化平台系统总体设计方案, 包括平台的整体架构、平台私有云环境、系统各个功能模块设计等。在平台整体架构方面, 采用了微服务架构来设计系统, 通过服务之间的互相调用, 实现了模块之间的解耦, 从而提高了系统的可扩展性和可维护性。在平台私有云环境搭建方面采用 WireGuard 搭建虚拟内网, 实现系统各个服务器之间安全高效的数据传输, 以及运维开发人员随时随地接入系统所涵盖的所有服务器, 便于系统的维护。在阐述系统功能模块设计时, 重点介绍了主机模块添加主机流程、镜像模块发布镜像流程、数据集模块创建数据集流程、开发环境模块容器启动流程、模型模块模型生成流程和部署模型启动推理服务流程。最后简要概述了平台数据库设计方案。

第五章 深度学习平台系统实现

结合相关技术概述所列各个技术的特点、系统的需求分析以及第四章系统整体设计,本章主要介绍私有云环境下基于微服务架构的深度学习容器化平台系统的详细实现方案,将系统功能设计所提及的系统功能进一步划分为系统基础功能和系统业务功能两大部分,并对其实现进行说明。

5.1 系统基础功能实现

系统的基础功能是系统设计中的重要组成部分,它是一个系统中最基本、最重要的功能之一。基础功能模块为上层业务功能模块的实现起到重要的支撑作用,基础功能模块的稳定性和可靠性将直接影响业务功能模块的性能,进而影响用户体验。本系统的基本功能主要包含以下几部分:

(1) 用户管理功能:用户管理功能是系统的基础功能中最为重要的一部分,它包括最基本的用户注册与登录、用户信息管理,以及一整套完善的权限管理体系,实现对用户访问受保护资源的访问控制,保护系统中的敏感数据,确保平台数据和信息的安全,防止非法访问和数据泄露等安全事故的发生。

(2) 数据存储功能:数据存储功能是系统的核心功能,用于存储深度学习平台中各种资源数据,数据可以说是整个深度学习平台的核心与灵魂。数据存储模块应具有高效的数据检索和读写性能,同时还应确保数据存储的可靠性。此处主要指使用对象存储保存本平台的数据集、模型文件等数据。

(3) 版本控制功能:通过版本控制功能可以帮助用户更好的管理开发代码、数据等资源,追踪、记录其变更历史;在团队协作中,版本控制还可以同步、协调不同成员之间的开发代码和数据,提高团队协作的效率。

(4) 监控功能:系统的监控功能可以帮助系统管理员监控整个系统,及时发现系统的问题和故障。

5.1.1 基于 Keycloak 的用户管理

本文通过整合 Keycloak 以微服务的思想实现符合本深度学习平台要求的包含用户注册、登录、用户信息管理和权限管理在内的用户管理功能。基于微服务架构,将用户管理模块与 Java 后台业务分离,实现两者之间的解耦和,进而提高整个系统的灵活性和稳定性;通过完善的权限管理和鉴权机制,确保平台资源和信息的安全性。

对于用户管理功能,超级系统管理员通过配置 Keycloak 定义不同对象实现对于

平台用户的管理，主要包括 Realm、Client、Group、User、Role。其中 Role 又分为 Realm Role 和 Client Role，本平台通过设置不同的 Realm Role 将用户分为系统管理员和平台普通用户，通过设置不同的 Client Role 对不同资源的成员进行角色区分。基于 Keycloak 实现用户信息管理的主要步骤如下：

(1) 创建 Realm：在 Keycloak 中，一个 Realm 表示一个安全域，它具有隔离用户、角色和客户端的作用。通过配置 Realm 的 Identity Providers，可以设置第三方身份提供商来集成微信登录或 Github 登录。

(2) 创建 Client：在 Realm 中创建不同的 Client，用于表示平台应用服务，此处将创建的 Client 与深度学习平台 Java 后端服务绑定，后续将基于此 Client 自行设计实现细粒度的资源权限管理和访问控制。

(3) 创建 Group：可以在 Realm 中创建不同的 Group，用于管理具有相同属性和相同特性的一类用户。

(4) 创建 User：在 Keycloak 中，可以通过管理员界面或 API 接口来创建用户，也可以使用微信或 Github 等第三方身份提供者注册用户。

(5) 实现登录认证：Java 后台通过封装 Keycloak 核心 API 来实现 KCAdapter 类。当用户访问深度学习平台时，首先跳转到用户登录界面，通过 KCAdapter 类验证用户凭据，返回 token 给前端。当用户携带 token 调用 Java 后台所提供的接口时，Java 通过实现 Authenticator 类来拦截用户的 HTTP 请求，验证用户 token 是否有效，验证通过执行后续 HTTP 请求。

(6) 用户信息管理：通过 Java 后台封装的 KCAdapter 类可以实现平台用户的创建、删除和修改。

表 5.1 为 Java 后台 KCAdapter 类封装的 Keycloak 部分核心 API 介绍。

表5.1 Keycloak 核心 API 列表

URL	Method	作用
/realms/{realm}/users	POST	创建用户
/realms/{realm}/users/{id}	PUT	修改用户
/realms/{realm}/users/{id}	DELETE	删除用户
/realms/{realm}/users	GET	查询用户
/realms/{realm}/protocol/openid-connect/token	POST	请求用户 token
/realms/{realm}/protocol/openid-connect/token/introspect	POST	验证用户 token
/realms/{realm}/authz/protection/resource_set	POST	创建资源
/realms/{realm}/authz/protection/resource_set/{id}	DELETE	删除资源
/admin/realms/{realm}/clients/{client}/roles	POST	创建 Client Role

深度学习平台通过配置 Keycloak 集成了微信登录，用户也可以通过用户名密码来进行登录，图 5.1 为微信扫码登录流程时序图。进入登入界面，PC 端会请求获取二维码，通过轮询二维码状态判断用户是否扫描二维码；扫描二维码后，用户授权将用户信息发送给认证服务 Keycloak，Keycloak 根据用户信息生成临时 token 返回给用户；用户在手机端携带临时 token，确定登陆；Keycloak 生成 PC 端 token，用户可通过此 token 在 PC 端调用平台接口访问其他服务。

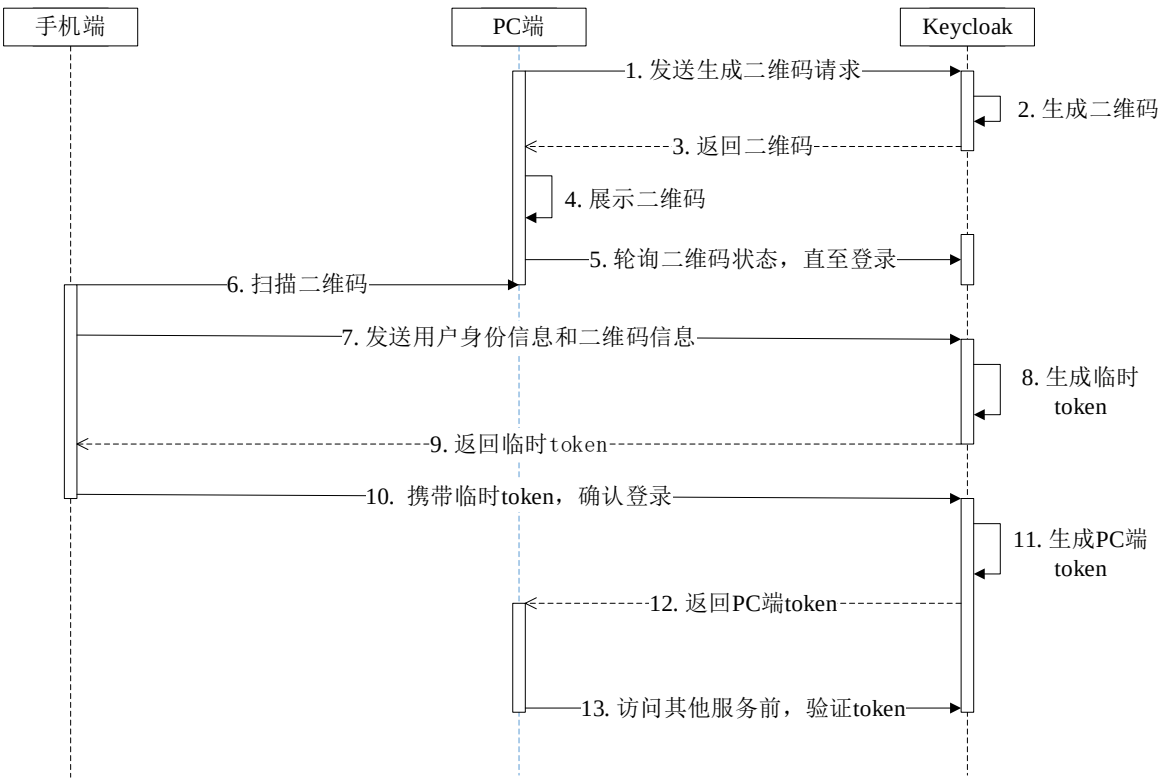


图5.1 微信扫码登录时序图

平台登录界面如图 5.2 所示，平台支持微信登陆和 Github 登陆。



图5.2 平台登录界面

当用户通过微信扫码登录或使用用户名密码登录后，会返回给前端一个包含用户信息的 token，当用户携带 token 调用 Java 后台的 API 接口时，Authenticator 类会在

后台接收到用户 HTTP 请求时拦截用户请求，从请求头中获取 token，首先查看缓存是否含有此用户信息，若无则调用 KCAdapter 类的 VerifyAccessToken 验证用户 token 是否有效并解析 token 获取用户信息，更新缓存。若 token 无效，返回状态码 401，提示用户需要进行身份认证。相关类图如图 5.3 所示。

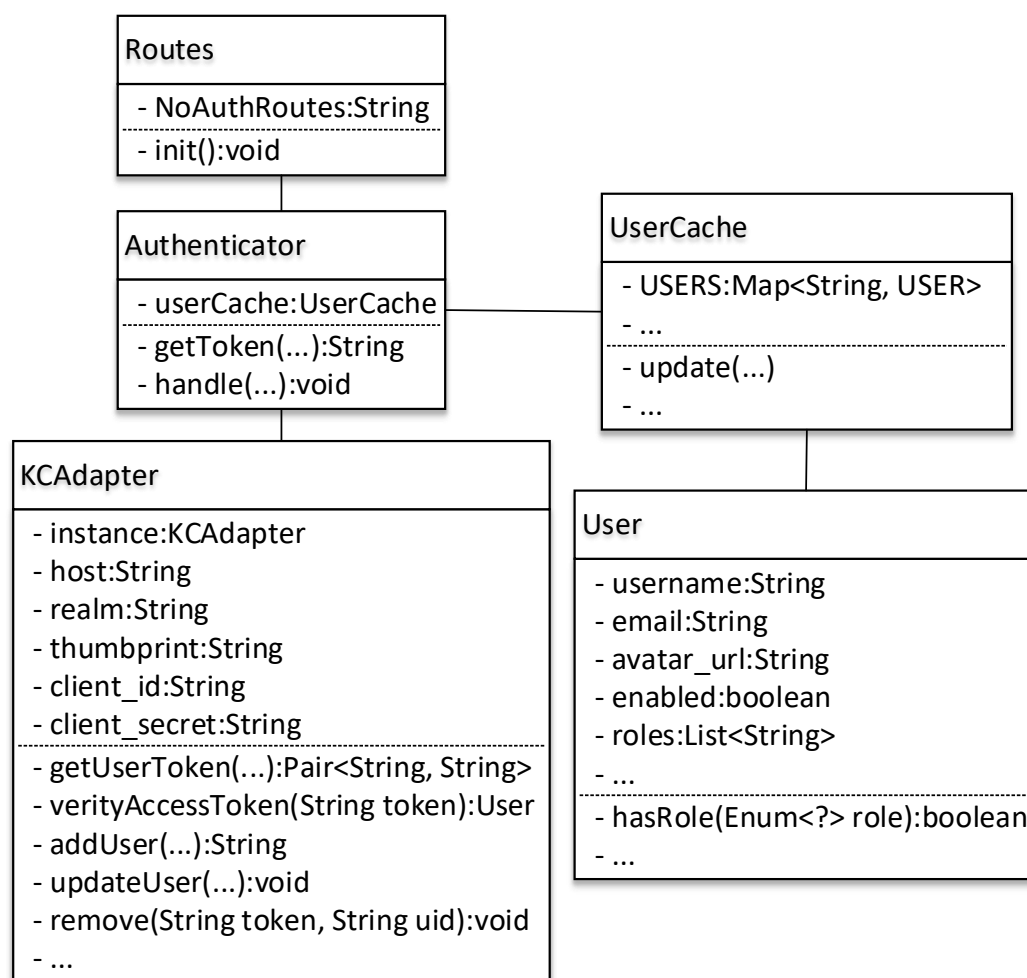


图5.3 身份认证相关类图

对于权限管理功能，为了确保特定受保护资源实体只有具有特定权限或角色的用户可以访问，实现对平台受保护资源实体的精细化权限管理。此处通过 Keycloak 提供的 Resource、Permission、Policy、Scope 和 Role 等对象，自行设计实现基于角色访问控制策略（RBAC）的权限管理和访问控制机制。此处的资源是指深度学习平台中所具有的资源实体，如项目、数据集、模型等；Resource 则指 Keycloak 中的对象。

首先，对深度学习平台中需要进行资源访问控制的对象，根据其特性以及可以对其执行的所有操作，设置对应 Scope 集，用以表示用户可以通过客户端访问资源时，能够访问的内容或执行的操作。根据对资源实体可执行的操作类型，大致可以将 Scope 分为与资源本身相关的 Scope、与分支操作相关的 Scope、与用户相关的 Scope 等。Scope 与具体操作的映射关系如表 5.2 所示。

表5.2 Scope 列表

类型	名称	说明
与资源本身相关的 Scope	view	获取并查看资源实体信息
	updateInfo	更新资源实体元信息
	delete	删除资源实体
	updatePublicStatus	修改资源实体对外状态
	transferOwner	转移资源实体所有权
	enabling	禁用/启用
与分支操作相关的 Scope	viewBranch	查看分支
	createDevBranch	创建开发分支
	deleteDevBranch	删除开发分支
	pushDevBranch	本地 push 到远程开发分支
	pushMasterBranch	本地 push 到远程 Master 分支
	manageTag	管理基于分支发布的版本
与用户相关的 Scope	getMember	获取成员信息
	setMember	添加/删除成员

其次，将深度学习平台中的资源实体与 Keycloak 中的 Resource 对象绑定，用于表示被保护的资源实体，通过自行设计的访问控制机制结合 Keycloak 的鉴权机制保护平台中各种资源实体，确保只有被授予权限的用户才能访问具体资源，执行特定操作。Resource 对象 JSON 模板样例如图 5.4 所示。

```
{
  "name": ${resource_name},
  "type": ${type},
  "owner": ${uid},
  "ownerManagedAccess": true,
  "_id": ${resource_id},
  "resource_scopes":[
    "scope_1",
    "scope_2",
    ...
  ],
  "attributes":[
    "key_1":"value_1",
    "key_2":"value_2",
    ...
  ]
}
```

图5.4 Resource 对象 JSON 模板样例

其中 `type` 字段表示资源实体的类型，这里采用 `PUBLIC/PRIVATE` 设置该字段，用以表示资源实体是公开还是私有；`owner` 字段表示 `Resource` 对象创建者的用户 ID，若不设置则 `Resource` 的所有者默认为资源服务器；`ownerManagedAccess` 字段表示是否允许资源所有者管理对该资源的访问；`resource_scopes` 字段表示客户端用户在访问此资源实体时能够访问的内容或执行的操作；`attributes` 字段表示 `Resource` 可能具有的额外属性，可用于提供有关资源的附加信息。

然后，通过设置 `Policy` 对象规定资源实体访问控制规则。结合深度学习平台功能设计，平台主要通过用户角色进行权限设定，此处采用基于角色的访问控制策略（RBAC），配置 `Role-based Policy`。以项目资源实体为例，用户角色主要分为系统层面的角色（系统管理员、平台普通用户）、项目资源实体层面的成员角色（`Owner`、`Admin`、`Editor`、`Viewer`）。系统层面的角色通过 `Realm Role` 进行设置，项目资源实体层面的成员角色通过 `Client Role` 进行设置。`Role-based Policy` 对象 JSON 模板样例如图 5.5 所示。通过设置 `roles` 字段规定访问控制规则，即通过用户角色限制访问权限；`logic` 字段可以为 `POSITIVE` 或 `NEGATIVE`，`POSITIVE` 表示策略的最终评定结果保持不变，`NEGATIVE` 表示策略评定结果的否定为最终结果。

```
{
  "roles":[
    {
      "id":${role_id},
      "required":true
    }
  ],
  "name":${policy_name},
  "description": "",
  "logic": "POSITIVE/NEGATIVE"
}
```

图5.5 Role-based Policy 对象 JSON 模板样例

最后，通过 `Permission` 对象将 `Resource` 对象与 `Policy` 对象进行绑定，规定哪些用户可以访问此资源实体，并允许执行哪些操作。`Resource-based Permission` 可以配置用户在满足访问策略的前提下，拥有某个资源实体或某类资源实体的所有操作权限；`Scope-based Permission` 则允许用户在满足访问策略的条件下，拥有某个资源实体或某类资源实体的特定操作权限。根据不同角色的特性，采用不同类型的 `Permission` 进行配置。如表 5.3 所示，系统管理员具有平台中所有资源实体的所有操作权限，不论是 `PUBLIC` 还是 `PRIVATE` 类型的资源实体，因此采用 `Resource-based Permission`；平台普通用户只对 `PUBLIC` 类型的资源实体最基本的查看和使用权限，对 `PRIVATE` 类型的则无任何权限，因此采用 `Scope-based Permission`；资源实体的成员用户针对特定资

源实体根据其角色不同具有不同的操作权限，因此采用 Scope-based Permission。

表5.3 角色-Permission 对照表

角色	Permission 类型	是否使用 resourceType	说明
系统管理员	Resource-based Permission	是	系统管理员具有平台中所有资源实体的所有操作权限
平台普通用户	Scope-based Permission	是	普通用户只对平台中 PUBLIC 的资源实体具有部分权限
资源实体成员角色	Scope-based Permission	否	资源实体成员角色根据角色不同对特定资源具有不同的权限

如图 5.6 所示,为不同角色类型所采用的 Permission 对象 JSON 模板样例。其中，对于系统管理员所采用的模板，通过 policies 字段与 resourceType 字段将一个或多个 Policy 与某一类 Resource 绑定；对于平台普通用户有采用的模板，采用相同的方式将 Policy 与 Resource 绑定，但通过 scopes 字段开放部分操作权限；对于资源实体成员用户所采用的模板，通过 policies 字段与 resources 字段将 Policy 与一个或多个特定的 Resource 绑定，并通过 scopes 字段开放特定操作权限。

系统管理员	平台普通用户	资源实体成员角色
<pre>{ "policies":[\${policy}], "resourceType":\${type}, "name":\${permission}, "description": "", ... }</pre>	<pre>{ "resourceType":\${type}, "policies":[\${policy}], "scopes":[\${scope}], "name": "\${permission}", "description": "", ... }</pre>	<pre>{ "resources":[\${resource}], "policies":[\${policy}], "scopes":[\${scope}], "name": "\${permission}", "description": "", ... }</pre>

图5.6 Permission 对象 JSON 模板样例

受保护资源实体创建流程如图 5.7 所示，当受保护实体资源创建后，首先，会根据图 5.4 所示 Resource 对象 JSON 模板创建 Resource 对象；其次，根据资源实体创建从属于此资源实体的 Client Role；然后，根据图 5.5 所示 Policy 对象 JSON 模板创建 Policy 对象，设置资源实体访问规则；随后，根据图 5.6 所示 Permission 对象 JSON 模板，结合不同的角色类型，创建不同的 Permission 对象，将 Resource 对象与 Policy 对象绑定，明确在满足 Policy 对象随设置的条件后，用户可以对 Resource 对象随对应的资源执行哪些操作权限；最后，给成员用户绑定不同的 Clie Role。

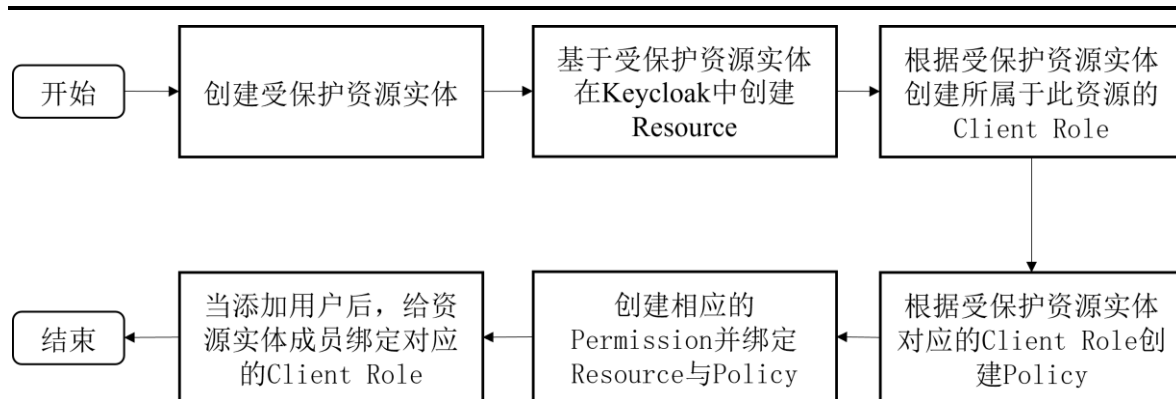


图5.7 受保护资源实体创建流程图

用户访问客户端调用 Java 后台接口的整体鉴权流程如图 5.8 所示, 本文将 API 路由与特定资源请求所需的 Scope 进行一一对应, 当用户访问受保护资源实体时, 在原有 API 认证流程的基础上, Authenticator 类在验证完用户 token 是否有效后, 会查询路由与 Scope 对应表, 调用 KCAdapter 类的方法获取 Requesting Party Token 并验证用户是否具有访问特定资源实体或执行某项操作的权限。

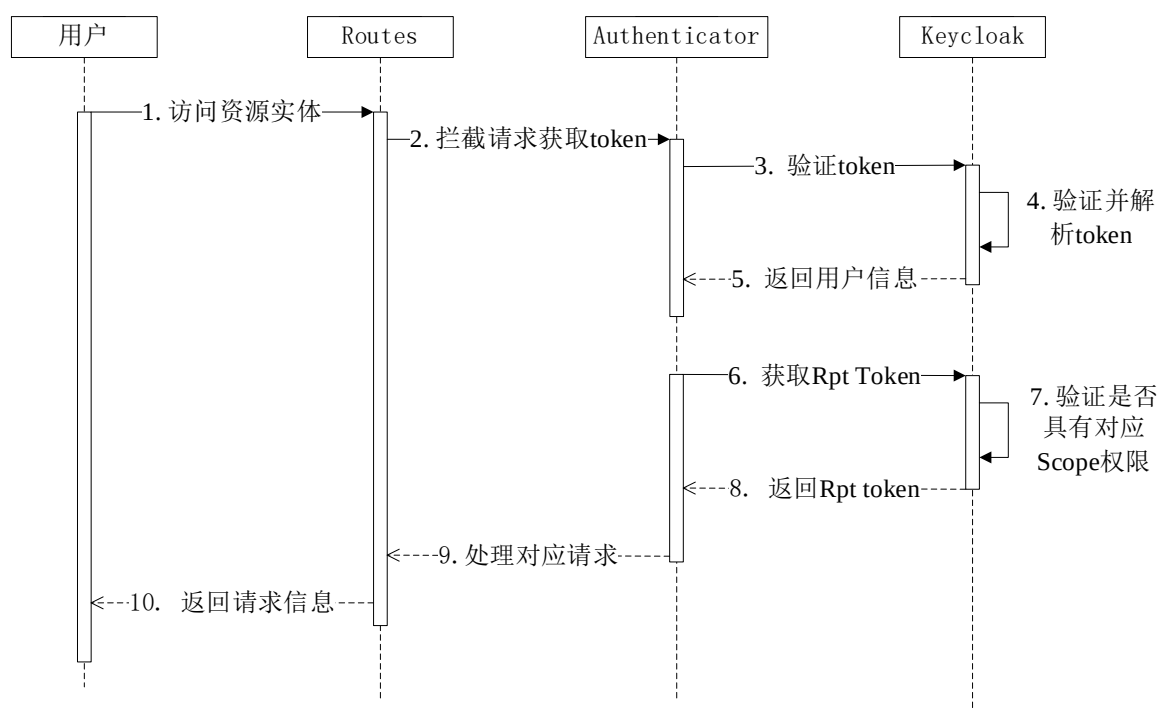


图5.8 Java 后台鉴权时序图

5.1.2 基于 Ceph 的分布式对象存储

本文通过整合基于 Ceph 提供的对象存储服务, 用于存储本深度学习平台的各种资源文件, 解决数据存储空间扩展性差、扩展成本高、大规模数据管理困难、数据检索效率差等问题; 同时, 对象存储通过多副本和备份的形式保证数据的可靠性和可用

性,可实现数据自动备份和数据恢复;另外,对象存储支持定制化的数据访问控制策略,可根据业务需求自定义数据访问规则,确保平台数据安全性。

平台对象存储服务大致可分为三部分:Java 后台以 Ceph Admin 的身份管理对象存储服务中的数据、Ceph RGW 的 STS 服务给 Keycloak 用户发放临时凭证以访问对象存储服务中的数据、将对象存储服务中的数据挂载为本地文件系统便于平台中其他功能模块快捷访问数据。Ceph 对象存储服务集成 Amazon S3 并兼容 Amazon S3 的 RESTful API,Java 后台通过使用 AWS Java SDK 实现 S3Adapter 类对 Amazon S3 核心方法进行封装,从而实现对 Ceph 集群的访问和数据管理,对应类图如图 5.9 所示。

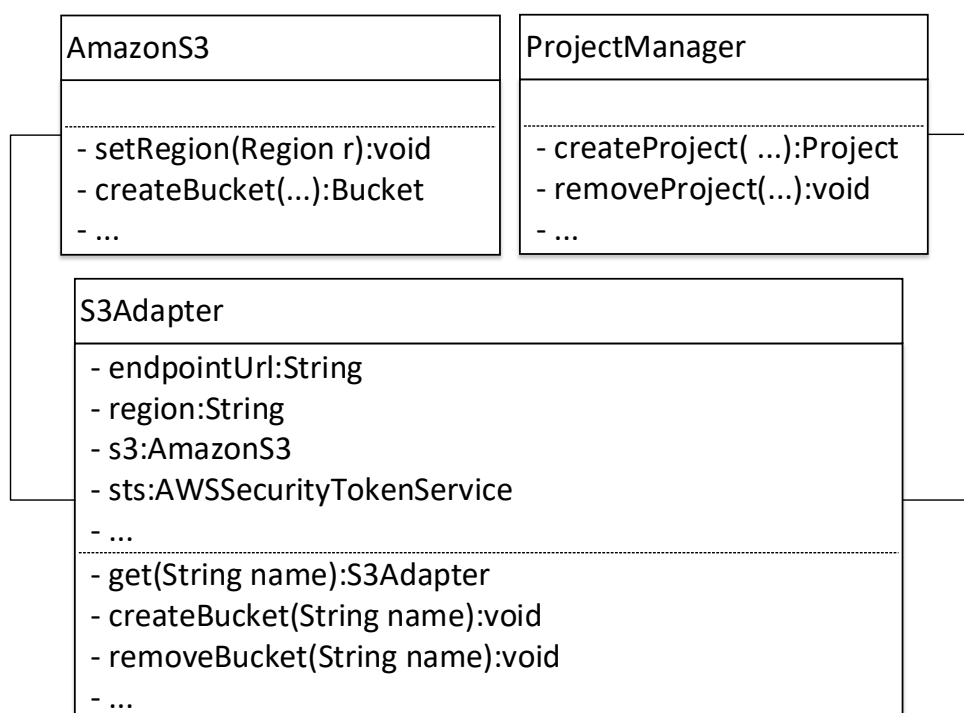


图5.9 Ceph 对象存储相关类图

Java 后台通过调用 S3Adapter 类以 Ceph Admin 身份创建 Bucket 时,采用 Canned ACL 管理 Bucket 和 Object 的访问权限。Canned ACL 是 Amazon S3 支持的预定义授权策略,如表 5.4 所示。

表5.4 Canned ACL 表

Canned ACL	适用范围	绑定的 ACL 权限
private	Bucket 和 Object	所有者具有 FULL_CONTROL 权限，其他人无访问权限
public-read	Bucket 和 Object	所有者具有 FULL_CONTROL 权限，其他人具有 READ 权限
public-read-write	Bucket 和 Object	所有者具有 FULL_CONTROL 权限，其他人具有 READ 和 WRITE 权限
bucket-owner-read	Object	Object 所有者具有 FULL_CONTROL 权限，Bucket 所有者具有 READ 权限
bucket-owner-full-control	Object	Bucket 所有者和 Object 所有者具有 FULL_CONTROL 权限

ACL 权限与 Action 的映射关系如表 5.5 所示。

表5.5 ACL 权限与 Action 映射关系表

ACL 权限	与 Bucket 绑定的映射关系	与 Object 绑定的映射关系
READ	s3:ListBucket s3:ListBucketVersions s3:ListBucketMultipartUploads	s3:GetObjects s3:GetObjectVersion
WRITE	s3:PutObject	不适用
READ_ACP	s3:GetBucketAcl	s3:GetObjectAcl s3:GetObjectVersion
WRITE_ACP	s3:PutBucketAcl	s3:PutObjectAcl s3:PutObjectVersion
FULL_CONTROL	等同于具有 READ、WRITE、READ_ACP 和 WRITE_ACP 的权限	等同于具有 READ、READ_ACP 和 WRITE_ACP 的权限

在 Ceph 中，STS（Short-term Token Service）是一项用于生成短期访问令牌的服务，授予用户临时权限，在 Ceph 集群中执行特定的操作，例如读取、写入、删除、创建等操作。STS 使用了 AWS STS（Amazon Web Services Security Token Service）的 API 接口和认证机制，可以在 Ceph 集群中实现类似的短期访问令牌服务。通过将其与 Keycloak 进行结合，授予平台用户访问 Ceph 中数据的权限，具体配置流程如下：

(1) 配置 Ceph RGW 的 STS: 通过配置 `rgw sts token introspection_url` 参数, 指定 Keycloak 为验证短期访问令牌的 OIDC 提供商 (OpenID Connect Provider), 在使用 STS 服务生成短期令牌后, RGW 会通过 `introspection_url` 向 Keycloak 验证短期访问令牌的有效性;

(2) 在 Ceph RGW 中注册 OIDC Provider: 配置 Keycloak 服务 URL、thumbprint 等信息;

(3) 创建 Ceph Role: 给 Keycloak 用户创建对应的 Ceph Role 并配置 AssumeRolePolicyDocument 信息, 将 Keycloak 身份提供商作为可访问 Ceph Role 的实体, 确保 Role 仅被授权的实体使用, 并防止未被授权的实体访问敏感信息和资源;

(4) 创建并绑定 IAM Policy: 通过给 Role 绑定 IAM Policy 规定此 Role 具有的访问权限。

如图 5.10 所示, Ceph RGW 的 STS 服务授权流程如下:

- (1) 用户通过 Ceph 的认证服务 Keycloak 进行身份验证获取访问令牌 Id Token;
- (2) 使用 Id Token 调用 AWS STS 的 API 接口请求生成临时访问凭证 (Access Key、Secret Key 和 Session Token);
- (3) AWS STS 检查用户身份和授权信息, 发放临时凭证;
- (4) 后台服务器将临时凭证转发给客户端用户;
- (5) 用户使用临时凭证访问 Ceph 集群中的特定资源;
- (6) Ceph 集群返回结果。

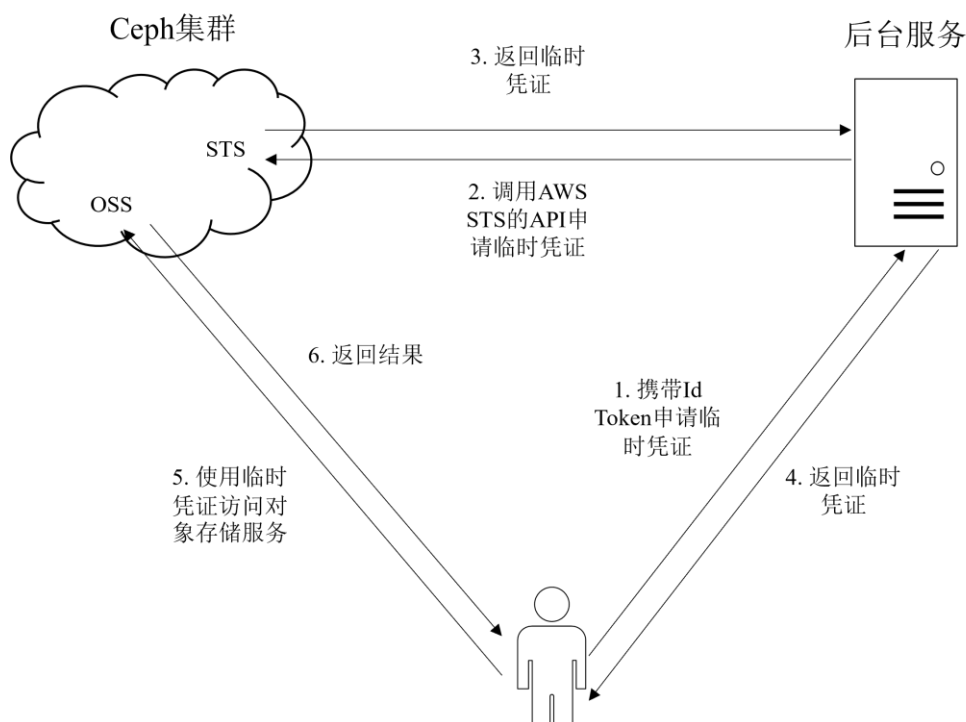


图5.10 STS 授权及使用流程图

平台支持将 Ceph 对象存储服务中的数据挂载为本地文件系统，为平台其他模块提供数据支持。数据挂载主要在 JupyterLab 开发环境容器启动后以只读的形式挂载模型训练所需的数据集和之后小节基于 Ceph 和 JGit 实现的版本管理模块使用。数据挂载通过 S3FS 实现，将对象存储通过 FUSE 挂载到特定的文件目录下。

5.1.3 基于分布式对象存储的 Git 版本管理

版本管理是平台用于管理项目和在线标注数据集的基础模块，用以实现数据的版本管理和支持 Smart HTTP 协议的 Git HTTP 服务，从而支持用户使用 Git 命令管理数据。版本管理具有众多优点，如：

(1) 历史记录追踪：它可以记录所有文件的修改记录，即何人在何时修改了哪个文件的哪些内容；

(2) 多人协作：它允许多人在同一时间修改同一份文件而不会产生冲突；

(3) 版本回滚：允许将文件回滚到之前的版本，例如当新版本程序代码出现问题或错误时，可以将其回滚到上一个稳定版本。

版本管理模块使用 Ceph 对象存储服务将服务端不同的 Git 版本仓库存储至不同的存储桶中，通过对象存储服务的挂载机制，将存储桶中的 Git 版本仓库映射到 Git 后台服务所在服务器的文件系统中，进而实现对存储在对象存储服务中 Git 版本仓库的管理，如图 5.11 所示。



图5.11 仓库存储示意图

自行实现的 Git 后台服务基于 Eclipse 基金会开发的、用户操作 Git 的纯 Java 库 JGit 实现，通过定义 AuthFilter、RepoResolver、RepoReceivePackFactory 类分别实现 Filter、RepositoryResolver、ReceivePackFactory 接口，从而实现用户认证与鉴权、服务端 Git 版本库解析、服务端 Git 仓库接收数据前后的数据处理等功能；通过 GitServlet 类定义整个 Git HTTP 服务，实现 Git HTTP 请求的服务端功能。图 5.12 为版本管理模块实现类图。

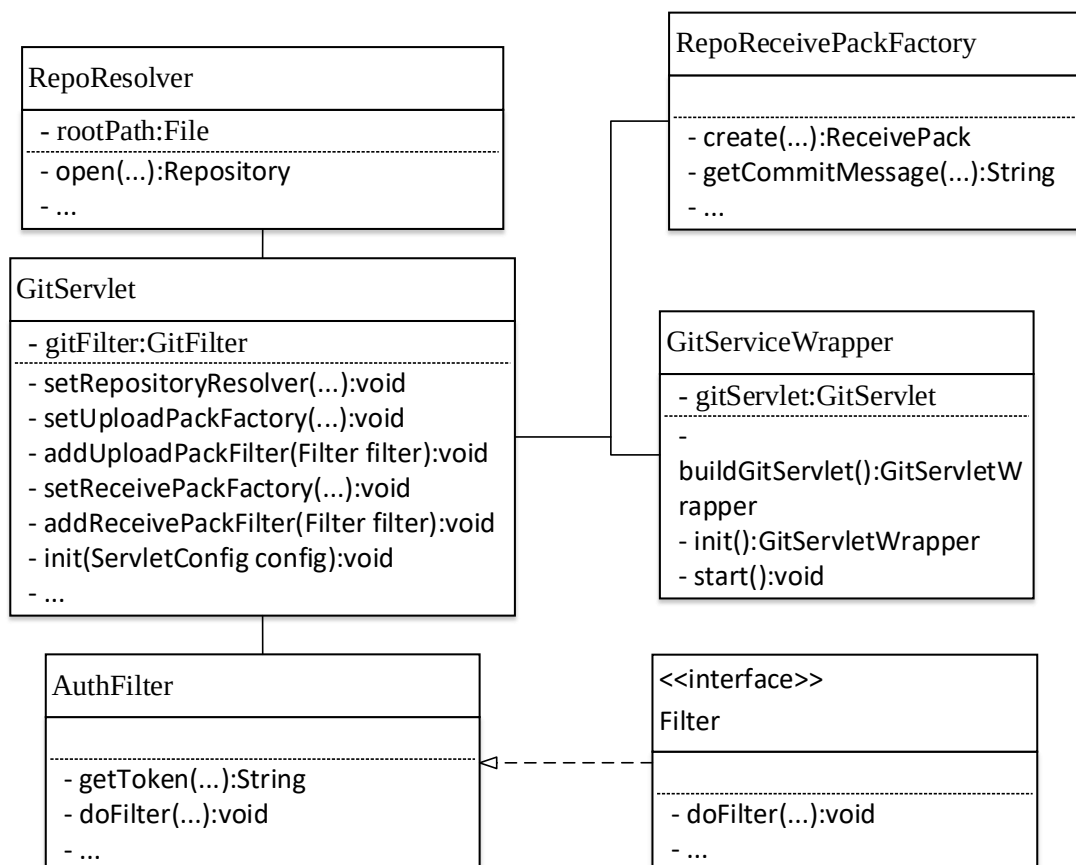


图5.12 版本管理相关类图

对于服务端来说，Git HTTP 服务主要分为以 push 为代表的 Receive-Pack 过程和以 clone、pull、fetch 为代表的 Upload-Pack 过程。每个过程包含两次 HTTP 请求，分别为引用发现（GET）和数据传输（POST）。此处以 push 操作为例介绍 Git HTTP 服务的实现过程。

如图 5.13 所示，当服务端接收到第一次请求时，服务端执行流程如下图所示：

（1）响应请求通过 NoCacheFilter 设置 HTTP 请求头禁用缓存；

（2）RepositoryFilter 通过调用 RepositoryResolver 类判断存储服务端 Git 仓库的 Bucket 是否在服务端挂载，若未挂载则挂载 Bucket，解析 Git 仓库文件，获取 Repository 对象；

（3）ReceivePackFilter 调用 AuthFilter 类进行用户认证和鉴权。AuthFilter 首先会判断请求是否采用 Basic Access Authentication 的方式即是用用户名密码的方式进行认证。若采用此方式，则会向 Keycloak 请求获取用户 token；若请求直接携带用户 token，则验证 token 的有效性。然后，根据用户角色结合基于 Keycloak 实现的细粒度权限管理判断用户是否有权执行此操作。

（4）InfoRefsServlet 获取服务端 Git 仓库的引用信息返回给客户端。

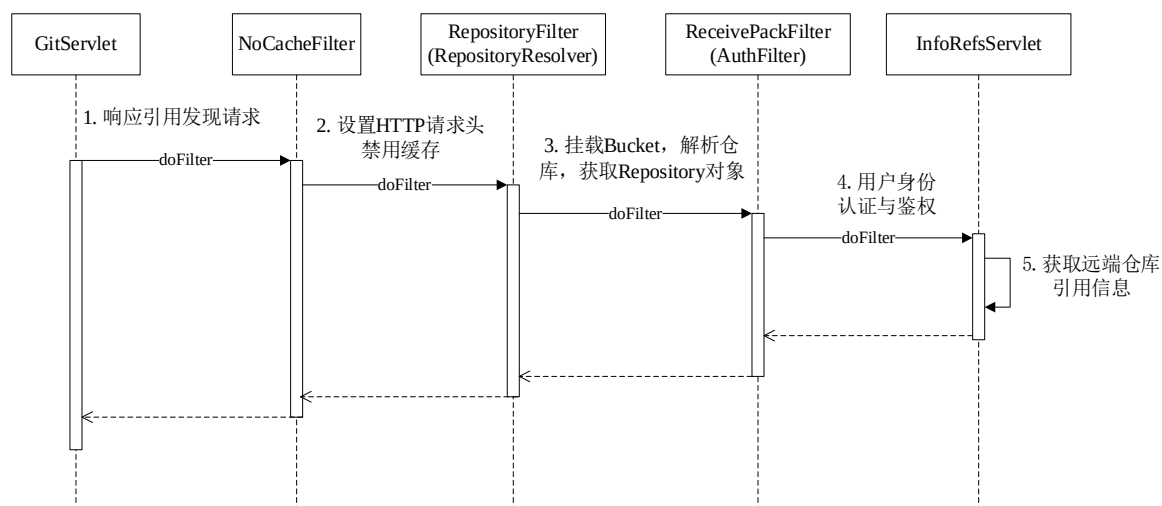


图5.13 Push 操作第一次请求引用发现流程图

客户端根据第一次请求获取的服务端 Git 仓库的引用信息后，客户端计算出服务端仓库所缺失的数据，通过 POST 请求将数据打包发送给服务端。服务端接收数据并更新服务端仓库的引用信息，如图 5.14 所示。由于服务端端的 Git 仓库为裸仓库并不包含工作目录文件，为了实现通过 Ceph RGW 直接访问 Bucket 中的数据，在线查看服务端仓库不同分支下的文件。服务端通过 JGit 库实现 Post-Receive Hook，在整个流程结束后，若此次 push 操作对象为分支，则给新创建的分支创建 *worktree* 用于展示服务端仓库不同分支的内容或更新 *worktree* 内容，即服务端仓库的每个分支都有对应的 *worktree* 用于展示分支内容；若此次 push 操作的对象为 tag，则根据 *commit_id* 将对应的版本检出到 *bucket* 的 */.release* 路径下，发布对应版本。

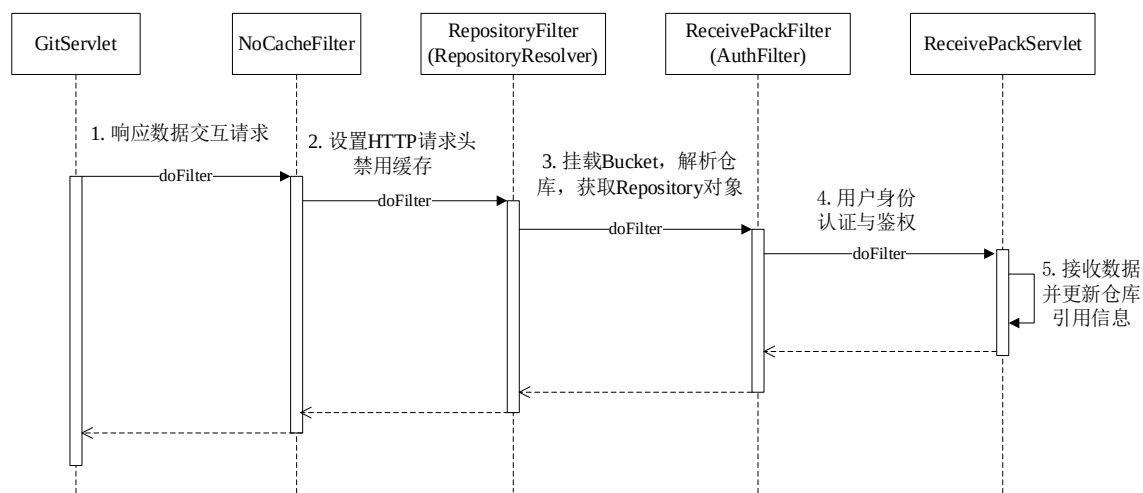


图5.14 Push 操作第二次请求数据交互流程图

5.1.4 监控模块实现

监控模块在一个系统中具有十分重要的作用，它可以对系统中各个组件、服务以

及系统的整体运行情况、资源使用情况进行实时监控，收集和分析数据，及时发出预警等。

本平台的监控模块通过 **Exporter + Prometheus + Grafana** 实现，**Exporter** 作为采集组件从目标服务或服务器搜集数据，并将其转化为 **Prometheus** 支持的格式；**Prometheus** 作为一种基于时间序列数据库的监视系统，通过 **HTTP** 协议周期性抓取采集组件搜集的数据，并提供对数据的存储、分析、查询功能；**Grafana** 作为可视化工具，将 **Prometheus** 中数据以图标、仪表盘等形式进行展示。

平台通过此监控系统监控系统中的以下服务：**MySQL**、**Redis**、**OpenResty**、**Ceph**，以及系统中的各个服务器。监控模块部分可视化界面如图 5.15 所示。

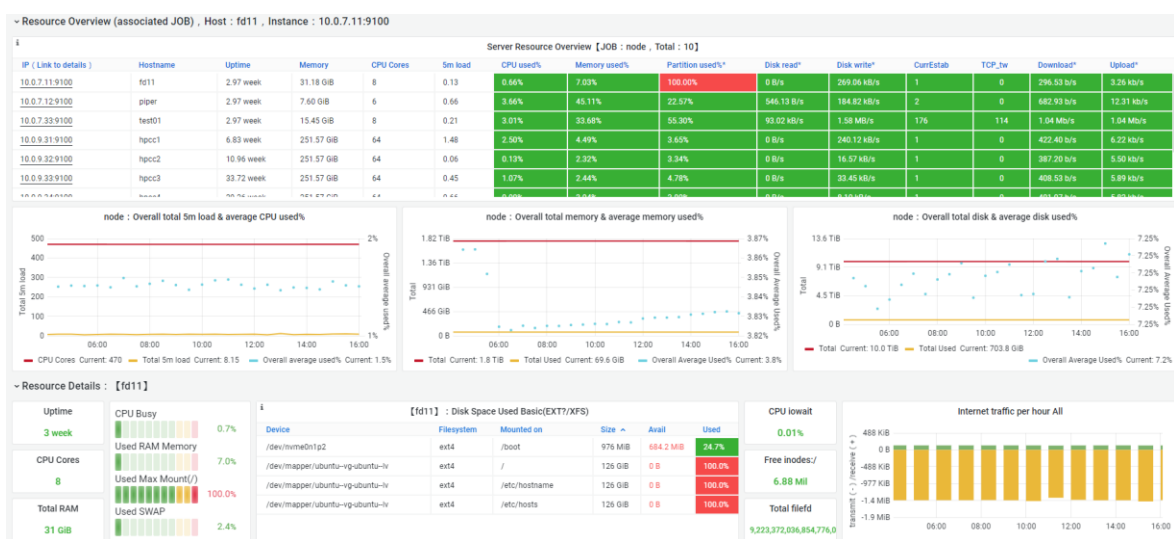


图5.15 监控模块部分可视化界面

5.2 系统业务功能实现

系统的业务功能是在基础功能的基础上进行实现，系统的业务功能主要包含主机管理、镜像管理、数据集管理、项目管理、开发环境管理、模型管理和模型托管。下面将会根据各个功能模块的设计逻辑详细介绍每个模块主要功能的实现逻辑，以及展示部分功能页面。

5.2.1 主机管理模块

主机管理模块是系统十分重要的模块之一，它管理着本平台用以部署用户开发环境容器的服务器以及部署模型启动推理服务实现深度学习应用真正落地的 **Kubernetes** 集群服务器。通过主机管理模块可以远程连接到服务器，执行所需的 **Shell** 命令管理服务器，主机管理模块相关类图如图 5.16 所示。

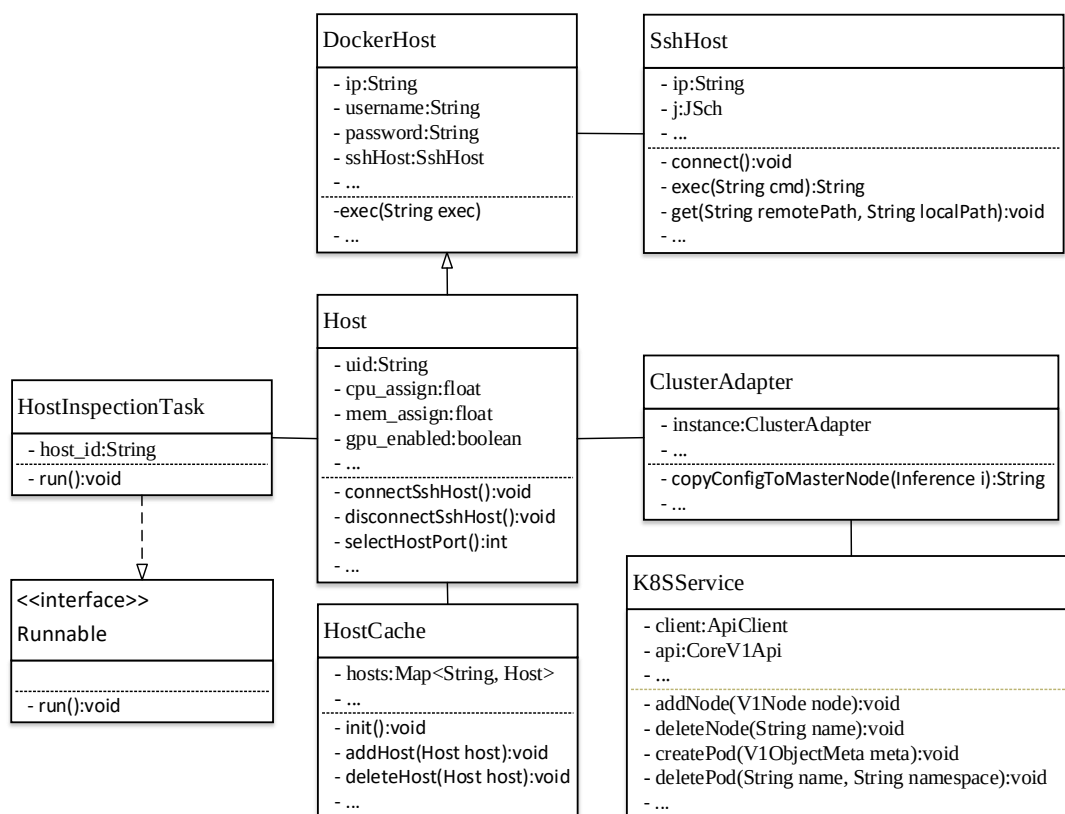


图5.16 主机管理模块相关类图

主机管理模块的核心功能就是管理平台提供给用户使用的共享服务器，此模块只对系统管理员开放。Host 类用于记录平台中所有提供给用户使用的共享服务器，记录每个服务器的 ip、username、password 和主机资源使用情况等信息，并将主机信息保存至 MySQL，Host 类继承自 DockerHost 类；SshHost 类则通过引入 JSch 库实现连接远程服务器的功能，JSch 是一个纯 Java 的 SSH2 实现库，通过 JSch 提供的简单易用的 API 实现与远程服务器进行通信，执行远程 Shell 命令和文件传输；HostInspectionTask 类则通过实现 Runnable 接口以周期任务的形式获取服务器 CPU、GPU、MEM、IO 和 NET 时序信息，查看主机资源使用情况。ClusterAdapter 类与 K8SService 相互配合，通过引入 Kubernetes-client 库实现对 Kubernetes 集群的管理，Kubernetes-client 是一个 Java 编写的 Kubernetes API 客户端，它实现了 Kubernetes API 的所有核心概念，提供了完善的 API 调用和状态更新接口，可实现 Java 应用程序与 Kubernetes 集群进行通信，从而管理 Kubernetes 集群。

主机管理模块会为用户提供合适的服务器用以部署用户开发环境容器，主机管理模块会根据开发环境容器所申请的资源结合自身主机分配规则合理为开发环境容器选取宿主机。主机选取流程如图 5.17 所示。

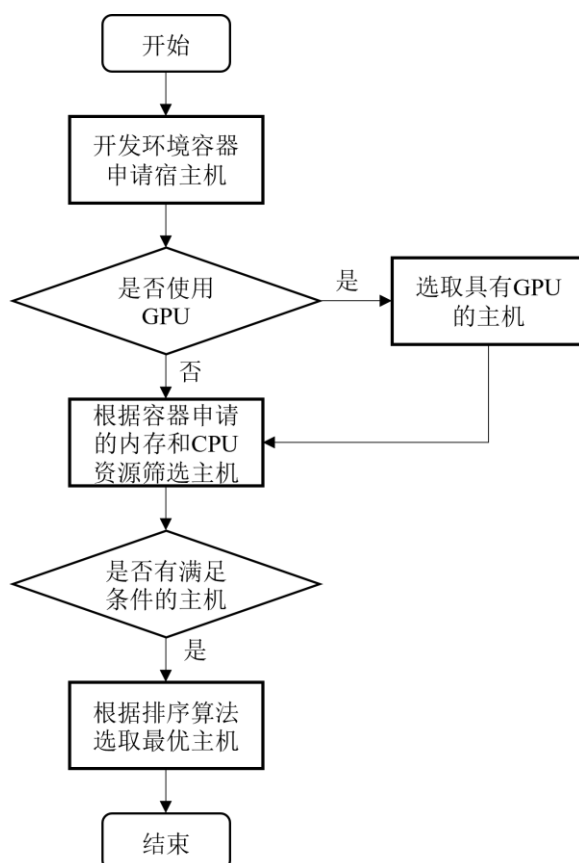


图5.17 主机选取流程图

- (1) 用户设置开发环境所需的主机资源额度，并发起请求为该容器申请主机；
- (2) Java 后台接收到 HTTP 请求验证用户身份后，将处理请求转发至主机管理模块，根据是否使用 GPU，若使用 GPU，则首先对主机列表中的主机进行筛选，选取有闲置 GPU 的主机；

- (3) 根据该容器申请的内存和 GPU 资源额度进一步筛选主机，筛选公式如下：

$$hostCpus - cpu - (hostGpus - gpu) \times CpuRes \geq 0 \quad (5-1)$$

$$hostMem - mem - (hostGpus - gpu) \times MemRes \geq 0 \quad (5-2)$$

其中， $hostCpus$ 、 $hostGpus$ 和 $hostMem$ 分别表示主机当前空闲的 CPU 核数、空闲 GPU 数和空闲内存数； cpu 、 gpu 和 mem 表示容器申请的 CPU、GPU 和内存额度； $CpuRes$ 和 $MemRes$ 表示为空闲 GPU 需要预留的 CPU 和内存额度。

- (4) 若存在满足条件的主机，则根据排序算法对主机进行排序，选取权重最大的最优主机，权重计算如下：

$$wight = hostMem \times MemWight + hostCpus \times CpuWight \quad (5-3)$$

其中， $MemWight$ 和 $CpuWight$ 分别表示每 GB 内存的权重和每核 CPU 的权重。

主机管理界面如图 5.18 所示。



图5.18 主机管理界面

5.2.2 镜像管理模块

镜像管理模块的主要功能是为用户提供可以启动集成了不同深度学习框架的 JupyterLab 开发环境的镜像，使得用户可以通过配置文件一键在宿主机上部署自己的开发环境容器。本平台通过部署 Docker Registry 作为平台私有的镜像仓库，Docker Registry 作为镜像仓库用于存储和分发 Docker 镜像，它提供了一系列 HTTP API 用于 Docker 客户端或其他 HTTP 客户端与之进行交互通信。此处使用 Docker Auth 作为 Docker Registry 的身份验证组件，用于管理用户和团队，负责处理身份验证请求并授予 Docker Registry 的访问权限。

如图 5.19 所示，ImageUtil 工具类调用 ResourceInfoFetcher 类的方法获取管理员 token，携带此 token 调用 Docker Registry API 获取私有镜像仓库中的镜像信息，通过 Image 类的构造方法生成 Image 对象保存至 MySQL 数据库中。

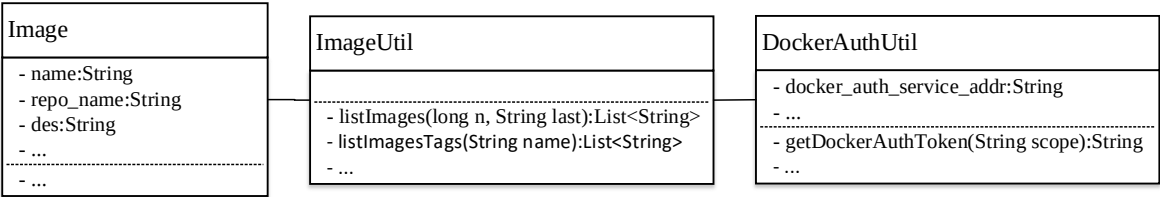


图5.19 镜像管理模块相关类图

默认情况下，系统管理员具有 Docker Registry 镜像仓库的所有权限，既可以执行 push 或 pull 操作推送或拉取镜像，也可以调用 Docker Registry API 对镜像仓库中的镜像执行其他操作；而其他普通用户则只可以执行 pull 操作，使用平台提供的镜像启动容器。以 pull 操作为例，Docker Registry 镜像仓库的认证流程如图 5.20 所示：

- (1) Docker 客户端请求拉取镜像，docker-daemon 转发请求到 Docker Registry；
- (2) 若 Docker Registry 开启认证，则会返回 401 Unauthorized 响应，并且在响应头中设置如何认证的信息，提示用户需要认证才能拉取镜像；
- (3) docker-daemon 向 Docker Auth 发起认证请求，获取 token；
- (4) Docker Auth 根据用户提供的凭证信息，进行用户身份认证，返回 token；

- (5) docker-daemon 携带用户 token 向 Docker Registry 重新发送请求拉取镜像；
- (6) Docker Registry 验证用户 token 的有效性，随后执行镜像拉取流程。

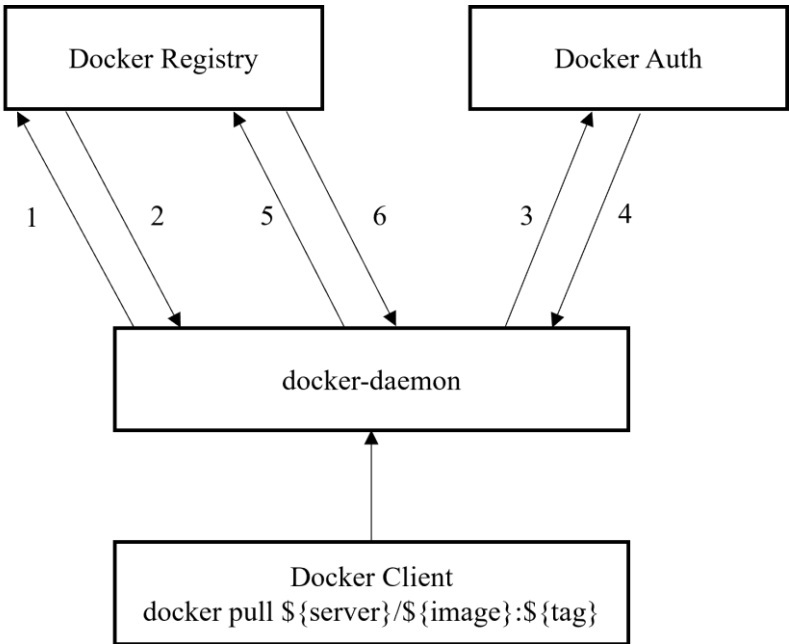


图5.20 Docker Registry 认证流程图

当系统管理员将构建的镜像推送到平台私有镜像仓库后，管理员可以通过认领的方式，补充镜像的基本信息并将其发布。镜像管理界面如图 5.21 所示。

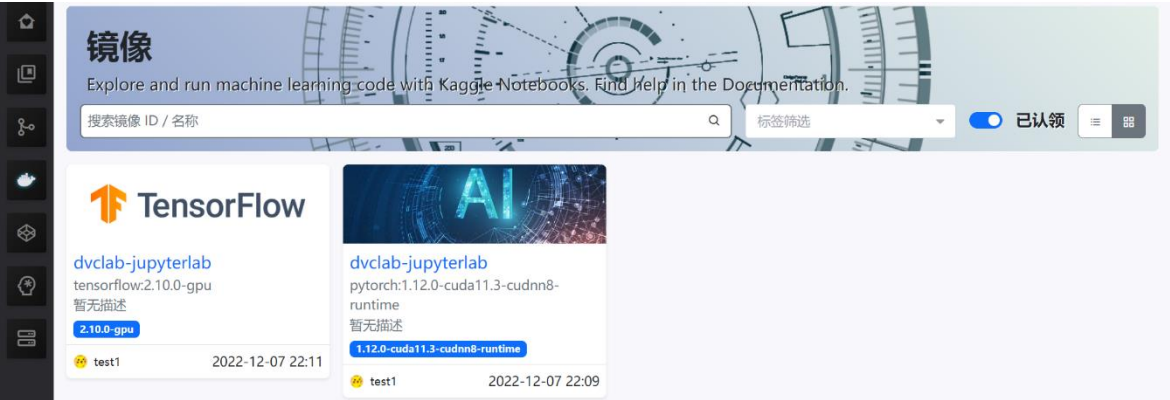


图5.21 镜像管理界面

5.2.3 资源管理模块

资源管理主要包括数据集管理、项目管理和模型管理三大模块，用于管理和维护平台中的数据集、项目和模型，为深度学习平台提供一个统一的数据存储和共享环境。资源管理模块采用基于 Ceph 集群提供的分布式对象存储服务存储数据文件。

数据集模块支持两种不同形式的数据集：一般存储数据集和在线标注数据集。一般存储数据集直接采用分布式对象存储服务进行存储，而在线标注数据集通过系统自行设计实现的 Git 版本管理功能进行管理并采用分布式对象存储服务存储相应的文

件。在创建数据集时，用户填写数据集基本信息后向 Java 后台发送请求，在身份验证通过后，DatasetRoute 类接收创建数据集的请求调用 DatasetManager 类的 createDataset 方法根据参数标明的数据集类型，创建不同类型的数据集。图 5.22 所示为数据集相关类图。

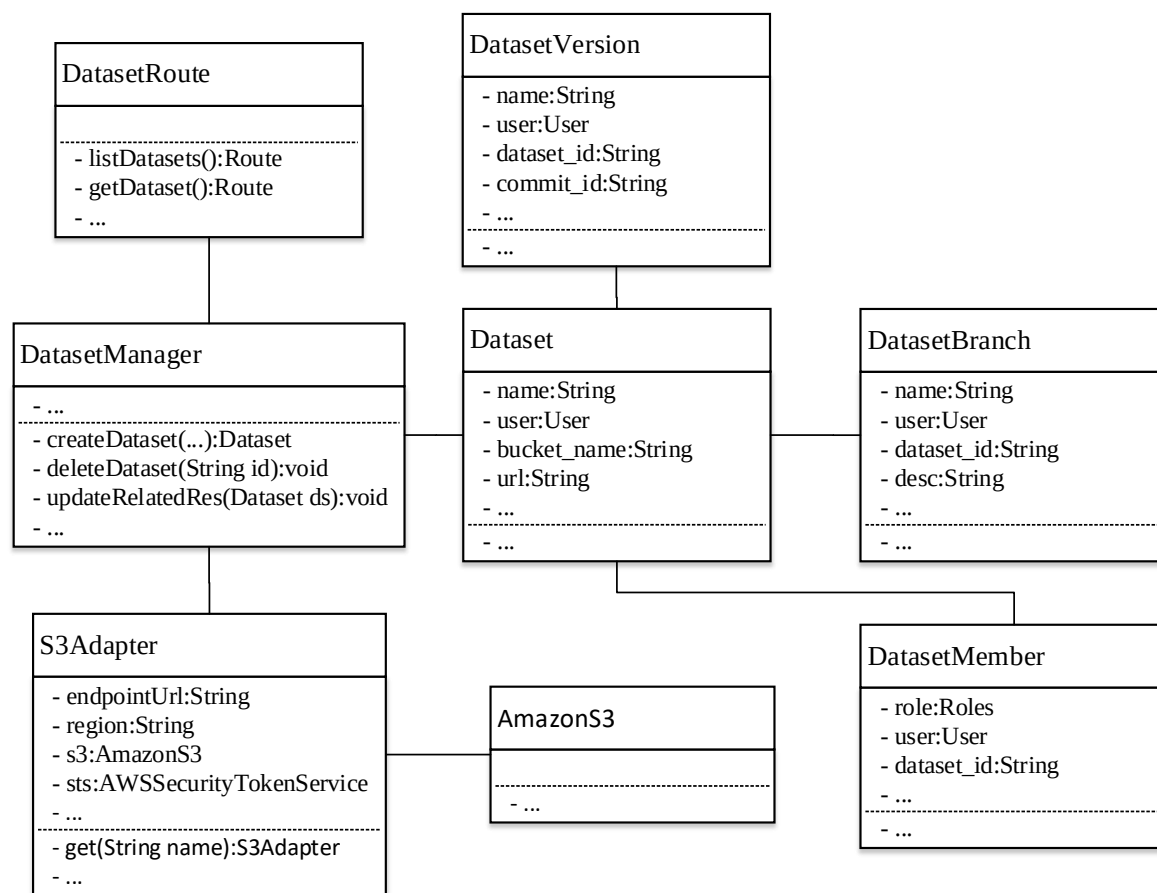


图5.22 数据集模块相关类图

对于在线标注数据集的版本管理功能，基于系统自行设计实现的 Git 版本管理功能实现。平台数据集的访问方式可以包含以下三种方式：

- (1) 用户可以在自己的开发环境容器中以只读的形式挂载已经发布的数据集；
- (2) 用户可以通过 Git 版本管理支持的 Smart HTTP 协议，将服务端数据集仓库 clone 到客户端，用户可以在本地管理数据集，对数据集文件进行修改等操作，最后将修改后的数据集推送到服务器；
- (3) 用户也可以通过 Git 版本管理模块封装的 Git 指令，通过分布式对象存储提供的数据在线访问接口，选择分支，从对应的 **worktree** 工作目录中读取数据集文件，用户可在线修改数据集文件或标注文件，通过封装 Git 指令的方法将修改提交到服务端 Git 版本库。

项目管理模块同样是基于系统自行设计实现的 Git 版本管理功能实现，项目资源

主要用于根据深度学习需要解决的实际问题配置需要引用的数据集和采用的深度学习框架，以及解决此问题所编写的项目代码。模型管理模块基于分布式对象存储服务实现，用于管理已经训练好的神经网络模型。对于项目管理和模型管理的实现都是基于上一节系统基础功能实现的，且与数据集管理的实现方式类似，此处不再过多赘述。

5.2.4 开发环境管理模块

开发环境管理模块主要负责管理用户启动的 JupyterLab 开发环境容器，主要包括开发环境容器调度、容器状态管理、容器日志分析等功能。

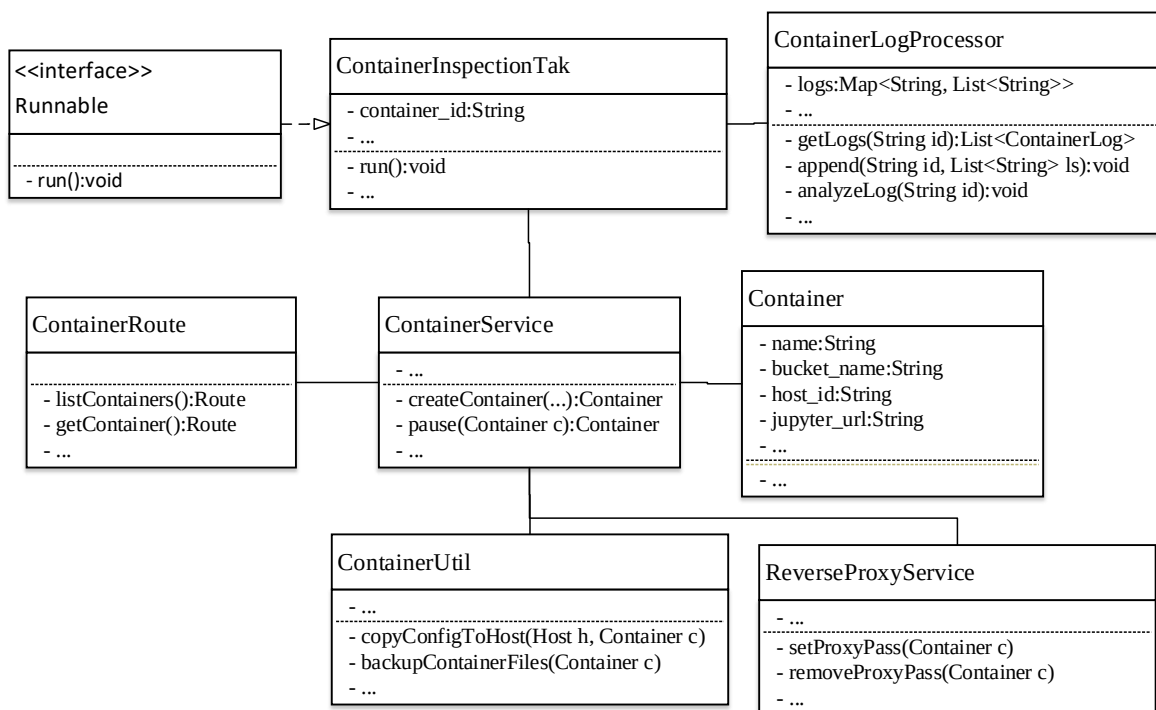


图5.23 开发环境管理相关类图

如图 5.23 所示，开发环境容器调度基于主机管理模块的主机选取算法实现，根据容器启动申请的资源额度将容器调度到合适的主机上进行部署。`ContainerRoute` 类接收到用户创建开发环境的请求后，通过 `ContainerService` 类创建启动开发环境容器的 Docker Compose 配置文件，根据容器申请的资源额度分配主机和容器访问端口，并在 MySQL 中创建容器记录；`ContainerUtil` 类则对 `docker-compose` 命令进行封装，将配置文件传输至目标主机执行 `docker-compose` 命令启动容器。容器启动的 Docker Compose 配置文件模板如图 5.24 所示，其中，通过 `deploy.resources` 下 `limites` 字段限制容器所使用的 CPU 和内存的最大额度，`reservations` 字段给容器配置预留的 GPU 设备，并通过 `reservations.devices.device_ids` 指定具体预留哪个 GPU。

```

version: "3.8"
services:
  jupyterlab:
    restart: always
    image: ${image_name}
    container_name: ${container_id}
    privileged: true
    tty: true
    volumes:
      - ./workspace:/workspace
    ports:
      - ${port}:8988
    environment:
      - ...
    entrypoint: /opt/init_container.sh
      ${runtime}
    deploy:
      resources:
        limits:
          cpus: "${cpus}"
          memory: "${mem}G"
        reservations:
          devices:
            - driver: nvidia
              device_ids: [${device_ids}]
            capabilities: [gpu]

```

图5.24 容器启动 Docker Compose 配置文件模板

开发环境容器采用的镜像皆为镜像管理模块提供的集成了不同深度学习框架的 JupyterLab 镜像，在开发环境容器调度到合适的主机上进行部署后，开发环境容器会周期性的向 Java 后台发送容器的心跳包信息，告知 Java 后台容器自身的状态。当 Java 后台超过最大阈值时常仍未接收到容器的心跳包时，若部署容器的主机也处于断开状态，则将容器判定为 Disconnected 状态；若部署容器的主机处于连接状态，则将容器判定为出错，系统会自动将此开发环境容器的工作区文件备份，然后将容器删除。

另外，当 JupyterLab 开发环境容器转变为 Running 状态后，ContainerService 类通过调用 ContainerInspectionTask 类启动周期任务，调用 Jupyter Server API 获取日志信息和 Kernel 信息。Kernel 是 JupyterLab 的后端计算引擎，它执行用户在前端发出的代码运行请求，并将运行结果返回给前端。ContainerLogProcessor 类调用 analyzeLog 方法结合 Kernel 信息对日志信息进行分析，记录 Kernel 的 execution_state 由 idle 转为 busy 和由 busy 转为 idle 的时间节点，即用户在开发环境中执行任务的起止时间；当任务完成后，检测开发环境中是否生成最新模型文件，若生成文件，则根据模型文件生成模型版本，将模型文件转存至预先配置好到模型存储 bucket 中。其中模型版本生成公式如 5-3 所示：

$$id = SHA1(SHA1(file_1)+SHA1(file_2)+\dots+SHA1(file_n)) \quad (5-4)$$

其中，SHA1 表示数据加密算法 SHA-1，file_n 代表模型文件。

对于开发环境后台无任务运行且长期处于空转状态的容器，Java 后台会自动将开

发环境容器暂停从而释放容器所占用主机资源。

开发环境容器生命周期如图 5.25 所示。

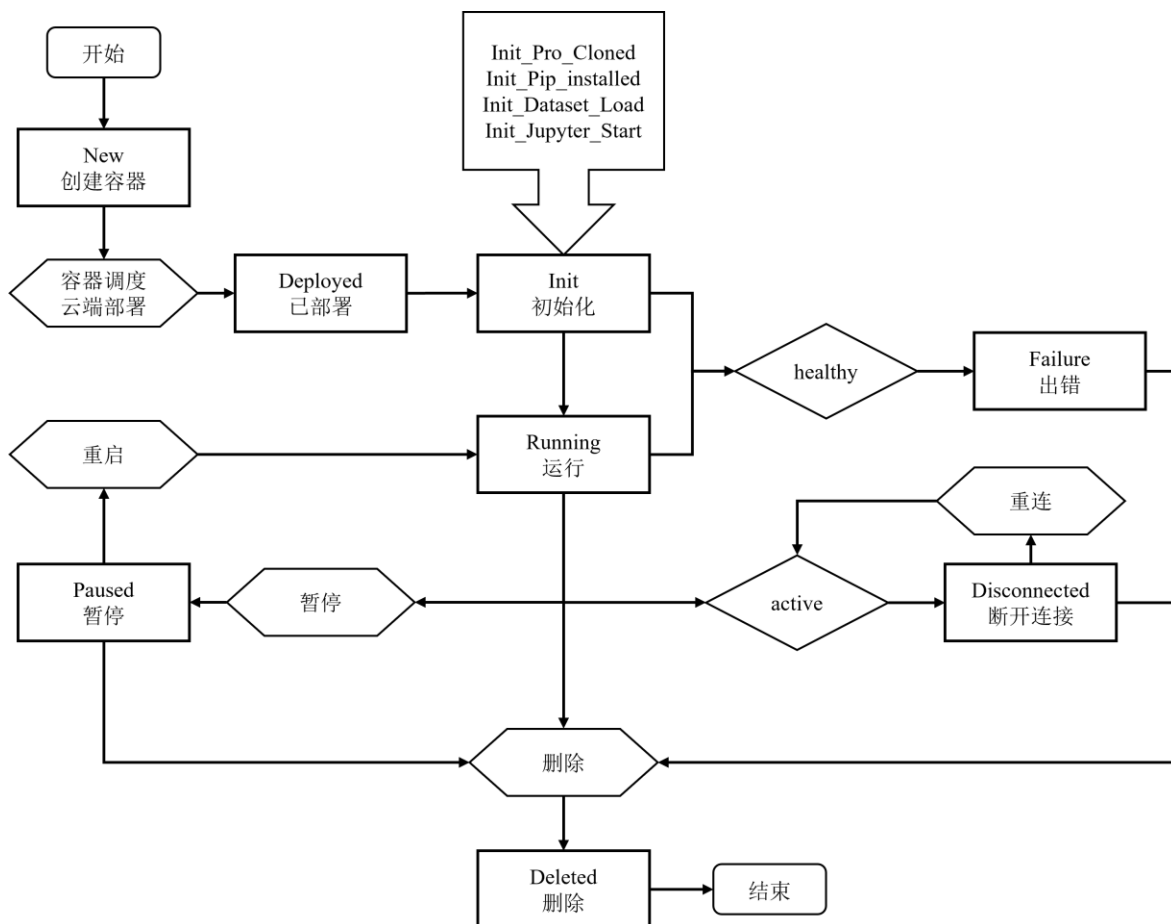


图5.25 开发环境生命周期

基于安全性考虑，平台并不会直接将用户部署的 JupyterLab 开发环境容器的访问 ip 和 port 直接提供给用户，Java 后台通过 ReverseProxyService 类将访问开发环境的 ip 和 port 隐藏，替换为 `https://{server}/containers/{id}` 的形式暴露给用户，并将关联关系存储到 Redis 中。同时，基于 OpenResty 和 Redis 实现的动态路由机制灵活的访问不同用户的开发环境。如图 5.26 所示，实现方式如下：

- （1）ReverseProxyService 类将用户开发环境的访问 ip 和 port 映射为 `https://{server}/containers/{id}` 存储到 Redis 中；
- （2）接收到请求后，Openresty 通过 Lua 脚本解析 URI 和参数，获取容器 ID；
- （3）OpenResty 访问 Redis 数据库，获取对应的服务地址信息；
- （4）将请求路径和服务地址进行拼接，生成完整的 URL，进行代理转发。

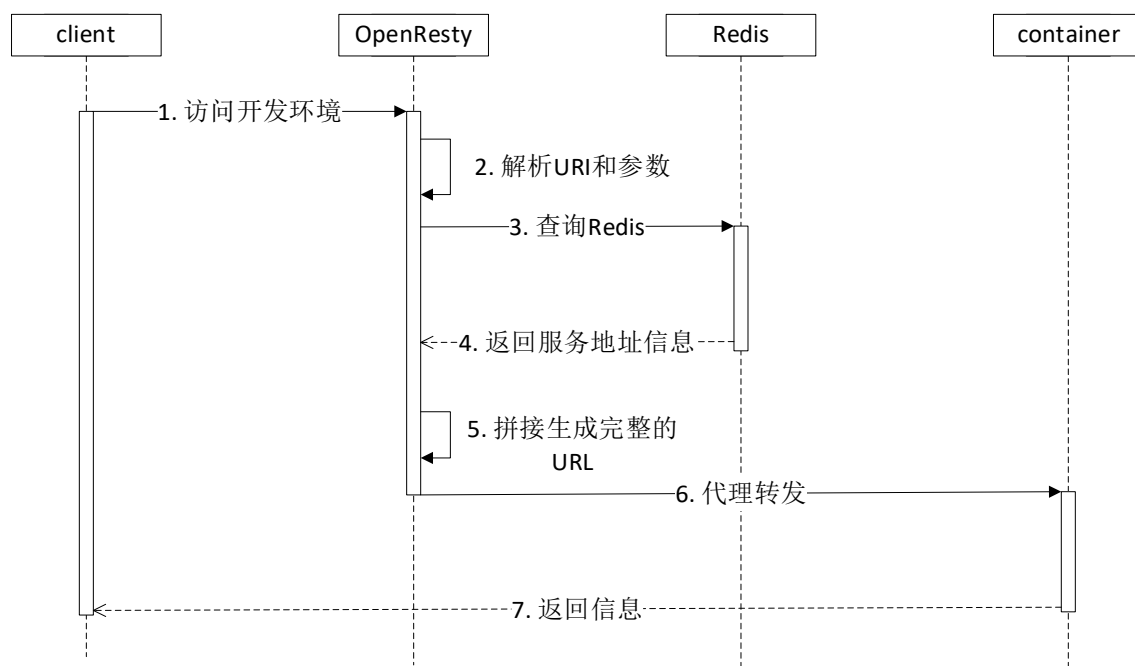


图5.26 动态路由访问时序图

图 5.27 所示为 JupyterLab 开发环境页面，通过终端可以查看到给此容器分配的 GPU 型号为 NVIDIA GeForce GTX 1080 Ti，CPU 型号为 Intel(R) Core(TM) i5-9400 CPU @ 2.90GHz，核数位 6 核。用户可在容器启动后，在 JupyterLab 开发环境中，通过创建新的终端页面，验证是否给开发环境容器分配了相应的系统资源。

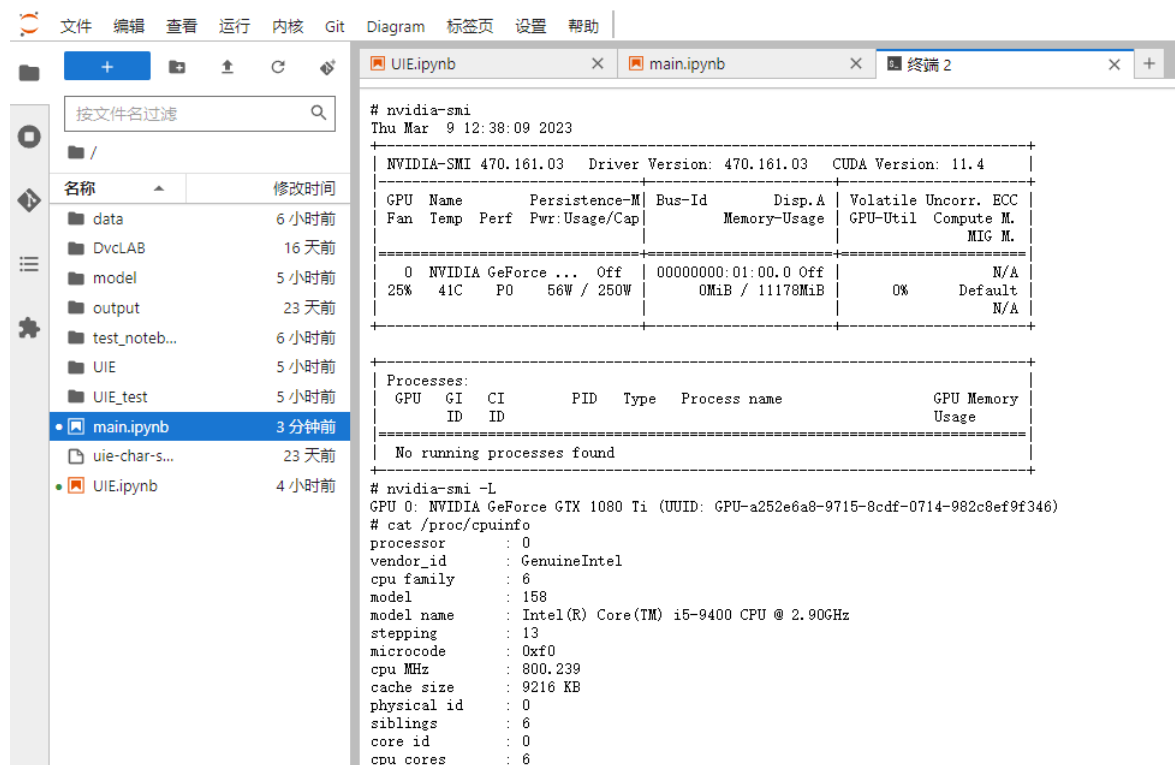


图5.27 JupyterLab 开发环境页面

5.2.5 模型托管模块

模型托管模块主要负责深度学习模型的部署、对外提供推理服务和运行状态管理等。模型托管是深度学习应用落地过程中非常重要的一环，它可以提供一种便捷的解决方案，将训练好的模型打包成容器，通过模型托管模块将模型部署到 Kubernetes 集群中，从而简化模型部署流程、加快深度学习应用上线速度，提高应用的可靠性和稳定性。

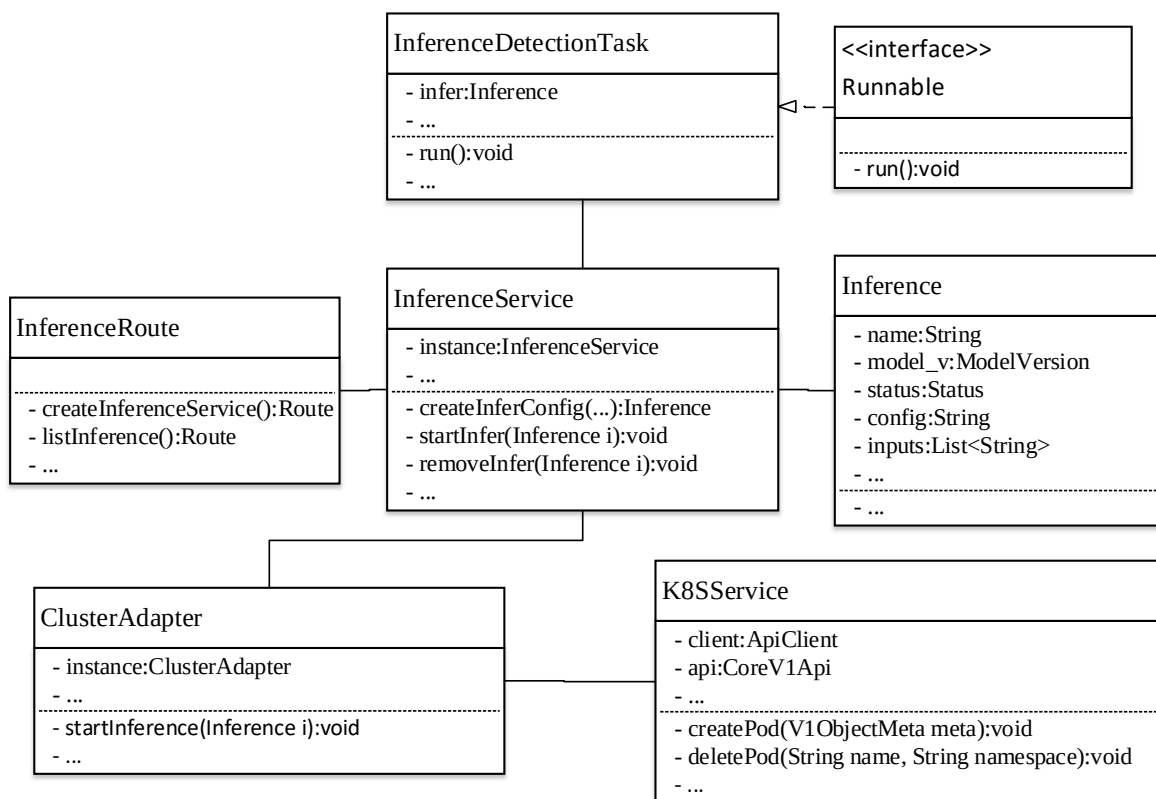


图5.28 模型托管相关

如图 5.28 所示，Java 后台通过 InferenceService 类根据部署模型接口传来的模型信息等参数，创建部署模型 yaml 配置文件，通过 K8SService 类调用 createPod 方法将 Pod 调度到合适的集群节点上部署推理服务，流程如下：

- (1) 选取要部署的模型；
- (2) 根据模型的输入输出信息，编写数据前处理和后处理函数，以便通过前处理函数将原始数据转换为模型期望的数据格式，通过后处理函数将模型的输出结果转换为可读的格式；
- (3) InferenceService 类调用 createInferConfig 方法生成 yaml 配置文件；
- (4) K8SService 类调用 createPod 方法创建 Pod，部署模型；
- (5) InferenceDetectionTask 类检测模型是否部署成功；
- (6) 对外提供推理服务。

以文本输入为例，推理服务接口调用界面如图 5.29 所示。

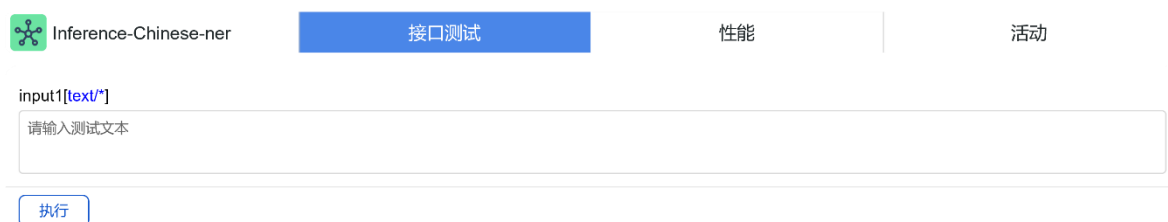


图5.29 推理服务接口调用界面

5.3 本章小结

本章主要介绍了私有云环境下基于微服务架构的深度学习微服务平台系统的详细实现过程，主要包括系统的基础功能模块和系统的业务功能模块两大部分。在系统基础功能方面，采用了 keycloak 作为用户认证和授权的中心，实现了对用户身份的认证和权限的控制。同时，还引入了 Ceph 对象存储技术，实现了对数据的可靠存储和管理；通过基于对象存储的 Git 版本管理，实现了对代码和数据的版本控制。此外，还实现了监控功能，对系统的运行状态进行实时监控。系统基础功能的实现为业务功能的实现奠定基础。在业务功能方面，实现了主机管理、镜像管理、资源管理、开发环境和模型托管等多个功能。主机管理模块实现了对私有云环境下的主机资源的管理和监控；镜像管理模块实现了对镜像的构建、管理和版本控制；资源管理模块实现了对项目、数据集、模型等资源的管理；开发环境模块实现了对开发环境的自动化构建和管理；模型托管模块实现了对深度学习模型的部署和应用管理。系统各个业务功能模块相互关联、相互依存，从而构建了完整的深度学习一站式服务平台。

第六章 系统部署与测试

基于前面章节设计实现的基于微服务架构的深度学习容器化平台，本章主要围绕对系统的部署和功能测试进行。对于系统部署部分，主要介绍系统各个应用模块软件版本参数以及核心应用的部署流程。测试部分则通过对系统各个功能模块的核心功能测试，验证平台的可靠性和可用性。

6.1 系统部署

本深度学习平台采用微服务的思想进行设计和开发，将整个系统拆分为较小的、独立的服务单元，每个服务单元都有自己的业务实现逻辑和数据存储模块。基于微服务思想，每个服务单元都可以独立开发、部署和测试，服务间通过轻量级通信机制进行通信协作。采用微服务架构，使得系统开发部署变得更加灵活高效，提高系统的伸缩性、可靠性和可维护性。

6.1.1 系统软件环境

系统各个服务都是通过 docker-compose 以容器的形式进行部署，表 6.1 介绍了系统软件环境信息。

表6.1 系统软件环境

名称	版本	描述
Docker	20.10.21	容器化虚拟技术
Docker Compose	1.29.2	定义和运行 Docker 容器应用的工具
Docker Registry	2.3	存储和分发 Docker 镜像
Docker Auth	1.6.0	Docker Registry 的授权模块，用于对 Docker Registry 的访问进行认证和授权
Kubernetes	1.18.2	容器编排系统
Ceph	15.2.17 (stable)	分布式存储系统
S3FS	v1.86	将 Amazon S3 存储桶挂载为本地文件系统的挂载工具
S3cmd	2.0.2	用于管理 Amazon S3 和其他 S3 兼容对象存储服务 的命令行工具
WireGuard	V1.0.20200513	内核级虚拟专有网络技术
Bind9	9.18.1-1ubuntu1-Ubuntu	DNS 服务器

名称	版本	描述
OpenResty	1.19.3.1	基于 Nginx 的 Web 应用服务器
Keycloak	19.0.2	身份认证与访问控制解决方案
MySQL	8.0.27	关系型数据库
Redis	6.2.6	高性能 key-value 数据库
Prometheus	v2.28.0	新一代云原生监控系统
Grafana	8.1.2	数据可视化工具

6.1.2 Docker 部署

Docker 是深度学习平台系统运行的基本环境，系统其他服务都是基于 Docker 容器启动。目前采用的 Docker 版本为 20.10.21，Docker 19.0.3 版本后添加了对 nvidia-container-runtime 的支持，允许 Docker 访问 NVIDIA GPU，简化了 Docker 使用 GPU 的配置复杂度。Docker 以及 Docker Compose 的安装 Shell 脚本如图 6.1 所示。

```
#!/bin/bash
DOCKER=`docker -v`
DOCKER_COMPOSE=`docker-compose -v`

if [[ "$DOCKER" != *"version"* ]];
then
    curl -fsSL "https://get.docker.com" | bash -s docker --mirror Aliyun
    systemctl enable docker
    systemctl start docker
else
    echo "docker installed"
fi

if [[ "$DOCKER_COMPOSE" != *"version"* ]];
then
    curl -L
    "https://github.com/docker/compose/releases/download/1.29.2/docker-
    compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
    chmod +x /usr/local/bin/docker-compose
else
    echo "docker-compose installed"
fi
```

图6.1 Docker 以及 Docker Compose 的安装 Shell 脚本

配置完成后重启 Docker，执行命令“docker version”查看 Docker 版本验证是否安装完成，详细版本信息如图 6.2 所示。

```

Client: Docker Engine - Community
 Version:           20.10.21
 API version:       1.41
 Go version:        go1.18.7
 Git commit:        baedalf
 Built:             Tue Oct 25 18:02:21 2022
 OS/Arch:           linux/amd64
 Context:           default
 Experimental:      true

Server: Docker Engine - Community
 Engine:
  Version:           20.10.21
  API version:       1.41 (minimum version 1.12)
  Go version:        go1.18.7
  Git commit:        3056208
  Built:             Tue Oct 25 18:00:04 2022
  OS/Arch:           linux/amd64
  Experimental:      false
 containerd:
  Version:           1.6.9
  GitCommit:         1c90a442489720eec95342e1789ee8a5e1b9536f
 runc:
  Version:           1.1.4
  GitCommit:         v1.1.4-0-g5fd4c4d
 docker-init:
  Version:           0.19.0
  GitCommit:         de40ad0

```

图6.2 Docker 版本信息

6.1.3 Kubernetes 集群部署

Kubernetes 集群通过官方提供的用于快速部署安装 Kubernetes 的命令行工具 kubeadm 进行安装^[46]，在每个节点上安装 kubectl、kubelet 和 kubeadm 工具，并在集群主节点执行以下命令：

```

kubeadm init \
  --pod-network-cidr=${ip}/16 \
  --kubernetes-version v1.18.2 \
  --apiserver-advertise-address ${ip}

```

Kubernetes 集群组件信息如表 6.2 所示。

表6.2 Kubernetes 集群组件列表

名称	版本	描述
kubeadm	1.18.2	引导启动 Kubernetes 集群命令行工具
kubelet	1.18.2	执行启动 Pod 和 Container 等操作
kubectl	1.18.2	用于操作集群的命令行工具

集群部署完成后，查看集群系统关键组件的运行状态，如图 6.3 所示。

```
root@hpcc7:~# kubectl get pod -n kube-system -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
calico-kube-controllers-5b8b769fcd-glnt6	1/1	Running	0	89d	10.1.62.194	hpcc7	<none>	<none>
calico-node-86nf8	1/1	Running	1	89d	10.0.9.33	hpcc3	<none>	<none>
calico-node-h99zc	1/1	Running	0	89d	10.171.27.36	hpcc6	<none>	<none>
calico-node-nc8j2	1/1	Running	0	89d	10.171.27.38	hpcc8	<none>	<none>
calico-node-rl9x6	1/1	Running	0	89d	10.0.9.34	hpcc4	<none>	<none>
calico-node-tlxnm	1/1	Running	0	89d	10.0.9.37	hpcc7	<none>	<none>
calico-node-wwdzl	1/1	Running	0	89d	10.171.27.35	hpcc5	<none>	<none>
coredns-546565776c-pvznd	1/1	Running	1	89d	10.1.189.97	hpcc3	<none>	<none>
coredns-546565776c-t7z8z	1/1	Running	0	89d	10.1.37.130	hpcc6	<none>	<none>
etcd-hpcc7	1/1	Running	0	89d	10.0.9.37	hpcc7	<none>	<none>
kube-apiserver-hpcc7	1/1	Running	0	89d	10.0.9.37	hpcc7	<none>	<none>
kube-controller-manager-hpcc7	1/1	Running	1	89d	10.0.9.37	hpcc7	<none>	<none>
kube-proxy-2c9j8	1/1	Running	0	89d	10.0.9.37	hpcc7	<none>	<none>
kube-proxy-7bm9	1/1	Running	1	89d	10.0.9.33	hpcc3	<none>	<none>
kube-proxy-cjm9d	1/1	Running	0	89d	10.0.9.34	hpcc4	<none>	<none>
kube-proxy-d2b5b	1/1	Running	0	89d	10.171.27.35	hpcc5	<none>	<none>
kube-proxy-pml7r	1/1	Running	0	89d	10.171.27.38	hpcc8	<none>	<none>
kube-proxy-wggjg	1/1	Running	0	89d	10.171.27.36	hpcc6	<none>	<none>
kube-scheduler-hpcc7	1/1	Running	1	89d	10.0.9.37	hpcc7	<none>	<none>
metrics-server-57bc7f4584-c9qx5	1/1	Running	0	89d	10.1.62.196	hpcc7	<none>	<none>
nvidia-device-plugin-daemonset-4dff4	1/1	Running	1	89d	10.1.189.96	hpcc3	<none>	<none>
nvidia-device-plugin-daemonset-4t7nl	1/1	Running	0	89d	10.1.62.218	hpcc7	<none>	<none>
nvidia-device-plugin-daemonset-94lrv	1/1	Running	0	89d	10.1.37.129	hpcc6	<none>	<none>
nvidia-device-plugin-daemonset-r5kwg	1/1	Running	0	89d	10.1.192.64	hpcc4	<none>	<none>
nvidia-device-plugin-daemonset-rm2lx	1/1	Running	0	89d	10.1.104.193	hpcc8	<none>	<none>
nvidia-device-plugin-daemonset-xzmbk	1/1	Running	0	89d	10.1.225.129	hpcc5	<none>	<none>

图6.3 k8s 集群组件运行状态

6.1.4 Ceph 集群部署

Ceph 集群的部署通过 cephadm 进行部署，它是一个便捷的 Ceph 管理工具，提供快速部署、自动化管理、容器化等功能^[47]。借助 cephadm 管理工具在管理节点执行“cephadm bootstrap”指令引导 Ceph 集群部署。Ceph 部署完成后，关键组件运行状态信息如图 6.4 所示。

IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
ceph/ceph:v15	"/usr/bin/ceph-mon --"	3 weeks ago	Up 3 weeks		ceph-898fa908-ef9e-11eb-bf9c-8727584d5bae-mon.f3d0
ceph/ceph:v15	"/usr/bin/ceph-osd --"	3 weeks ago	Up 3 weeks		ceph-898fa908-ef9e-11eb-bf9c-8727584d5bae-osd.2
ceph/ceph:v15	"/usr/bin/ceph-osd --"	3 weeks ago	Up 3 weeks		ceph-898fa908-ef9e-11eb-bf9c-8727584d5bae-osd.0
ceph/ceph:v15	"/usr/bin/ceph-osd --"	3 weeks ago	Up 3 weeks		ceph-898fa908-ef9e-11eb-bf9c-8727584d5bae-osd.1
ceph/ceph-grafana:6.7.4	"/bin/sh -c 'grafana--'"	3 weeks ago	Up 3 weeks		ceph-898fa908-ef9e-11eb-bf9c-8727584d5bae-grafana.f3d0
ceph/ceph:v15	"/usr/bin/radosgw --"	3 weeks ago	Up 3 weeks		ceph-898fa908-ef9e-11eb-bf9c-8727584d5bae-rgw.psial.xian.f3d0.nazem
prom/node-exporter:v0.18.1	"/bin/node_exporter --"	3 weeks ago	Up 3 weeks		ceph-898fa908-ef9e-11eb-bf9c-8727584d5bae-node-exporter.f3d0
ceph/ceph:v15	"/usr/bin/ceph-mgr --"	3 weeks ago	Up 3 weeks		ceph-898fa908-ef9e-11eb-bf9c-8727584d5bae-mgr.f3d0.jdpjyt
prom/prometheus:v2.18.1	"/bin/prometheus --c--"	3 weeks ago	Up 3 weeks		ceph-898fa908-ef9e-11eb-bf9c-8727584d5bae-prometheus.f3d0
ceph/ceph:v15	"/usr/bin/ceph-mds --"	3 weeks ago	Up 3 weeks		ceph-898fa908-ef9e-11eb-bf9c-8727584d5bae-mds.cephfs.f3d0.fcuyfb
prom/alertmanager:v0.20.0	"/bin/alertmanager --"	3 weeks ago	Up 3 weeks		ceph-898fa908-ef9e-11eb-bf9c-8727584d5bae-alertmanager.f3d0
ceph/ceph:v15	"/usr/bin/ceph-crash--"	3 weeks ago	Up 3 weeks		ceph-898fa908-ef9e-11eb-bf9c-8727584d5bae-crash.f3d0

图6.4 Ceph 集群关键组件运行状态

Ceph 集群管理界面如图 6.5 所示。

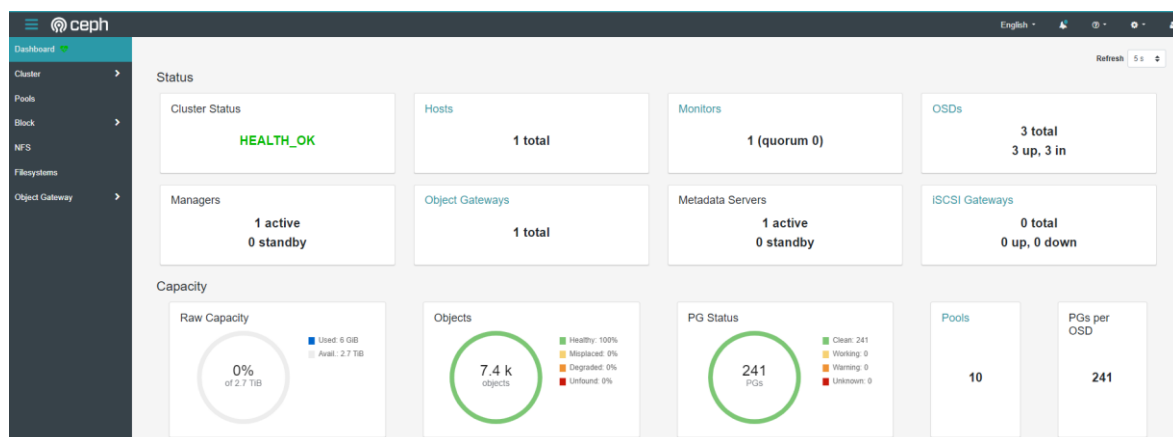


图6.5 Ceph 集群管理界面

6.2 系统功能测试

系统功能测试是指对系统各个功能模块进行集成测试,验证系统是否满足设计要求,各个模块是否能够按要求正常工作,从而确保产品质量。本小节将根据系统需求设计测试用例,以验证用户管理、主机管理、镜像管理、资源管理、开发环境管理和模型托管模块的功能。

6.2.1 用户管理模块测试

用户管理是平台的重要功能之一,主要包括用户注册、用户个人信息管理、用户权限管理等。用户管理测试用例表如表 6.3 所示。

表6.3 用户管理测试用例表

测试编号	01		
测试类型	系统功能测试		
测试目的	测试用户管理模块是否符合设计要求, 是否能够正常执行并产生预期结果		
场景编号	场景描述	测试步骤	测试结果
1	用户注册	用户进入平台登录页面, 点击微信登录后, 若系统识别到用户为第一次登录, 则跳转到注册界面进行注册	与预期一致, 系统管理员可在用户列表界面查看到新添加用户
2	用户信息修改	进入平台界面后, 用户可点击个人主页, 编辑用户个人信息	与预期一致, 用户信息更新
3	用户权限管理	非成员用户访问私有资源时, 查看资源基本信息, 然后获取资源	与预期一致, 用户可查看资源基本信息, 但无法获取资源, 返回 403 状态码
4	用户合法性	用户携带非法/过时的 Access Token 或未携带 Access Token 访问 Java 后台接口	与预期一致, 接口返回 401 状态码

6.2.2 主机管理模块测试

主机管理模块的测试主要包括管理员添加、删除主机, 普通用户则无权执行此操作, 只可查看主机列表及主机基本信息; 同时, 还需测试能否实时获取主机状态信息,

监控主机的运行状态。表 6.4 为主机管理测试用例。

表6.4 主机管理测试用例表

测试编号	02		
测试类型	系统功能测试		
测试目的	测试主机管理模块是否符合设计要求，能否正常管理主机并监控主机状态		
场景编号	场景描述	测试步骤	测试结果
1	添加主机	系统管理员进入主机管理界面，点击添加主机；普通用户进入主机管理界面则不显示添加界面	与预期一致，系统管理员可正常添加主机，普通用户无法进入添加主机界面，若直接调用 Java 后台接口则返回 403 状态码
2	删除主机	系统管理员可删除主机；普通用户无权操作	与预期一致
3	状态监控	Java 后台周期性获取主机时序信息并发送给前端用以显示	与预期一致，通过 Postman 模拟前端访问获取到主机 CPU、IO、内存等信息

6.2.3 镜像管理模块测试

镜像管理模块的测试主要是对深度学习平台所支持的集成了各种深度学习框架及不同版本的 JupyterLab 镜像的发布、使用等进行测试，验证镜像管理功能的完整性，确保可以正常存储、分发镜像。表 6.5 为镜像管理测试用例。

表6.5 镜像管理测试用例表

测试编号	03		
测试类型	系统功能测试		
测试目的	测试镜像管理模块是否符合设计要求，能否正常存储、分发镜像		
场景编号	场景描述	测试步骤	测试结果
1	发布镜像	系统管理员将构建好的镜像上传至平台私镜像仓库，然后在镜像管理界面发布镜像	与预期一致，系统管理员可以上传、发布镜像，普通用户无法上传镜像
2	拉取镜像	用户通过“docker login”登录后，执行“docker pull”拉取镜像	与预期一致，拉取镜像成功
3	删除镜像	系统管理员可删除镜像，普通用户无权操作	与预期一致

6.2.4 资源管理模块测试

资源管理模块的测试主要包括项目、数据集和模型资源的增删改查、版本管理、资源文件上传和下载以及资源存储 bucket 挂载等功能的测试。此测试用例以项目资源为例进行测试，表 6.6 为项目资源管理测试用例。

表6.6 项目资源管理测试用例表

测试编号	04		
测试类型	系统功能测试		
测试目的	测试资源管理模块是否符合设计要求，能否正常管理资源		
场景编号	场景描述	测试步骤	测试结果
1	创建项目	用户在项目管理界面配置项目基本信息创建项目	与预期一致，项目创建成功后，在分布式对象存储中创建了对应的存储 bucket，并在其中初始化 Git 版本仓库
2	克隆项目	用户在自己 PC 端使用 Git 客户端执行“git clone”	与预期一致，普通用户无权克隆项目，其他测试用户可以执行
3	推送 master 分支	用户在自己 PC 端使用 Git 客户端执行“git push”	与预期一致，普通用户、Viewer 和 Editor 成员用户无权操作，其他用户可以执行
4	推送其他分支	用户在自己 PC 端使用 Git 客户端执行“git push”	与预期一致，普通用户和 Viewer 成员用户无权操作，其他用户可以执行
5	发布版本	项目 Owner 和 Admin 可以发布项目版本	与预期一致
6	挂载 bucket	通过 S3FS 工具挂载 bucket	与预期一致

6.2.5 开发环境管理模块测试

开发环境管理模块的测试主要包括开发环境的创建和管理，测试开发环境启动后，容器内部能否正常克隆项目代码、挂载训练用的数据集以及能否启动 Jupyter 进程；开发环境部署后，测试能否根据 JupyterLab 容器日志检测容器中的训练任务以及是否生成模型文件并进行转存。表 6.7 为开发环境管理测试用例。

表6.7 开发环境管理测试用例表

测试编号	05		
测试类型	系统功能测试		
测试目的	测试开发环境管理模块是否符合设计要求，能否正常创建开发环境容器		
场景编号	场景描述	测试步骤	测试结果
1	创建开发环境	用户基于项目配置信息启动深度学习开发环境容器	与预期一致
2	模型训练并转存模型文件	用户可在开发环境容器中编写项目代码进行模型训练，模型训练完成后系统自动转存模型文件	与预期一致，系统自动转存模型文件生成模型的 MySQL 记录

6.2.6 模型托管模块测试

模型托管模块的测试主要包括模型部署以及能否对外提供推理服务。测试用户能否通过平台提供的模型或自己开发环境训练产生的模型在平台提供的 Kubernetes 集群上快速部署，并对外提供推理服务，通过 API 接口调用推理服务。

表6.8 模型托管测试用例表

测试编号	06		
测试类型	系统功能测试		
测试目的	测试模型托管模块是否符合设计要求，能否正常进行模型部署		
场景编号	场景描述	测试步骤	测试结果
1	模型部署	用户在模型托管界面部署模型	与预期一致
2	推理服务接口调用	用户在推理服务接口测试页面，输入对应类型的参数调用推理服务接口	与预期一致

6.3 系统性能测试

本小节主要对深度学习平台进行系统性能测试。系统性能测试主要是对系统在特定负载条件下进行性能测试的过程，目的是评估平台系统在实际应用条件下系统的稳定性和可靠性，以确保系统在高负载条件下仍能正常运行。

6.3.1 对象存储性能测试

对象存储性能测试主要针对平台对外提供的分布式对象存储服务的文件上传和

下载速率进行测试，测试对象存储服务的性能瓶颈，从而验证其是否满足设计需求，为用户提供可靠的对象存储服务。测试步骤如下：

- (1) 构建网络相对稳定的测试环境，本次测试在平台私有云环境下进行测试；
- (2) 选取私有云环境下非 Ceph 集群服务器进行测试，服务器均位于 WireGuard 搭建的虚拟内网中，随机选取虚拟内网环境中四台相同型号服务器进行多批次测试；
- (3) 分别在选取的服务器上通过管理兼容 Amazon S3 的存储服务工具 s3cmd 进行文件上传和下载性能测试。

存储性能测试结果如表 6.9 所示，分布式对象存储服务不论是上传文件还是下载文件都具有不错的数据传输性能。

表6.9 存储性能测试结果

测试点	测试环境	测试次数	测试结果（平均值）
上传文件	服务器 1	50	3.06MB/s
	服务器 2	50	3.08MB/s
	服务器 3	50	3.04MB/s
	服务器 4	50	3.03MB/s
下载文件	服务器 1	50	3.16MB/s
	服务器 2	50	3.17MB/s
	服务器 3	50	3.12MB/s
	服务器 4	50	2.74MB/s

单次文件上传速率如图 6.6 所示，分布式对象存储服务将大文件数据以 15MB 大小进行划分，然后将数据上传至存储模块，数据传输速率普遍介于 2.5-3.5MB/S，具有不错的数据传输速率。

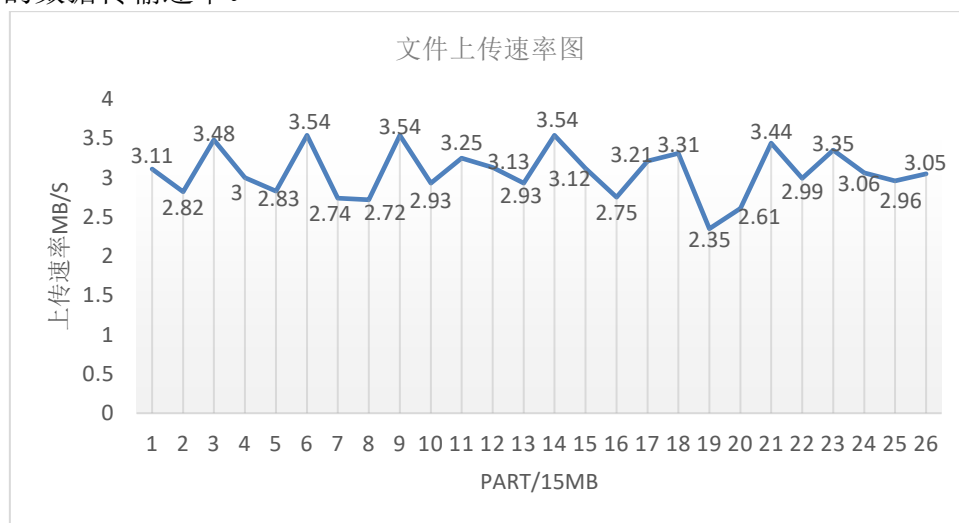


图6.6 对象存储服务文件单次上传速率图

6.3.2 接口压力测试

接口压力测试主要是针对平台提供的后台服务接口在高并发、大流量场景下模拟大量用户同时访问接口，以检测系统的稳定性和可靠性，从而预估系统的承载能力并发现系统存在的潜在问题。

接口压力测试使用 JMeter 工具进行，通过在 JMeter 中创建 Thread Group，配置不同的线程数来模拟不同数量的用户进行接口请求；在 Thread Group 下创建 HTTP Request 来配置测试接口信息^[48]。

表6.10 JMeter 线程参数配置表

参数	值
Number of Threads (users)	200/400/600/800/1000
Ramp-up period (seconds)	2
Loop Count	1

如表 6.10 所示，“Number of Threads (users)”表示线程数，代表不同的用户请求数量；“Ramp-up period (seconds)”表示虚拟用户全部启动的时间；“Loop Count”表述请求循环次数。接口压力测试的用户数量从 200 开始逐渐递增，模拟用户在 2 秒的时间内请求接口的情况。接口性能测试以数据集信息获取接口为例进行展示，如表 6.11 所示，接口在高并发情况下仍然具有较高的数据处理速率和准确率。

表6.11 数据获取接口性能测试结果

线程数量	平均响应时间	异常率
200	131ms	0.00%
400	137ms	0.00%
600	686ms	0.00%
800	850ms	0.00%
1000	2081ms	0.00%

6.4 本章小结

本章主要介绍了私有云环境下基于微服务架构的深度学习容器化平台的部署和测试。在系统部署方面，我们分别介绍了系统软件环境的配置、Docker 和 Kubernetes 的部署、以及 Ceph 集群的部署。在系统测试方面，我们进行了系统功能测试和系统性能测试。在系统功能测试中，我们对系统的各个功能模块进行了全面的测试，包括用户管理、主机管理、镜像管理、资源管理、开发环境管理、模型托管等。通过测试，

验证了系统功能的正确性和稳定性。在系统性能测试中，主要对对象存储的性能进行了测试，包括吞吐量、响应时间等指标。此外，还进行了接口压力测试，测试了系统在高并发情况下的性能表现。通过测试，验证了系统的性能指标符合要求，并且在高并发情况下具备良好的稳定性和可用性。

第七章 总结与展望

7.1 工作总结

本文采用微服务的设计思想,结合 Docker、Kubernetes 等云原生技术,设计并实现了私有云环境下的深度学习平台。本文首先介绍了项目的研究背景及意义,结合深度学习开发训练过程中所存在的痛点,分析设计本平台的重要意义。随后围绕项目所针对的痛点问题展开,以 Docker、Kubernetes 作为底层容器管理解决方案,结合 Ceph 分布式存储、Keycloak 用户认证与授权以及基于分布式对象存储实现的自定义 Git 版本管理,在 Java 后台服务的整合下提供数据集存储、开发环境快速部署、模型托管等功能的一站式深度学习平台。主要工作如下:

(1) 私有云环境下基于微服务架构的深度学习容器化平台的设计工作。基于深度学习开发、训练和部署流程,进行需求分析,分析平台需要具有的功能模块,并根据实际应用场景进行详细设计;设计平台整体架构并进行技术选型,深入了解 Docker、Kubernetes 等技术的特点及使用方式和场景。

(2) 私有云环境下基于微服务架构的深度学习容器化平台的实现工作。基于系统架构设计和功能模块的详细设计,实现不同功能。功能如下:基于 Keycloak 的用户管理模块,用于整个平台用户的登录认证以及授权管理;基于 Ceph 的分布式对象存储服务,用以存储平台及用户的各种数据资源;基于分布式对象存储实现的 Git 版本管理,用以管理数据资源的不同版本以及成员之间的协作开发;以及主机管理、镜像管理、数据集管理、项目管理、开发环境管理、模型托管等功能。各个功能模块各自分工、相互协作、相互串联,共同实现一站式深度学习平台。

(3) 平台的系统部署与测试。平台各个服务基于微服务的架构以 Docker 容器的形式进行部署并对外提供服务,各服务独立运行。功能测试方面,主要对系统的主要业务功能进行测试,验证运行结果是否符合设计要求;性能测试方面,主要对系统的存储性能和接口进行测试,验证系统性能是否符合设计标准。

7.2 工作展望

目前本文研究并设计实现的深度学习平台已经在私有云环境下部署并通过相应测试,初步实现了本文所提及的功能,具备了基本的业务流程,但要成为一个成熟的深度学习平台还存在不小的差距,具体还需在以下方面进行后续完善工作:

(1) 数据集管理模块的在线标注功能只具备了初步雏形,Java 后台初步实现了数据集标注文件的版本管理和存储工作,其他数据集标注工作还需根据前端反馈进行

相应 Java 后台接口的调整;

(2) 目前平台支持的深度学习框架仅包括 PyTorch 和 TensorFlow 两种, 后续应进行扩展以支持更多不同的深度学习框架;

(3) 平台开发环境容器调度方面, 开发环境的调度部署采用自行设计的算法通过 Java 后台服务将容器调度到合适的服务器上, 考虑到容器暂停、重启等功能, 并未通过 Kubernetes 进行开发环境容器的调度部署, Kubernetes 集群仅用于部署模型提供推理服务, 后续可对其进行深入研究统一使用 Kubernetes 管理。

参考文献

- [1] 李慧.人工智能:改变世界的技术浪潮[J].信息安全与通信保密,2016(12):27-29.
- [2] Wu F, Lu C, Zhu M, et al. Towards a new generation of artificial intelligence in China[J]. Nature Machine Intelligence, 2020, 2(6): 312-316.
- [3] 2020 信息技术应用创新产业发展峰会召开[J].智能制造,2020(10):12-13.
- [4] 杨赞.AI 焕新 方兴未艾[N].人民邮电,2022-04-22(007).
- [5] Liu J, Chang H, Forrest J Y L, et al. Influence of artificial intelligence on technological innovation: Evidence from the panel data of china's manufacturing sectors[J]. Technological Forecasting and Social Change, 2020, 158: 120142.
- [6] 刘艳.三维坐标推动人工智能迈向发展新阶段[N].科技日报,2022-04-15(006).
- [7] FINAL E C J C. 2030 Digital Compass: The European Way for the Digital Decade [Z]. 2021.
- [8] Potdar A M, Narayan D G, Kengond S, et al. Performance evaluation of docker container and virtual machine[J]. Procedia Computer Science, 2020, 171: 1419-1428.
- [9] Shirinbab S, Lundberg L, Casalicchio E. Performance evaluation of containers and virtual machines when running Cassandra workload concurrently[J]. Concurrency and Computation: Practice and Experience, 2020, 32(17): e5693.
- [10] Rossi F, Cardellini V, Presti F L, et al. Geo-distributed efficient deployment of containers with Kubernetes[J]. Computer Communications, 2020, 159: 161-174.
- [11] Kosińska J, Zieliński K. Autonomic management framework for cloud-native applications[J]. Journal of Grid Computing, 2020, 18: 779-796.
- [12] 邻章.AI 原生云时代, 市场竞争天平又将倾向何方? [J].大数据时代,2021(08):6-12.
- [13] 邢文娟, 雷波, 赵倩颖. 算力基础设施发展现状与趋势展望[J]. 电信科学, 2022, 38(6): 51-61.
- [14] Simić M, Stojkov M, Sladić G, et al. On container usability in large-scale edge distributed system[J]. On container usability in large-scale edge distributed system, 2019, pp: 97-101.
- [15] 傅一平.容器简史: 从 1979 到现在[J].计算机与网络,2020,46(06):39-41.
- [16] 李瑜,赵勇,郭晓栋,刘国乐.全系统一体的访问控制保障模型[J].清华大学学报(自然科学版),2017,57(04):432-436.
- [17] Casalicchio E, Iannucci S. The state - of - the - art in container technologies: Application, orchestration and security[J]. Concurrency and Computation: Practice and Experience, 2020, 32(17): e5668.
- [18] Rizki R, Rakhmatsyah A, Nugroho M A. Performance analysis of container-based hadoop cluster: Openvz and lxc[C]. 2016 4th International Conference on Information and Communication

- Technology (ICoICT). IEEE, 2016: 1-4.
- [19] Pothecary R. Containers[M]. Running Microsoft Workloads on AWS: Active Directory, Databases, Development, and More. Berkeley, CA: Apress, 2021: 161-178.
- [20] 宋汉松. 容器网络技术研究 with 前景展望[J]. 金融电子化, 2019(12):88-89.
- [21] Hall A, Ramachandran U. Opportunities for Optimizing the Container Runtime[C]. 2022 IEEE/ACM 7th Symposium on Edge Computing (SEC). IEEE, 2022: 265-276.
- [22] 张耀方, 李源, 孙宏强. 容器技术在软件项目中的应用研究[J]. 电脑编程技巧与维护, 2021, (05):38-39.
- [23] Katti J, Agarwal J, Bharata S, et al. University Admission Prediction Using Google Vertex AI[C]//2022 First International Conference on Artificial Intelligence Trends and Pattern Recognition (ICAITPR). IEEE, 2022: 1-5.
- [24] 崔德龙, 夏曼. 虚拟化技术在航空计算领域的应用[J]. 航空工程进展, 2022, 13(2): 71-77.
- [25] David Bernstein. Containers and Cloud: From LXC to Docker to Kubernetes.[J]. IEEE Cloud Computing, 2014, 1(3): 81-84.
- [26] 蒋筱斌, 熊轶翔, 张珩, 等. 基于 Kubernetes 的 RISC-V 异构集群云任务调度系统[J]. 计算机系统应用, 2022, 31(9): 3-14.
- [27] 何震苇, 黄丹池, 严丽云, 等. 基于 Kubernetes 的融合云原生基础设施方案与关键技术[J]. 电信科学, 2020, 36(12): 77-88.
- [28] Khatami A A, Purwanto Y, Ruriawan M F. High availability storage server with kubernetes[C]. 2020 International Conference on Information Technology Systems and Innovation (ICITSI). IEEE, 2020: 74-78.
- [29] Santos J, Wauters T, Volckaert B, et al. Towards network-aware resource provisioning in kubernetes for fog computing applications[C]. 2019 IEEE Conference on Network Softwarization (NetSoft). IEEE, 2019: 351-359.
- [30] 张可颖, 彭丽苹, 吕晓丹, 吕尚青. 开源云上的 Kubernetes 弹性调度[J]. 计算机技术与发展, 2019, 29(02):109-114.
- [31] 王家柱, 范中磊, 毕强, 等. 对象存储系统中基于监控的动态负载均衡方法[J]. 微电子学与计算机, 2022, 39(12): 69-76.
- [32] Shen D, Dumas C, Sardina C, et al. Cholulu: Fast, Fault-tolerant, and Strongly Consistent Off-chain Object Storage[C]. 2022 Fourth International Conference on Blockchain Computing and Applications (BCCA). IEEE, 2022: 189-194.
- [33] Levin A, Garion S, Kolodner E K, et al. AIOps for a cloud object storage service[C]. 2019 IEEE International Congress on Big Data (BigDataCongress). IEEE, 2019: 165-169.
- [34] Aghayev A, Weil S, Kuchnik M, et al. File systems unfit as distributed storage backends: lessons

- from 10 years of Ceph evolution[C]. Proceedings of the 27th ACM Symposium on Operating Systems Principles. 2019: 353-369.
- [35] Jeong K, Duffy C, Kim J S, et al. Optimizing the Ceph distributed file system for high performance computing[C]. 2019 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP). IEEE, 2019: 446-451.
- [36] Nurdin J B, Mulyana E. NextCeph: Nextcloud Platform Based Application for Ceph Cluster Management[C]. 2019 IEEE 13th International Conference on Telecommunication Systems, Services, and Applications (TSSA). IEEE, 2019: 25-30.
- [37] Chen H M, Li C J, Ke B S. Designing A Simple Storage Services (S3) Compatible System Based on Ceph Software-Defined Storage System[C]. Proceedings of the 2017 2nd International Conference on Multimedia Systems and Signal Processing. 2017: 6-10.
- [38] Li Z, Wang Y. An adaptive read/write optimized algorithm for Ceph heterogeneous systems via performance prediction and multi-attribute decision making[J]. Cluster Computing, 2023, 26(2): 1125-1146.
- [39] 张高毓,韩琮师,马玉玺.基于 Keycloak 的生产平台安全策略方案[J].信息技术与信息化,2022,No.262(01):131-134.
- [40] Divyabharathi D N, Cholli N G. A review on identity and access management server (keycloak)[J]. International Journal of Security and Privacy in Pervasive Computing (IJSPPC), 2020, 12(3): 46-53.
- [41] Sherazi S N A, Zahoor E, Akhtar S, et al. A blockchain based approach for the authorization policies delegation in emergency situations[J]. Transactions on Emerging Telecommunications Technologies, 2022, 33(5): e4461.
- [42] Gall Michael,Pigni Federico. Taking DevOps mainstream: a critical review and conceptual framework[J]. European Journal of Information Systems, 2022, 31(5): 548-567.
- [43] 刘希平,李文,王刚,赵明明.基于 WireGuard 的高性能虚拟专用网络架构设计与实现[J].网络安全技术与应用,2021(02):17-19.
- [44] 温馨,樊婧雯,王富强.基于 OpenResty 平台的 API 网关系统的设计与实现[J].信息化研究,2020,46(03):62-68.
- [45] 曹弯弯,马永,程航.基于 bind9 软件的 DNS 服务归集的设计与实现[J].机电信息,2019,(32):158-159.
- [46] Chahoud M, Otoum S, Mourad A. On the feasibility of Federated Learning towards on-demand client deployment at the edge[J]. Information Processing & Management, 2023, 60(1): 103150.
- [47] Puccini M, Mariano A. CEPH FILESYSTEM IN THE ENEAGRID INFRASTRUCTURE[J]. High Performance Computing on CRESCO Infrastructure: research activity and results 2020, 2021: 135.

- [48] 杨柳.基于 JMeter 的网站性能测试研究[J].信息记录材料,2022,23(10):204-206.